

7.

Sinteza visokog nivoa

Pod sintezom visokog nivoa (*High Level Synthesis, HLS*) podrazumeva se proces automatskog generisanja RTL modela digitalnog elektronskog sistema na osnovu njegove algoritamske specifikacije. Algoritamska specifikacija željenog ponašanja digitalnog sistema najčešće se zadaje u vidu modela napisanog u nekom od jezika visokog nivoa, kao što su C, C++, *SystemC*, *OpenCL*. Sa druge strane, automatski generisani RTL model digitalnog elektronskog sistema najčešće je opisan korišćenjem nekog od standardnih jezika za modelovanje hardvera, kao što su VHDL, Verilog ili SystemVerilog.

Sinteza visokog nivoa predstavlja poslednji korak u podizanju nivoa apstrakcije modela digitalnih elektronskih sistema koji se automatski mogu prevesti od odgovarajuću hardversku implementaciju. Korišćenjem HLS alata moguće je željeno ponašanje digitalnog sistema opisati na algoritamskom (bihevijoralnom) nivou. Podizanjem nivoa apstrakcije modela ujedno se i pojednostavljuje sam model, pa se na taj način i skraćuje vreme potrebno za njegov razvoj i smanjuje verovatnoća nenamernog unošenja grešaka u model.

Sinteza visokog nivoa sastoji se iz sledećih faza:

- **Faze vremenskog planiranja (*Scheduling*)** – u ovoj fazi određuje se kada će (u kom taktu globalnog sinhronizacionog signala) svaka do operacija prisutnih u algoritamskom modelu biti izvršena. Vremensko planiranje vrši se na osnovu:
 - Informacije o potrebnoj periodi, odnosno učestanosti, globalnog sinhronizacionog signala

- Vremenu koje je neophodno da se svaka od operacija kompletira, što zavisi od odabrane ciljne komponente (FPGA familije ili tehnološkog procesa ASIC fabrikacije) na kojoj će digitalni sistem koji se projektuje biti realizovan
- Skupu optimizacionih direktiva definisanih od strane dizajnera sistema
- **Alokacije hardverskih resursa** (*Resource Allocation and Binding*) – u ovoj fazi određuje se na kojem od raspoloživih hardverskih resursa će biti izvršena svaka od operacija prisutnih u algoritamskom modelu. Na osnovu ovog mapiranja moguće je automatski kreirati podsistem za obradu podataka (*datapath*).
- **Ekstrakcije upravljačke logike** – na osnovu planiranog redosleda izvršavanja operacija iz algoritamskog modela sistema, kao i informacija o tome na kojim hardverskim resursima će svaka od operacija biti izvršena, vrši se ekstrakcija upravljačke logike koja se zatim implementira kao konačni automat koji zapravo predstavlja upravljačku jedinicu (*control path*) projektovanog sistema.

Čitav opisani proces odvija se potpuno automatski, unutar alata za sintezu visokog nivoa. Imajući ovo u vidu, pomoću HLS alata moguće je relativno brzo generisati različite hardverske implementacije, modifikujući optimizacione direktive HLS alata. Na ovaj način hardverski dizajner može relativno efikasno vršiti pretragu prostora mogućih implementacije (*Design Space Exploration*), u potrazi za optimalnom implementacijom.

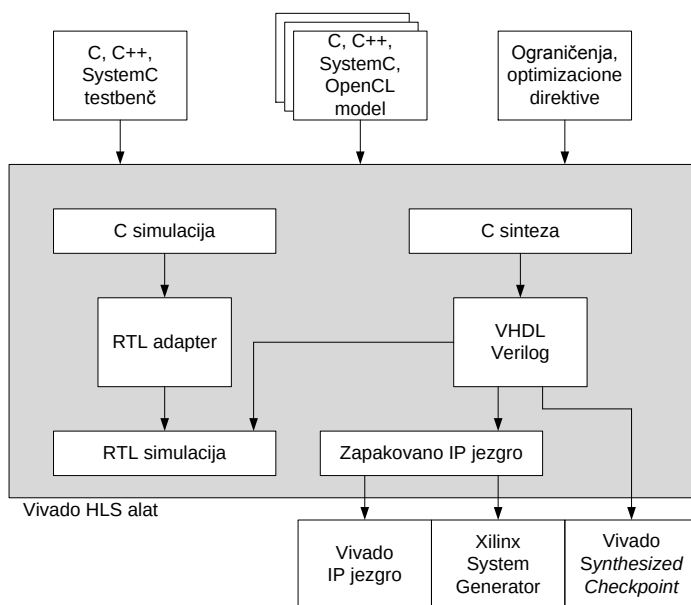
7.1. Vivado HLS alat za sintezu visokog nivoa

Vivado HLS alat kompanije Xilinx omogućava automatsku sintezu algoritamskog modela digitalnog sistema, opisanog korišćenjem standardnih jezika visokog nivoa, u odgovarajuće IP jezgro, koje se zatim može jednostavno integrisati u složenije sisteme.

Na slici 7.1 prikazan je tok projektovanja digitalnog sistema korišćenjem Vivado HLS alata. Vivado HLS kao svoje ulaze koristi sledeće izvorne fajlove:

- Algoritamski model digitalnog sistema koji se projektuje, napisan korišćenjem C, C++, SystemC ili OpenCL jezika – algoritamski model sistema predstavlja primarni ulaz Vivado HLS alata. Algoritamski model mora biti napisan u okviru jedne C funkcije, koja može a ne mora da unutar sebe zadrži čitavu hijerarhiju pod-funkcija.
- Ograničenja – sledeća ograničenja uvek moraju biti definisana, kako bi Vivado HLS alat mogao izvršiti potrebne korake prilikom automatske sinteze hardvera:

- željena perioda klok signala na kojoj sistem koji se razvija treba da radi,
 - maksimalno odstupanje od željene periode koje se može tolerisati (*clock uncertainty*),
 - FPGA familija koja će biti korišćena za realizaciju projektovanog sistema
- Optimizacione direktive – direktive predstavljaju opcioni ulaz u Vivado HLS alat i ne moraju se uvek definisati. Pomoću direktiva dizajner može da upravlja procesom automatske sinteze hardvera kako bi sintetizovani sistem imao željene karakteristike (željenu brzinu izvršavanja algoritma, željeni propusni opseg, željenu potrošnju hardverskih resursa, itd.).
- Testbenč fajl i njemu asocirane fajlove – Vivado HSL koristi testbenč, napisan u C, C++, SystemC ili OpenCV jeziku, kako bi simulirao rad algoritamskog modela sistema pre HLS sinteze (korišćenjem C simulacije), kao i rad automatski generisanog HDL modela na RTL nivou (korišćenjem C/RTL kosimulacije) nakon HLS sinteze.



Slika 7.1. Tok projektovanja digitalnih sistema pomoću Vivado HLS alata

Izlazni fajlovi, koji se generišu prilikom rada Vivado HLS alata su:

- HLD model na RTL nivou ulaznog algoritamskog modela, koji je nastao kao rezultat sinteze visokog nivoa – ovaj skup HDL fajlova čini primarni izlaz

Vivado HLS alata i predstavljaju rezultat sprovođenja koraka sinteze visokog nivoa opisanih ranije (vremensko planiranje, alokacija resursa, ekstrakcija upravljačke logike). Korišćenjem Vivado HLS alata moguće je automatski generisati odgovarajući HDL model opisan u VHDL (IEEE 1076-2000 verzija standarda) ili Verilog (IEEE 1364-2001 verzija standarda) jeziku. Korišćenjem ovog RTL modela moguće je, uz pomoć Vivado Design Suite alata opisanog u glavi 1, izvršiti automatsku sintezu, implementaciju i generisanje potrebnog konfiguracionog fajla, na taj način implementirajući digitalni sistem sa želejnom funkcionalnošću na odabranoj FPGA komponenti. Vivado HLS pakuje sve automatski generisane HDL fajlove u formi IP jezgra, u suštini sprovodeći postupak generisanja IP jezgra opisan u glavi 6 automatski. Ovako zapakovano IP jezgro se zatim može vrlo lako koristiti unutar drugih Xilinx razvojnih alata, kao što su Vivado Design Suite ili System Generator alat za MATLAB.

- Različite izveštaje – ove izveštaje čine odgovarajući tekstualni fajlovi generisani prilikom HLS sinteze, C simulacije, C/RTL kosimulacije i procesa pakovanja IP jezgra. Na osnovu njihovog sadržaja dizajner može da prati tok rada Vivado HLS alata, analizira postignute rezultate i eventualno izvrši potrebne korekcije u cilju postizanja zahtevanih performansi sistema koji se projektuje.

Unutar svakog C/C++ programa funkcija na najvišem nivou hijerarhije je *main()* funkcija (u slučaju SystemC jezika to je *sc_main()* funkcija). Unutar Vivado HLS alata *main()* funkcija se **ne može** sintetizovati. Razlog zbog čega je to tako biće diskutovan nešto kasnije. Bilo koja funkcija koja se nalazi unutar *main()* funkcije može se označiti kao funkcija na najvišem nivou hijerarhije, od koje treba da započne sinteza visokog nivoa. Pored ovog pravila Vivado HLS nameće i sledeća pravila:

- Samo jedna funkcija se može označiti kao funkcija na najvišem nivou hijerarhije, od koje treba da započne sinteza visokog nivoa.
- Sve pod-funkcije, koje se nalaze unutar označene funkcije za sintezu, takođe će biti sintetizovane.
- Ukoliko je neophodno sintetizovati veći broj funkcija, koje se ne nalaze unutar jedne zajedničke funkcije, one se prethodno moraju objединiti u jednu, zajedničku funkciju, koja se zatim mora označiti kao funkcija od koje postupak sinteze visokog nivoa treba da započne.

Podrška verifikaciji na algoritamskom nivou (C simulacija)

Prilikom korišćenja Vivado HLS alata, bilo bi neefikasno da se verifikacija željene funkcionalnosti, modelovane unutar algoritamskog modela, vrši HDL verifikacijom automatski sintetizovanog RTL modela. U cilju povećanja produktivnosti, Vivado HLS alat podržava proces verifikacije na algoritamskom nivou, korišćenjem verifikacionih okruženja razvijenih pomoću C, C++ i SystemC jezika. Na ovaj način, pre same sinteze

visokog nivoa, dizajner se može uveriti u funkcionalnu korektnost ulaznog algoritamskog modela. Unutar Vivado HLS alata C/C++ *main()* funkcija koristi se za definisanje verifikacionog okruženja. Sada je jasno zašto se *main()* funkcija ne može označiti kao funkcija od koje je potrebno započeti proces sinteze visokog nivoa. Unutar *main()* funkcije potrebno je uključiti i funkciju koja predstavlja predmet HLS sinteze visokog nivoa (ona će zapravo predstavljati instancu modula koji se verifikuje, DUV ili DUT), ali i sve ostale funkcije pomoću kojih su modelovane standardne komponente savremenog verifikacionog okruženja (stimulus generator, čeker, monitor, itd.). Ove funkcije neće biti sintetizovane pomoću HLS alata, njihova isključiva namena je da omoguće funkcionalnu verifikaciju jedne, specifične funkcije koja je namenjena sintezi visokog nivoa.

Vivado HLS podržava najveći broj jezičkih konstrukcija, definisanih unutar C, C++ i SystemC jezika, kao i sve osnovne tipove, uključujući i *float* i *double* tipove podataka.

Ipak, Vivado HLS ne može da sintetizuje neke jezičke konstrukcije kao na primer:

- **Dinamičku alokaciju objekata** – kako FPGA komponenta sadrži konačnu i nepromenljivu količinu hardverskih resursa, dinamička alokacija i oslobađanje objekata nije podržano.
- **Pozive operativnog sistema** – Svi podaci koji teku ka i od FPGA komponente moraju da budu prosleđeni kroz odgovarajuće ulazne i izlazne portove. Operacije operativnog sistema, na primer operacije sa datotekama ili upiti o tekućem vremenu i datumu, nisu podržane od strane Vivado HLS alata prilikom sinteze visokog nivoa. Ove operacije se međutim mogu koristiti prilikom verifikacije modelovane funkcionalnosti unutar testbenča i njihovi rezultati mogu biti prosleđeni kao ulazni argumenti funkcije koja je označena za sintezu visokog nivoa.

Podrška predefinisanim bibliotekama funkcija

Vivado HLS alat dolazi sa podrškom većeg broja C biblioteka, unutar kojih se nalaze deklaracije funkcija i tipova podatka koji su optimizovani za hardversku sintezu. Korišćenjem ovih biblioteka može se obezbediti visok nivo kvaliteta krajnjih rezultata, odnosno može se projektovati digitalni elektronski sistem visokih performansi sa optimalnim korišćenjem raspoloživih hardverskih resursa. Kako su biblioteke dostupne u formi C, C++, SystemC ili OpenCL biblioteka, vrlo lako se mogu integrisati unutar funkcije koja je označena za sintezu visokog nivoa.

Vivado HLS trenutno nudi sledeće C/C++ biblioteke, pomoću kojih je moguće proširiti mogućnosti standardnih C/C++ jezika:

- „*Arbitrary Precision Data Types*“ biblioteku (*ap_cint.h*), koja definiše tipove podataka sa proizvoljnom preciznošću, za reprezentaciju numeričkih vrednosti u formatu fiksne tačke proizvoljnog formata, kao i 16-bitne (*half-precision*) podatke za reprezentaciju numeričkih vrednosti u formatu pokretne tačke.

- „HLS Math“ biblioteku (*hls_math.h*), koja omogućava automatsku sintezu standardnih *math.h* i *cmath.h* C, odnosno C++ biblioteka.
- „HLS Video“ biblioteku (*hls_video.h*), u kojoj su definisane različite funkcije obrade video signala.
- „HLS IP“ biblioteke (*hls_fft.h*, *hls_fir.h*, *hls_dds.h*, *ap_shift_reg.h*), u kojoj su definisane funkcije koje se direktno mapiraju u odgovarajuća IP jezgra kompanije Xilinx, kao što su brza Furijeova transformacija (FFT), digitalni filtri sa konačnim impulsnim odzivom (FIR filtri), itd.
- „HLS Linear Algebra“ biblioteku (*hls_linear_algebra.h*), u kojoj su definisane različite funkcije koje omogućavaju hardversku implementaciju osnovnih operacija linearne algebre.
- „HLS DSP“ biblioteku (*hls_dsp.h*), u kojoj se nalaze funkcije koje omogućavaju hardversku implementaciju osnovnih gradivnih blokova koji se koriste u sistemima za digitalnu obradu signala.

Sinteza, optimizacija i analiza

Vivado HLS je projektno orijentisan alat, kao i Vivado Design Suite alat opisan u glavi 1. Svaki projekat sadrži jedan skup izvornih fajlova (C, C++, SystemC) i može da sadrži veći broj različitih implementacija (unutar Vivado HLS alata ove različite implementacije nazivaju se *Solutions*). Svaka implementacija može da ima svoj skup ograničenja i optimizacionih direktiva. Unutar Vivado HLS alata moguće je međusobno upoređivati karakteristike većeg broja različitih implementacija, u cilju utvrđivanja koje optimizacije dizajna rezultuju u najboljoj implementaciji.

Tipičan tok korišćenja Vivado HLS alata sastoji se u sprovođenju sledećih koraka:

1. Kreiranje projekta sa potrebnim izvornim fajlovima koji uključuju i početni skup optimizacionih direktiva
2. Simulaciju algoritamskog modela kako bi se utvrdila funkcionalna korektnost algoritamskog modela
3. Sintezu visokog nivoa kako bi se generisala početna implementacija
4. Analizu karaktetistika početne implementacije

Nakon što se analiziraju karakteristike početne implementacije, ukoliko one ne zadovoljavaju postavljene zahteve, može se kreirati nova implementacija sa modifikovanim skupom ograničenja i optimizacionih direktiva. Ovaj postupak se ponavlja sve dok se ne generiše implementacija koja ispunjava zahteve u pogledu performansi i broja potrebnih hardverskih resursa.

Proces profinjenja implementacije unutar Vivado HLS alata naziva se fazom *optimizacije dizajna*. Vivado HLS pruža nam širok izbor optimizacionih direktiva, koje uključuju:

- Zahtev da se odabrani postupak obrade izvrši primenom tehnike protočne obrade (*pipelining*), koja omogućava da se započne sa obradom narednog podatka pre nego što se završi obrada tekućeg.
- Definisane vremenske intervale u okviru kojega se mora završiti izvršavanje funkcija, petlji i odgovarajućih regiona unutar algoritamskog modela sistema.
- Definisane gornjeg limita u broju raspoloživih hardverskih resursa koji se smeju koristiti unutar hardverske implementacije algoritamskog modela sistema.
- Definisane načina na koji će biti sintetizovane petlje unutar algoritamskog modela.
- Izbor ulazno/izlaznog protokola za svaki od ulaznih i izlaznih argumenata funkcije koja se sintetizuje, u cilju postizanja zahtevanih performansi i obezbeđivanja lakog i jednostavnog povezivanja sa okruženjem u kojem će se projektovani digitalni sistem koristiti.

Optimizacione direktive se zadaju vrlo jednostavno, korišćenjem Vivado HSL GUI alata, ili se alternativno mogu specificirati pomoću odgovarajućih Tcl komandi. O ovome će biti više reči kasnije.

Kada se završi postupak sinteze visokog nivoa algoritamskog modela sistema, Vivado HLS automatski generiše odgovarajući izveštaj na osnovu kojega dizajner može da proceni performanse i zahtevane hardverske resurse tekuće implementacije digitalnog sistema. Unutar Vivado HSL alata postoje dodatni alati koji omogućavaju detaljniju analizu karakteristike generisane implementacije o kojim će biti reči kasnije.

Podrška verifikaciji na RTL nivou (C/RTL kosimulacija)

Vivado HLS alat pored verifikacije na algoritamskom nivou, omogućava da se sprovede i verifikacija automatski generisanog RTL modela sistema. Na ovaj način može se utvrditi da li je generisani RTL model (VHDL ili Verilog) funkcionalno identičan sa algoritamskim modelom (C, C++ ili SystemC), kao i da se proveriti da li su korektno implementirani ulazno/izlazni interfejsi na portovima čitavog sistema. Za izvođenje ove verifikacije na RTL nivou nije potrebno razvijati novo verifikaciono okruženje (u VHDL ili Verilog jeziku), već se za ovu svrhu može iskoristiti verifikaciono okruženje razvijeno za potrebe verifikacije na algoritamskom nivou (u C, C++ ili SystemC jeziku)! Vivado HLS automatski generiše potrebnu infrastrukturu za izvršavanje C/RTL kosimulacije i automatski izvršava simulaciju korišćenjem nekog od standardnih RTL simulatora (na primer Vivado Xsim simulator, ModelSim simulator, VCS, NCSim, Riviera, itd.).

Eksportovanje automatski generisanog RTL modela

Na kraju, kada je generisana hardverska implementacija algoritamskog modela sistema sa željenim karakteristikama, Vivado HLS omogućava pakovanje RTL HDL modela sistema u formi IP jezgra u jednom od sledećih formata:

- Vivado IP jezgro – ovo je standardni format koji se koristi unutar Vivado IP Catalog-a i koji omogućava jednostavnu integraciju IP jezgra unutar Vivado Design Suite alata.
- System Generator blok – ovaj format omogućava jednostavnu integraciju RTL modela unutar MATLAB System Generator alata.
- Sintetizovanog checkpointa (checkpoint) – ovaj format omogućava da se RTL model sistema direktno uveze u Vivado Design Suite alat (bez potrebe da se pakuje u formu IP jezgra) i da se u Vivado Design Suite alatu nastavi postupak njegove implementacije pomoću odabrane FPGA komponente. Ovo je preporučeni način eksportovanja generisanog RTL modela u slučaju kada će se on koristiti samo unutar jednog projekta.

7.2. C algoritamski model „Naive“ množača matrica

U nastavku će biti ilustrovan postupak korišćenja Vivado HLS alata na primeru HLS sinteze algoritamskog modela „Naive“ množača dve matrice, originalno analiziranog u primeru 4.3. Kao što smo ranije već rekli, Vivado HLS alat u stanju je da automatski generiše RTL model digitalnog sistema, na osnovu specificiranog algoritamskog modela. U našem primeru, algoritamski model „Naive“ množača matrica biće razvijen korišćenjem modifikovanog „Naive“ algoritma za množenje dve matrice, prikazanog na slici 7.2.

```
for (i = 0; i < n; i++)
    for (j = 0; j < p; j++)
    {
        temp = 0;
        for (k = 0; k < m; k++)
            temp = temp + a(i,k)*b(k,j);
        c(i,j) = temp;
    }
return c;
```

Slika 7.2. „Naive“ algoritam za množenje dve matrice

7.2.1. Razvoj algoritamskog modela „Naive“ množača matrica u C programskom jeziku

„Naive“ algoritam za množenje matrica sa slike 7.2 napisan je u pseudo-jeziku i kao takav ne može predstavljati ulazni izvorni fajl za Vivado HLS alat. Kao što smo rekli ranije, Vivado HLS očekuje algoritamski model sistema napisan korišćenjem C, C++, SystemC ili OpenCL jezika. U našem primeru koristićemo standardni C++ programski jezik za formiranje algoritamskog modela „Naive“ množača matrica. Jedan mogući opis prikazan je na slici 7.3.

```
#include "ap_int.h"
#include "matrix_mult.h"

void matrix_mult(
    ap_uint<3> N, ap_uint<3> M, ap_uint<3> P,
    ap_uint<8> a[MAX_SIZE][MAX_SIZE],
    ap_uint<8> b[MAX_SIZE][MAX_SIZE],
    ap_uint<23> c[MAX_SIZE][MAX_SIZE])
{
    ap_uint<23> temp;

    i: for (int i = 0; i < N; i++)
        j: for (int j = 0; j < P; j++)
        {
            temp = 0;
            k: for (int k = 0; k < M; k++)
                temp += a[i][k] * b[k][j];
            c[i][j] = temp;
        }
}
```

Slika 7.3. Algoritamski model „Naive“ množača matrica, napisan u C++ jeziku

Kao što se može videti sa slike 7.3, algoritamski model „Naive“ množača matrica nalazi se unutar funkcije *matrix_multiply*. Ovo je neophodno jer, kako je ranije već bilo rečeno, Vivado HLS očekuje da se algoritamski model čitavog sistema koji se projektuje nalazi unutar jedne funkcije. Unutar funkcije mogu se pozivati druge funkcije, kreirajući na taj način hijerarhiju unutar budućeg sistema, ali može postojati samo jedna funkcija koja se nalazi na najvišem nivou hijerarhije i od koje započinje sinteza visokog nivoa. U našem slučaju to će biti funkcija *matrix_multiply*. Algoritamski model sistema nije mogao biti smešten u *main()* funkciju, jer se ona koristi za definisanje testbenča.

Funkcija *matrix_multiply* ima ukupno šest argumenata, skalare N , M i P , kao i matrice $a[N][M]$, $b[M][P]$ i $c[N][P]$. Ovi argumenti biće zamenjene odgovarajućim ulaznim i izlaznim interfejsima u procesu sinteze visokog nivoa. Na osnovu načina na koji se koriste ovi argumenti unutar algoritamskog modela, Vivado HLS će odlučiti da li je reč o ulaznim, izlaznim ili ulazno/izlaznim argumentima. U našem primeru argumenti N , M , P , a i b su ulazni argumenti, dok je argument c izlazni.

Sam algoritamski model „Naive“ množača matrica (telo funkcije *matrix_multiply*), blisko prati „Naive“ algoritam prikazan na slici 7.2, ali je ovoga puta on napisan uz strogo poštovanje sintakse C++ programskog jezika. Obratite pažnju da su sve petlje u modelu sa slike 7.3 označene odgovarajućim labelama. Ovo je preporučljiv stil, jer će u znatnoj meri olakšati definisanje optimizacionih direktiva unutar Vivado HLS alata.

Algoritamski model „Naive“ množača matrica sa slike 7.3 koristi tip *ap_uint* iz biblioteke *ap_int.h* pomoću kojega se može specificirati broj bita koji treba da bude korišćen za reprezentaciju neoznačenih celih brojeva. U modelu sa slike 7.3 ulazni argumenti *N*, *M* i *P* deklarirani su kao 3-bitni, elementi matrica *a* i *b* reprezentovani su sa 8 bita, dok su elementi matrice *c* reprezentovani sa 23 bita. Ovaj izbor u skladu je sa ograničenjima koja su korišćena ranije, prilikom projektovanja „Naive“ množača matrica korišćenjem RT metodologije, u glavi 4.

Unutar algoritamskog modela koristi se i konstanta *MAX_SIZE*, koja predstavlja maksimalni broj elemenata po bilo kojoj dimenziji matrice, a nije deklarirana unutar samog modela. Ona je zapravo deklarirana u *matrix_mult.h* zaglavlju, čija struktura je prikazana na slici 7.4.

```
#ifndef __MATRIXMULT_H__
#define __MATRIXMULT_H__

#include "ap_int.h"
#include <cmath>
using namespace std;

// Compare TB vs HW C-model and/or RTL
#define HW_COSIM

// Maximum number of elements in any row of any matrix
#define MAX_SIZE 8

// Prototype of top level function for C-synthesis
void matrix_mult(
    ap_uint<3> N, ap_uint<3> M, ap_uint<3> P,
    ap_uint<8> a[MAX_SIZE][MAX_SIZE],
    ap_uint<8> b[MAX_SIZE][MAX_SIZE],
    ap_uint<23> c[MAX_SIZE][MAX_SIZE]);
```

Slika 7.4. Zaglavlje algoritamskog modela „Naive“ množača matrica

Prethodna dva izvorna fajla čine kompletni algoritamski model „Naive“ množača matrica.

7.2.2. Razvoj verifikacionog okruženja „Naive“ množača matrica u C programskom jeziku

Kao što ranije već bilo rečeno, Vivado HLS omogućava da se i funkcionalna verifikacija modela sistema koji se projektuje vrši već i na algoritamskom nivou, razvijajući odgovarajući testbenč, napisan u C, C++ ili SystemC programskom jeziku. U slučaju „Naive“ množača matrica, jedan mogući testbenč prikazan je u nastavku.

```
#include <iostream>
#include "ap_int.h"
#include "matrix_mult.h"

using namespace std;

const ap_uint<3> N = 3;
const ap_uint<3> M = 3;
const ap_uint<3> P = 3;

int main(int argc, char **argv)
{
    ap_uint<8> in_mat_a[MAX_SIZE][MAX_SIZE] = {
        {1, 2, 3},
        {4, 5, 6},
        {7, 8, 9}};

    ap_uint<8> in_mat_b[MAX_SIZE][MAX_SIZE] = {
        {10, 11, 12},
        {13, 14, 15},
        {16, 17, 18}};

    ap_uint<23> hw_result[MAX_SIZE][MAX_SIZE];
    ap_uint<23> exp_result[MAX_SIZE][MAX_SIZE];
    int err_cnt = 0;

    // Generate the expected result
    // Iterate over the rows of the A matrix
    for (int i = 0; i < N; i++)
        for (int j = 0; j < P; j++) {
            // Iterate over the columns of the B matrix
            exp_result[i][j] = 0;
            // Do the inner product of a row of A and col of B
            for (int k = 0; k < M; k++)
                exp_result[i][j] += in_mat_a[i][k] * in_mat_b[k][j];
        }

#ifdef HW_COSIM
    // Run the AutoESL matrix multiply block
    matrix_mult(N, M, P, in_mat_a, in_mat_b, hw_result);
#endif

    // Print result matrix
    cout << "Resulting matrix:\n";
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < P; j++) {
            // Check HW result against SW
            if (hw_result[i][j] != exp_result[i][j])
```

```

        err_cnt++;
        cout << " " << hw_result[i][j];
    }
    cout << "\n";
}
cout << "\n";

if (err_cnt)
    cout << "ERROR: " << err_cnt << " mismatches detected!" << endl;
else
    cout << "Test passes." << endl;

return err_cnt;
}

```

Slika 7.5. Testbenč za „Naive“ množač matrica, napisan u C++ jeziku

C++ testbenč za „Naive“ množač matrica na početku deklarise i inicijalizuje dve matrice, *in_mat_a* i *in_mat_b*, koje će zapravo biti ulazni stimulus. Dimenzije ovih matrica su fiksirane na 3x3, tako da će i rezultujuća matrica koja će predstavljati proizvod ove dve matrice takođe biti dimenzija 3x3. Nakon deklaracije i inicijalizacije svih potrebnih objekata, izračunava se očekivani izlaz na ovako definisan stimulus. Unutar tri ugnježdene petlje, koristeći vrednosti elemenata matrica *in_mat_a* i *in_mat_b* izračunavaju se vrednosti matrice *exp_result*, koja predstavlja njihov proizvod, a ujedno prestavlja i očekivani odziv sistema čija funkcionalnost se verifikuje. Nakon toga se poziva *matrix_mult* funkcija, unutar koje se zapravo nalazi algoritamski model „Naive“ množača matrica, kojoj se prosleđuju iste dve matrice *in_mat_a* i *in_mat_b*, a rezultat izvršavanja *matrix_mult* funkcije smešta se u matricu *hw_result*. Poslednji korak predstavlja upoređivanje očekivanih vrednosti (elementi matrice *exp_result*), sa vrednostima izračunati pomoću funkcije *matrix_mult* (elementi matrice *hw_result*). Svako razlikovanje vrednosti elemenata ove dve matrice rezultuje u uvećanju vrednosti brojača grešaka, *err_cnt*. Prilikom upoređivanja očekivanih i izračunatih vrednosti, na standardni izlaz se prosleđuju vrednosti *hw_result* matrice.

Ovim je postupak kreiranja svih potrebnih izvornih fajlova u slučaju projektovanja „Naive“ množača matrica korišćenjem Vivado HLS alata završen.

7.2. Kreiranje Vivado HLS projekta

Vivado HLS alat pokreće se dvoklikom na ikonicu sa desktopa ili selektovanjem opcije Start → All Programs → Xilinx Design Tools → Vivado 2016.2 → Vivado HLS → Vivado HLS 2016.2 iz *Start* menija. Nakon pokretanja Vivado HLS alata pojaviće se glavni prozor, prikazan na slici 7.6.



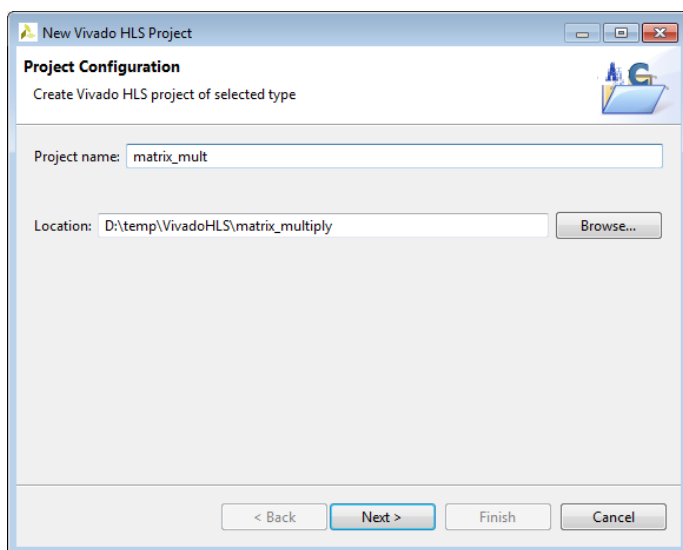
Slika 7.6. Izgled glavnog prozora Vivado HLS alata

U ovom trenutku na raspolaganju nam stoji nekoliko mogućnosti:

- možemo kreirati novi projekat, pritiskom na dugme „*Create New Project...*“ u gornjem levom uglu prozora,
- možemo otvoriti neki projekat koji smo ranije kreirali, pritiskom na dugme „*Open Project...*“,
- možemo otvoriti neki od primera, u vidu kompletnih projekata, koji mogu biti vrlo zgodni za upoznavanje sa alatom, pritiskom na dugme „*Open Example Project*“,
- možemo otvoriti dokument sa primerima koji detaljnije prikazuju različite mogućnosti i načine korišćenja Vivado HLS alata, „*Vivado Design Suite Tutorial: High-Level Synthesis (UG871)*“, pritiskom na dugme „*Tutorials*“,
- možemo otvoriti glavno uputstvo za Vivado HLS alat, „*Vivado Design Suite User Guide: High-Level Synthesis (UG902)*“, pritiskom na dugme „*User Guide*“,
- možemo pristupiti informacijama o novim mogućnostima Vivado HLS alata, koje su postale dostupne u poslednjoj verziji alata, kao i informacijama u vezi sa načinom instaliranja Vivado HLS alata, pritiskom na dugme „*Release Notes Guide*“,

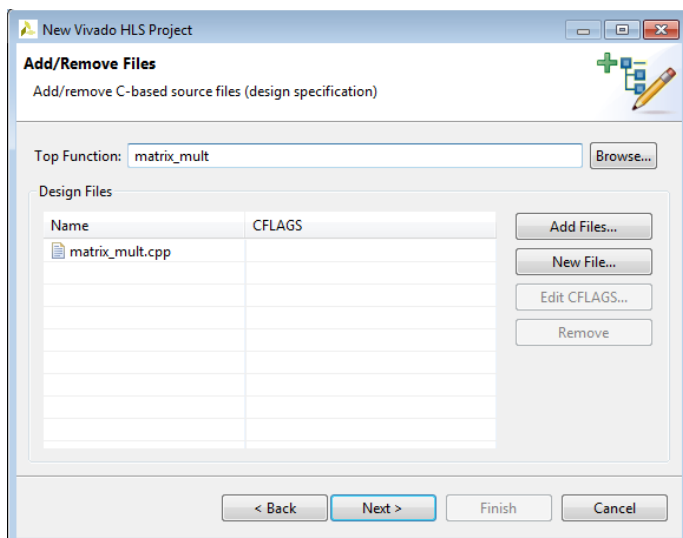
- možemo odabrati neki od projekata na kojima smo nedavno radili, u okviru prozora „Recent projects“.

Pošto mi započinjemo rad na novom projektu, odabraćemo opciju „Create New Project...“ (isti efekat postiže se i ako selektujemo *File/New Project* opciju iz menija) kako bismo definisali karakteristike novog projekta. Nakon pritiska na dugme „Create New Project...“ pojaviće se novi prozor, „Project Configuration“, prikazan na slici 7.7, u kojem započinjemo proces kreiranja novog projekta.



Slika 7.7. „New Project – Project Configuration“ prozor sa potrebnim podešavanjima

Prilikom kreiranja novog projekta prvi korak je da se definiše njegovo ime (polje *Project name* sa slike 7.7) i mesto na disku gde će biti smešteni svi projektni fajlovi (polje *Location*). U našem slučaju ime projekta će biti *matrix_mult*, lokacija na disku može se izabrati proizvoljno, uz napomenu da putanja do odabrane lokacije **ne sme** sadržati *Space* karaktere. Pritiskom na dugme *Next* prelazimo na sledeći prozor podešavanja projekta, „Add/Remove Files Source Files“, prikazan na slici 7.8.



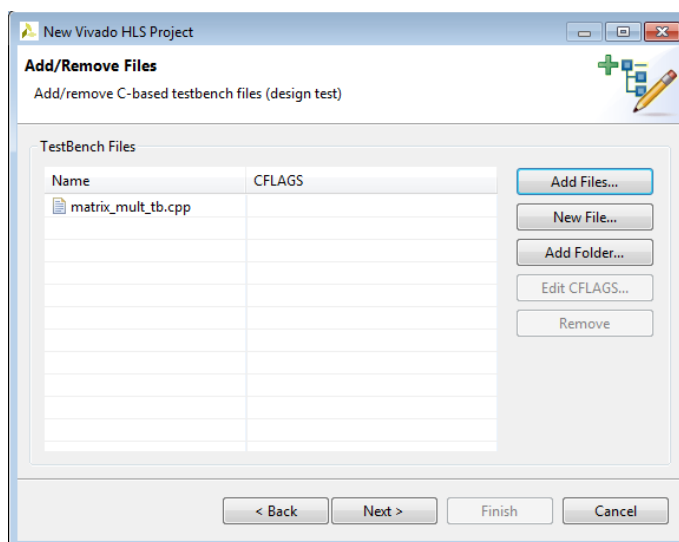
Slika 7.8. „New Project – Add/Remove Files Source Files“ prozor sa potrebnim podešavanjima

U ovom trenutku potrebno je specificirati koji sve izvorni dizajn fajlovi čine projekat. Potrebno je specificirati sve izvorne dizajn fajlove u kojima se nalazi algoritamski model sistema koji se projektuje. Dodavanje izvornih dizajn fajlova postiže se pritiskom na „Add Files...“ dugme, ukoliko fajlovi već postoje, ili se pak mogu kreirati potpuno novi izvorni fajlovi, pritiskom na „New File...“ dugme. Pošto smo mi već kreirali algoritamski model „Naive“ množača matrica, potrebno je samo u projekat uključiti postojeći `.cpp` fajl u kojem se model nalazi (pošto se u našem primeru čitav algoritamski model nalazi unutar jednog C++ fajla). Pored toga, potrebno je specificirati i ime funkcije koja se nalazi na najvišem nivou hijerarhije unutar algoritamskog modela, i od koje je potrebno započeti proces sinteze visokog nivoa. U našem primeru, izgled ovog prozora treba da odgovara onom sa slike 7.8.

NAPOMENA: Zaglavlja (`.h` fajlove) koja čine deo algoritamskog modela nije potrebno eksplicitno uključivati u projekat. Ukoliko se ona nalaze u direktorijumima u kojima se nalaze uključeni `.c` ili `.cpp` izvorni fajlovi biće automatski uključena. Ukoliko to nije slučaj, onda se putanja do `.h` fajla mora eksplicitno navesti, korišćenjem opcije „Edit CFLAGS...“.

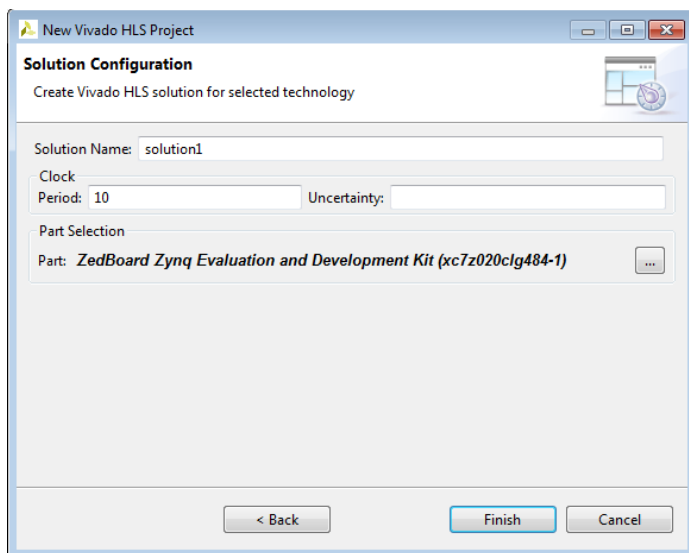
NAPOMENA: „Edit CFLAGS...“ dugme služi za specificiranje opcija kompajlera koje je potrebno koristiti prilikom kompajliranja odgovarajućeg `.c` ili `.cpp` fajla.

Kada smo dodali sve izvorne fajlove u projekat i specificirali funkciju na najvišem hijerarhijskom nivou, pritiskom na *Next* dugme prelazimo na „Add/Remove Files Testbench Files“ prozor, prikazan na slici 7.9.



Slika 7.9. „New Project – Add/Remove Files Testbench Files“ prozor sa potrebnim podešavanjima

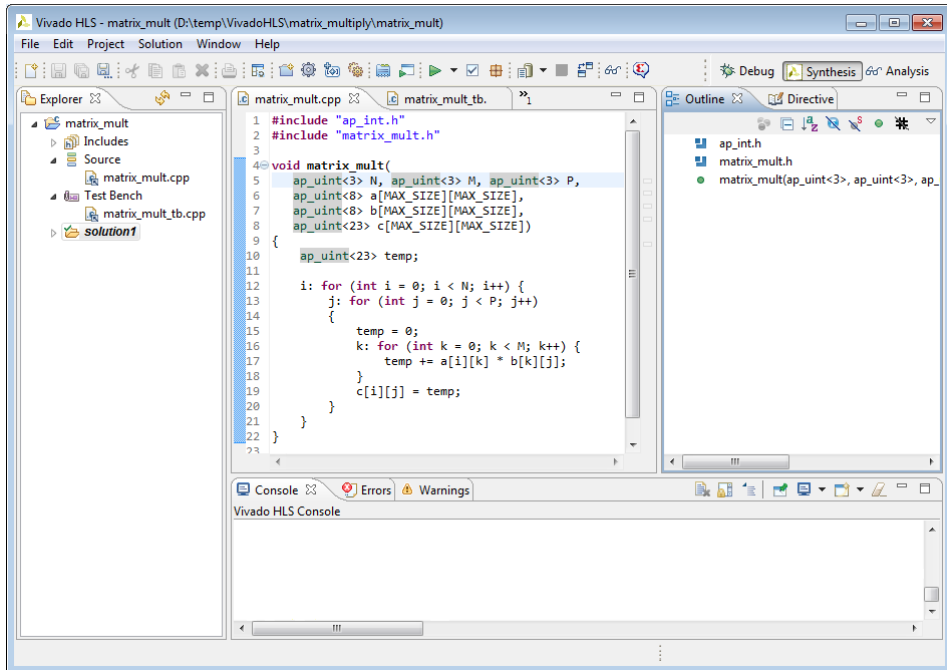
U okviru „Add/Remove Files Testbench Files“ prozora u projekat se uključuju svi fajlovi koji čine verifikaciono okruženje za algoritamski model sistema koji projektujemo. Postojeći fajlovi uključuju se u projekat pritiskom na „Add Files...“ dugme, dok se pritiskom na „New File...“ dugme mogu kreirati novi fajlovi i dodati u projekat. U našem primeru izgled „Add/Remove Files Testbench Files“ prozora nakon uključivanja svih potrebnih fajlova treba da odgovara onom sa slike 7.9. Pritiskom na *Next* dugme, prelazimo na poslednji prozor, „Solution Configuration“, prikazan na slici 7.10.



Slika 7.10. „New Project – Solution Configuration“ prozor sa potrebnim podešavanjima

U okviru „Solution Configuration“ prozora definišu se detalji inicijalne implementacije koja će biti generisana u procesu sinteze visokog nivoa. Unutar „Solution Name“ polja definiše se ima inicijalne implementacije, automatski generisana imena uvek imaju sufiks *solution* iza koga sledi redni broj tekuće implementacije. U okviru *Period* polja definiše se željena perioda klock signala na kojoj bi projektovani sistem trebao da radi. Ova informacija koristi se unutar Vivado HLS alata prilikom faze vremenskog planiranja. Polje *Uncertainty* definiše marginu u pretpostavljenoj minimalnoj periodu klock signala, koja modeluje nepoznata kašnjenja koja će nastati u procesima RTL sinteze, raspoređivanja i povezivanja. Prilikom sinteze visokog nivoa, perioda klock signala koja će biti korišćena dobija se kao perioda specificirana u *Period* polju umanjena za vrednost *Uncertainty* polja. Poslednje polje je *Part* polje, u okviru kojega je potrebno specificirati ciljnu FPGA komponentu pomoću koje će projektovani sistem na kraju biti realizovan. Ovaj podatak koristi se prilikom estimacije kašnjenja različitih hardverskih resursa prisutnih na FPGA komponenti, u procesu sinteze.

U našem primeru, potrebna podešavanja treba da odgovaraju onima sa slike 7.10. Pritiskom na dugme *Finish* završava se proces kreiranja novog projekta i otvara se glavni prozor Vivado HLS alata, prikazan na slici 7.11.



Slika 7.11. Glavni prozor Vivado HLS alata sa algoritamskim modelom „Naive“ množača matrica

Glavni prozor Vivado HLS alata sastoji se iz četiri celine:

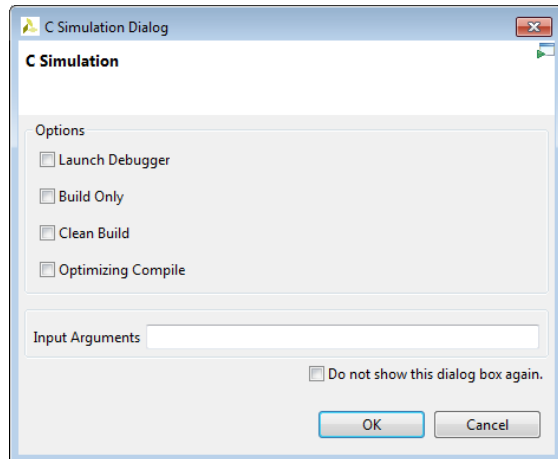
- *Explorer* prozora – u okviru kojega su prikazani svi fajlovi koji čine tekući projekat. Fajlovi su organizovani u direktorijume, koji odgovaraju organizaciji koja postoji i na disku.
- Centralnog, *Information* prozora – u okviru kojega se prikazuje sadržaj selektovanih fajlova iz projekta. Na slici 7.11 u okviru ovog prozora prikazan je algoritamski model „Naive“ množača matrica, koji je sadržan u okviru *matrix_mult.cpp* fajla.
- *Auxiliary* prozora – koji se nalazi na desnoj strani glavnog prozora Vivado HLS alata i koji sadrži informacije koje su relevantne za trenutno aktivan fajl u okviru *Information* prozora.
- *Console* prozora – koji se nalazi na dnu glavnog prozora Vivado HLS alata, u okviru kojega se ispisuju poruke, greške i upozorenja koja generiše Vivado HLS alat u toku svog rada.

7.3. Verifikacija algoritamskog modela

Pre započinjanja sinteze visokog nivoa poželjno je izvršiti funkcionalnu verifikaciju algoritamskog modela sistema koji se projektuje. Ovo se postiže sprovođenjem takozvane C simulacije u okviru Vivado HLS alata. Prilikom C simulacije simulira se kompletno verifikaciono okruženje, zajedno sa algoritamskim modelom sistema koji se projektuje. Verifikaciono okruženje trebalo bi da sadrži testbenč fajl sa *main()* funkcijom u okviru koje se poziva funkcija u kojoj je definisan algoritamski model sistema koji se verifikuje. Preporučena struktura testbenča unutar Vivado HLS alata trebalo bi da uključuje sledeće:

- Testbenč bi trebao da bude samoproveravajući (*self-checking*), verifikujući rezultate izvršavanja funkcije koja sadrži algoritamski model sistema koji se projektuje.
- U slučaju da su rezultati rada funkcije koja sadrži algoritamski model korektni, testbenč bi trebao da vrati povratnu vrednost 0, u protivnom bi trebao da vrati nenultu vrednost prilikom napuštanja *main()* funkcije.

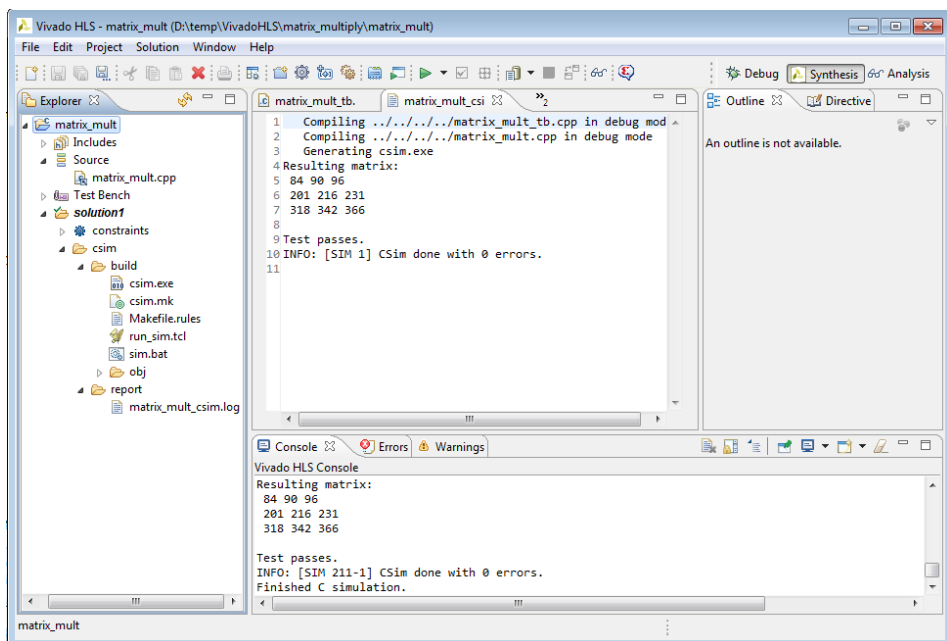
C simulacija pokreće se pritiskom na „Run C Simulation“ dugme, , ili izborom opcije *Project*→*Run C Simulation*, pri čemu se pojavljuje prozor sa slike 7.12.



Slika 7.12. „C Simulation“ konfiguracioni prozor

U okviru konfiguracionog prozora sa slike 7.12 moguće je konfigurisati kako će se predstojeća C simulacija izvršiti. Ukoliko želimo da u procesu C simulacije koristimo standardni C debager, potrebno je aktivirati opciju „Launch Debugger“. Ako samo želimo da generišemo izvršni fajl, koji nastaje kompajliranjem svih potrebnih C fajlova

koji čine verifikaciono okruženje i algoritamski model sistema, ali ne želimo da pokrenemo samu simulaciju, potrebno je aktivirati opciju „Build Only“. Ukoliko želimo da uklonimo sve među fajlove i izvršni fajl koji su kreirani u prethodnom kompajliranju, pre nego što ponovo kompajliramo sve neophodne fajlove, potrebno je aktivirati opciju „Clean Build“. Ukoliko želimo da prilikom kompajliranja generišemo optimizovanu verziju simulacije, potrebno je aktivirati opciju „Optimization Compile“. Optimizovana verzija simulacije izvršavaće se brže od neoptimizovane verzije ali neće uključivati informacije koje su neophodne ukoliko želimo da izvršimo proces debagovanja pomoću debagera. U našem slučaju nije potrebno aktivirati ni jednu od ovih opcija. Pritiskom na dugme OK pokreće se C simulacija, a Vivado HLS alat vraća se u glavni prozor. Tok i konačni rezultati simulacije mogu se pratiti i analizirati u okviru *Console* prozora, a na kraju simulacije automatski će biti otvoren i prikazan u okviru *Information* prozora sadržaj *matrix_mult_csim.log* fajla, kao što se može videti na slici 7.13.



Slika 7.13. Rezultat C simulacije algoritamskog modela „Naive“ množača matrica

Na slici 7.13 možemo videti vrednosti elemenata matrice C, izračunatih pomoću *matrix_mult* funkcije, koja predstavlja proizvod dve ulazne, test matrice A i B. Kako su elementi matrica A i B identični sa elementima matrica A i B koje su bile korišćene za funkcionalnu verifikaciju VHDL RTL modela „Naive“ množača matrica iz primera 4.3, strana 160, upoređivanjem sa očekivanim rezultatom množenja ove dve matrice, prikazanim na strani 160, možemo se uveriti da razvijeni algoritamski modela „Naive“

množača matrica u C++ programskom jeziku sa slike 7.4 korektno implementira potrebnu funkcionalnost.

Kada se pokrene C++ simulacija, Vivado HLS alat automatski kreira *csim* direktorijum u okviru aktivne implementacije, kao što se može videti na slici 7.13. Direktorijum *csim/build* predstavlja primarnu lokaciju na koju se smeštaju svi fajlovi koji imaju veze sa C simulacijom:

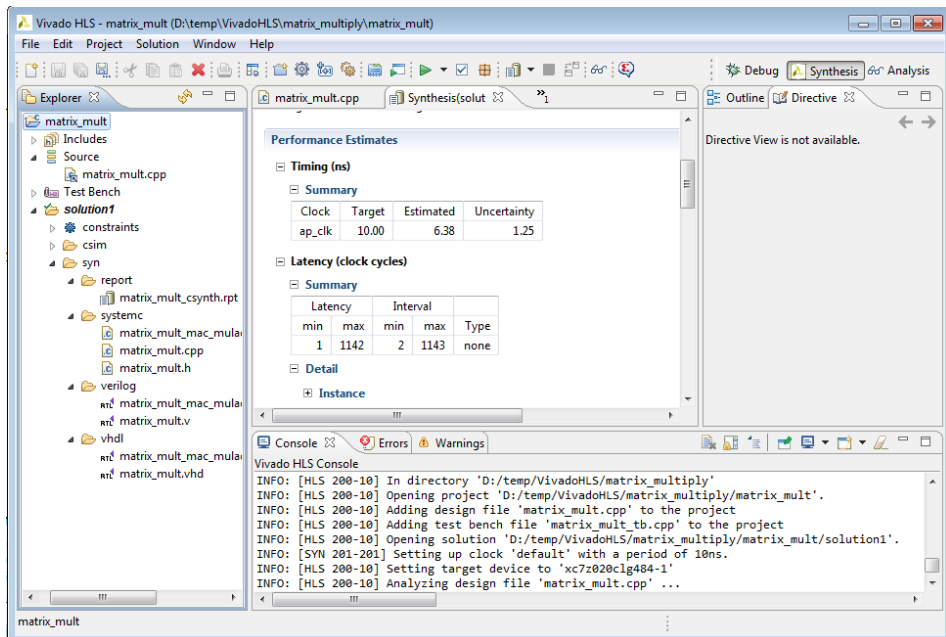
- Svi fajlovi koji se učitavaju prilikom izvršavanja testbenča kopiraju se u ovaj direktorijum.
- Izvršni fajl, *csim.exe*, čijim pokretanjem se zapravo pokreće C simulacija takođe se nalazi u ovom direktorijumu.
- Svi fajlovi koji se kreiraju ili u koje se upisuju podaci prilikom simulacije takođe se smeštaju u ovaj direktorijum.

Pored *csim/build* direktorijuma, Vivado HLS automatski kreira i *csim/report* direktorijum u koji smešta tekstualni log fajl koji sadrži izveštaj o toku simulacije.

Nakon uspešno završene funkcionalne verifikacije algoritamskog modela sistema koji projektujemo, možemo preći na sledeći korak, generisanje inicijalne implementacije.

7.4. Generisanje inicijalne implementacije

Pritiskom na „C Synthesis“ dugme, , ili izborom opcije *Solution*→*Run C Synthesis*, pokreće se sinteza visokog nivoa algoritamskog modela. Tok sinteze visokog nivoa može se pratiti unutar *Console* prozora. U okviru *Console* prozora Vivado HLS alat ispisuje sve važne informacije vezane za način na koji se algoritamski model transformiše u odgoravajući RTL model, koje se od zahtevanih optimizacionih direktiva mogu a koje ne mogu primeniti na algoritamski model, itd. Nakon što se proces sinteze visokog nivoa završi, automatski se prikazuje izveštaj o rezultatima sinteze, kao što se može videti na slici 7.14 u slučaju sinteze algoritamskog modela „Naive“ množača matrica. Takođe, nakon završene sinteze Vivado HLS automatski kreira i *syn* direktorijum, u okviru kojega smešta izveštaj o završenoj sintezi (u okviru *report* direktorijuma), kao i HDL modele generisane RTL implementacije digitalnog sistema (u okviru *vhdl*, *verilog* i *systemc* direktorijuma). Dvoklikom na neki od HDL modela možemo videti detalje generisane implementacije, iako se malo toga zapravo može videti, jer su automatski generisani HDL modeli uglavnom vrlo teško čitljivi za čoveka.



Slika 7.14. Izveštaj o rezultatu sinteze algoritamskog modela „Naive“ množača matrica

Sledeći korak u procesu projektovanja digitalnog sistema pomoću Vivado HLS alata predstavlja korak analize karakteristika inicijalne implementacije, u cilju provere da li ona zadovoljava zahtevane kriterijume, u pogledu vremena potrebnog za izvršavanje implementiranog algoritma i zahtevanih hardverskih resursa za njegovu implementaciju.

7.4. Analiza karakteristika generisane implementacije

Vivado HLS alat pruža dva načina da se izvrši analiza karakteristika generisane implementacije:

- Pregledom izveštaja o završenoj sintezi (*Synthesis Report*)
- Korišćenjem režima analize implementacije (*Analysis Perspective*)

U postupku analize proveravaju se performanse i hardverski zahtevi tekuće implementacije i upoređuju sa zahtevanim vrednostima. U slučaju da su zahtevane vrednosti zadovoljene tekućom implementacijom, postupak sinteze je završen i automatski generisani RTL model sistema može se koristiti unutar Vivado Design Suite alata u cilju generisanja konfiguracionog fajla za odabranu FPGA komponentu.

U slučaju da tekuća implementacija ne ispunjava zahtevane vrednosti u pogledu performansi ili hardverskih resursa, pristupa se procesu analize. Proces analize treba da otkrije mesta za potencijalnu optimizaciju tekuće arhitekture. Analizom načina kako je algoritamski model preveden u tekući RTL model, načina na koji se podaci prenose ka i od sistema koji se projektuje, uočavaju se uska grla u tekućoj implementaciji, kao i mesta za dodatnu paralelizaciju procesa obrade podataka. Nakon što se identifikuju potencijalna mesta za optimizaciju tekuće implementacije, definišu se odgovarajuće dodatne optimizacione direktive, i započinje se proces generisanja naredne implementacije.

Na ovom mestu detaljnije će biti objašnjene mogućnosti koje pruža Vivado HLS alat za analizu karakteristika tekuće implementacije algoritamskog modela u RTL model.

7.4.1. Analiza izveštaja o završenoj sintezi

Kada se završi proces sinteze RTL modela, Vivado HLS automatski prikazuje izveštaj o rezultatima sinteze, što se može videti na slici 7.14. Izveštaj se odnosi na sintetizovanu funkciju koja se nalazi na najvišem nivou hijerarhije unutar algoritamskog modela sistema. Izveštaj prikazuje informacije kako o performansama tekuće RTL implementacije, tako i o potrebnim hardverskim resursima za njenu realizaciju pomoću odabrane FPGA komponente. Izveštaj takođe sadrži i detalje u vezi sa korišćenim interfejsima pomoću kojih su ulazni i izlazni argumenti algoritamskog modela mapirani u hardver.

U nastavku će detaljnije biti objašnjene sekcije od kojih se sastoji izveštaj o sintezi.

Opšte informacije

Ovaj deo izveštaja o sintezi sadrži sledeće informacije:

- datum kada je sinteza izvršena,
- verziji Vivado HLS alata koja je bila korišćena,
- imenu projekta za koji je izvršena sinteza,
- imenu implementacije koja je generisana i
- ciljnoj FPGA komponenti koja je bila korišćena.

Slika 7.15 prikazuje ovaj deo izveštaja u slučaju generisanja inicijalne implementacije (označene kao *solution1*) „Naive“ množača matrica.

General Information	
Date:	Fri Aug 05 21:17:19 2016
Version:	2016.2 (Build 1577090 on Thu Jun 02 16:59:10 MDT 2016)
Project:	matrix_mult
Solution:	solution1
Product family:	zynq
Target device:	xc7z020clg484-1

Slika 7.15. Izveštaj o rezultatima sinteze, opšte informacije

Estimacija performansi

Naredna sekcija izveštaja odnosi se na informacije koje detaljnije prikazuju performanse koje se mogu postići korišćenjem tekuće implementacije. Ova sekcija sastoji se iz sledećih podsekcija:

- **Timing→Summary** – U ovom delu prikazane su informacije o zahtevanoj minimalnoj periodi klok signala (*Target*), estimiranoj minimalnoj periodi klok signala (*Estimated*) za koju projektovani digitalni sistem još uvek radi pouzdano, i nesigurnosti (*Uncertainty*) zahtevane minimalne periode klok signala (koja modeluje dodatna kašnjenja unutrašnjih signala koja će nastupiti u postupku RTL sinteze, raspoređivanja i povezivanja).
- **Timing→Latency→Summary** – u ovom delu prikazane su informacije o minimalnom i maksimalnom kašnjenju (*Latency*), kao i minimalnom i maksimalnom inicijalizacionom intervalu (*Initiation interval*) za blok na najvišem nivou hijerarhije, kao i za sve blokove sa nižih nivoa hijerarhije, ukoliko ih ima.

Kašnjenje (*Latency*) predstavlja broj perioda klok signala koji su neophodni da se izvrši implementirani algoritamski model i generišu nove vrednosti izlaznih signala koje predstavljaju izlaz algoritma. Manje kašnjenje odgovara bržoj implementaciji algoritma.

Inicijalizacioni interval (*Initiation interval*) predstavlja broj perioda klok signala koji mora da protekne pre nego što se nove vrednosti ulaznih signala mogu dovesti na ulaze projektovanog sistema. U slučaju da sistem ne koristi protočnu obradu podataka, vrednost inicijalizacionog intervala uvek je za jednu periodu klok signala veća od kašnjenja (jer se naredni podaci koje je potrebno obraditi učitavaju jedan takt nakon što se generišu rezultati prethodne obrade).

- **Timing→Latency→Detail** – prikazuje informacije o kašnjenju i inicijalizacionom intervalu za svaku podfunkciju i petlju koja je prisutna u algoritamskom modelu sistema. Ukoliko neke od petlji sadrže ugnježdene petlje, prikazana je kompletna hijerarhijska struktura ugnježđenih petlji.

Kolona *Latency* prikazuje informaciju o broju taktova potrebnih da se izvrši svaka od petlji prisutnih u algoritamskom modelu sistema. Kako petlje u sebi mogu sadržati naredbe uslovnog grananja ili njihove granice mogu biti promenljive, informacije o kašnjenju su prikazane u formi minimalnog i

maksimalnog kašnjenja. U slučaju da petlja ne sadrži naredbe uslovnog grananja i kada je broj prolaza kroz petlju konstantan, vrednosti minimalnog i maksimalnog kašnjenja su jednake.

Kolona „*Iteration Latency*“ prikazuje broj taktova koji je neophodan da se izvrši jedna iteracija petlje. Kako i ovaj broj taktova može biti promenljiv, iz istog razloga kao i kašnjenje cele petlje, i on se prikazuje u formi *min~max*.

Kolona „*Initiation Interval*“ prikazuje informacije o zahtevanom (*target*) i postignutom (*achieved*) inicijalizacionom intervalu za svaku petlju. Ukoliko korisnik nije specificirao zahtevane vrednosti inicijalizacionih intervala, ova kolona je prazna.

Kolona „*Trip Count*“ prikazuje informacije o minimalnom i maksimalnom broju prolaza kroz posmatranu petlju, jer broj prolaza kroz petlju u opštem slučaju može da bude promenljiv.

Na kraju, kolona *Pipelined* pruža informacije da li je posmatrana petlja optimizovana pomoću tehnike protočne obrade ili nije.

Na slici 7.16 prikazan je ovaj deo izveštaja o sintezi u slučaju generisanja inicijalne implementacije „*Naive*“ množača matrica.

Performance Estimates

[-] Timing (ns)

[-] Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	6.38	1.25

[-] Latency (clock cycles)

[-] Summary

Latency		Interval		Type
min	max	min	max	
1	1142	2	1143	none

[-] Detail

[+] Instance

[-] Loop

	Latency			Initiation Interval			
Loop Name	min	max	Iteration Latency	achieved	target	Trip Count	Pipelined
- i	0	1141	2 ~ 163	-	-	0 ~ 7	no
+ j	0	161	2 ~ 23	-	-	0 ~ 7	no
++ k	0	21	3	-	-	0 ~ 7	no

Slika 7.16. Izveštaj o rezultatima sinteze, estimacija performansi

Sa slike 7.16 možemo videti da je zahtevana minimalna perioda klok signala ispunjena u ovom slučaju, jer je estimirana minimalna perioda od 6.38 ns, što je vrednost koja je manja od ciljne vrednosti, $10.00 - 10.00 * 0.25 = 7.5$ ns.

Takođe možemo primetiti da su minimalna i maksimalna kašnjenja prilikom izvršavanja „Naive“ algoritma množenja dve matrice estimirana na 1, odnosno 1142 periode klok signala. Kako tekuća implementacija ne koristi protočnu obradu signala, estimirane vrednosti inicijalizacionog intervala za jednu periodu klok signala su veće od estimiranih vrednosti kašnjenja, 2 odnosno 1143 periode klok signala.

Iz izveštaja takođe možemo videti da u tekućoj implementaciji postoje tri ugnježdene petlje, i , j i k . Za svaku od ovih petlji prikazane su informacije o minimalnom i maksimalnom kašnjenju, minimalnom i maksimalnom broju taktova potrebnih da se završi jedan prolaz kroz petlju, kao i podaci o minimalnom i maksimalnom broju prolaza kroz petlju. Takođe možemo videti da ni jedna petlja ne koristi protočnu obradu. Korišćenjem ovih podataka možemo ručno proveriti informacije o ukupnom kašnjenju čitavog sistema, od 1 do 1141 periode klok signala.

Sa slike 7.16 vidimo da je potrebno 3 periode klok signala da se izvrši najdublja petlja, po iteratoru k . Kako se ova petlja izvršava od 0 do 7 puta (jer je to određeno ulaznim parametrom M , koji je deklarisan kao 3-bitni *uint3* tip podataka pa su njegove vrednosti iz intervala 0-7), minimalni i maksimalni broj prolaza kroz ovu petlju iznose 0, odnosno $7 * 3 = 21$ periodu klok signala.

Sledeća petlja, u kojoj se nalazi petlja k , je petlja po iteratoru j . Vidimo da prolaz kroz ovu petlju traje minimalno 2, a maksimalno 23 periode klok signala. Ove vrednosti su za 2 periode klok signala veće od minimalnog i maksimalnog kašnjenja ugnježdene petlje k . Razlog za ova dve dodatne periode klok signala je u tome što je potrebna jedna perioda klok signala da se iz petlje j uđe u petlju k , i još jedna dodatna perioda da se iz petlje k vratimo u petlju j . Ovaj način implementiranja ulaska i izlaska iz petlji je karakteristika Vivado HLS alata i ne može se menjati. Kako kroz petlju j takođe možemo prolaziti od 0 do 7 puta (ovog puta to je određeno vrednošću ulaznog parametra P , koji je takođe deklarisan kao 3-bitni tip podatka), minimalno i maksimalno kašnjenje kroz petlju j iznosi 0, odnosno $7 * 23 = 161$ periodu klok signala.

Poslednja petlja je petlja po iteratoru i , koja sadrži petlje j i k unutar sebe. Kao što se sa slike 7.16 može videti, prolaz kroz petlju i traje minimalno 2, a maksimalno 163 periode klok signala. I ove vrednosti su za 2 periode klok signala veće od minimalnog i maksimalnog kašnjenja prilikom izvršavanja petlje j iz istog razloga kao i u slučaju petlje j . Kako se kroz petlju i takođe prolazi od 0 do 7 puta (što je ovaj put određeno konkretnom vrednošću ulaznog parametra N , koji je kao i parametri M i P takođe deklarisan kao *uint3* tip), minimalno i maksimalno kašnjenje prilikom izvršavanja petlje i iznosi 0, odnosno $7 * 163 = 1141$ periode klok signala.

Kako se ceo „Naive“ algoritam množenja matrica sastoji iz ove tri ugnježdene petlje, ukupno kašnjenje prilikom izvršavanja celokupnog algoritma za jednu periodu klok signala je veće od minimalnog i maksimalnog kašnjenja izvršavanja petlje i i iznosi 1 odnosno 1142 periode klok signala. Ova dodatna perioda klok signala neophodna je da se odluči da li je uopšte potrebno započeti izvršavanje petlje i ili ne.

Estimacija zahtevanih hardverskih resursa

U okviru ove sekcije izveštaja o rezultatima sinteze prikazani su potrebni hardverski resursi, blok RAM memorije (BRAM), DSP blokovi (DSP48E), flip-flopovi (FF) i *lookup* tabele (LUT), neophodni za realizaciju tekuće implementaciju algoritamskog modela. Potrošnja ovih hardverskih resursa dalje je dekomponovana na sledeće kategorije:

- **DSP** – hardverski resursi neophodni za implementaciju DSP operacija, koje su prisutne u algoritamskom modelu sistema.
- **Expression** – hardverski resursi neophodni za implementaciju osnovnih aritmetičkih, logičkih i relacionih operacija, koje su prisutne u algoritamskom modelu sistema.
- **FIFO** – hardverski resursi neophodni za implementaciju FIFO bafera koji su identifikovani unutar algoritamskog modela sistema.
- **Instance** – hardverski resursi neophodni za implementaciju blokova koji su instancionirani na ovom nivou hijerarhije. Ukoliko dizajn ne instancira nikakve blokove ova vrsta je prazna.
- **Memory** – hardverski resursi neophodni za implementaciju memorija koje su identifikovane unutar algoritamskog modela sistema.
- **Multiplexer** – hardverski resursi neophodni za implementaciju multipleksera koji se koriste za kontrolu toka podataka.
- **Register** – hardverski resursi neophodni za implementaciju registara koji su neophodni za smeštanje sadržaja promenljivih koje su identifikovane unutar algoritamskog modela sistema.

Nakon tabele koja prikazuje zbirne podatke o potrebnim hardverskim resursima, detaljnije informacije mogu se pronaći u *Detail* delu, koji sledi nakon tabele.

Utilization Estimates				
Summary				
Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	1	-	-
Expression	-	-	0	15
FIFO	-	-	-	-
Instance	-	-	-	-
Memory	-	-	-	-
Multiplexer	-	-	-	29
Register	-	-	65	-
Total	0	1	65	44
Available	280	220	106400	53200
Utilization (%)	0	~0	~0	~0
Detail				
Instance				
DSP48				
Memory				
FIFO				
Expression				
Multiplexer				
Register				

Slika 7.17. Izveštaj o rezultatima sinteze, estimacija hardverskih resursa

U našem slučaju sa slike 7.17 možemo videti da je za inicijalnu implementaciju „Naive“ algoritma množača matrica potreban jedan DSP blok, za implementaciju operacije množenja vrednosti elemenata matrica A i B i akumuliranja ovih proizvoda. Kako je ovaj proces akumuliranja proizvoda vrlo čest u oblasti digitalne obrade signala (reč je o takozvanoj „Multiply and Accumulate“ operaciji, MAC), Vivado HLS alat ju je svrstao u DSP klasu unutar tabele. Pored jednog DSP bloka potreban je i izvestan broj flip flopova, za realizaciju registara u kojima se čuvaju vrednosti unutrašnjih promenljivih koje su identifikovane u „Naive“ algoritmu množenja matrica (na primer to su promenljive i , j i k , koje čuvaju tekuće vrednosti iteratora tri petlje). Vidimo da je takođe potreban i izvestan broj *lookup* tabela, pomoću kojih se realizuju multipleksterske mreže za rutiranje podataka kroz sistem, kao i sabirači i komparatori neophodni za korektnu implementaciju petlji koje su pristune u „Naive“ algoritmu (za inkrementaciju iteratora petlji kao i za proveru da li su iteratori dostigli svoje maksimalne vrednosti).

Informacije o korišćenim interfejsima

Poslednja sekcija izveštaja o rezultatima sinteze prikazuje informacije vezane za broj i vrstu interfejsa koji su iskorišćeni za implementaciju ulaznih i izlaznih argumenata algoritamskog modela.

Prva kolona (*RTL Ports*) tabele sadrži imena svih ulaznih i izlaznih portova koji su prisutni u implementiranom RTL modelu. *Dir* kolona sadrži informacije o smeru svakog

od prisutnih portova. Kolona *Bits* sadrži informacije o širini svakog porta. Kolona *Protocol* sadrži informacije o protokolu (interfejsu) kojem dati port pripada. Kolona „*Source Objekt*“ sadrži informacije kojem od ulaznih i izlaznih argumenata algoritamskog modela je asociran posmatrani port. Na kraju, kolona „*C Type*“ pruža informacije o tipu argumenta algoritamskog modela kojem je posmatrani port pridružen.

Interface					
Summary					
RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	matrix_mult	return value
ap_rst	in	1	ap_ctrl_hs	matrix_mult	return value
ap_start	in	1	ap_ctrl_hs	matrix_mult	return value
ap_done	out	1	ap_ctrl_hs	matrix_mult	return value
ap_idle	out	1	ap_ctrl_hs	matrix_mult	return value
ap_ready	out	1	ap_ctrl_hs	matrix_mult	return value
N_V	in	3	ap_none	N_V	scalar
M_V	in	3	ap_none	M_V	scalar
P_V	in	3	ap_none	P_V	scalar
a_V_address0	out	6	ap_memory	a_V	array
a_V_ce0	out	1	ap_memory	a_V	array
a_V_q0	in	8	ap_memory	a_V	array
b_V_address0	out	6	ap_memory	b_V	array
b_V_ce0	out	1	ap_memory	b_V	array
b_V_q0	in	8	ap_memory	b_V	array
c_V_address0	out	6	ap_memory	c_V	array
c_V_ce0	out	1	ap_memory	c_V	array
c_V_we0	out	1	ap_memory	c_V	array
c_V_d0	out	23	ap_memory	c_V	array

Slika 7.18. Izveštaj o rezultatima sinteze, informacije o korišćenim interfejsima

Analizom tabele prikazane na slici 7.18 vidimo način na koji su ulazni i izlazni argumenti algoritamskog modela „*Naive*“ množača matrica implementirani pomoću odgovarajućih portova automatski generisanog RTL modela. Vidimo da su ulazni argumenti N , M i P implementirani pomoću tri 3-bitna ulazna porta N , M i P . Podaci se preko ovih portova prenose bez nekog posebnog protokola, što je jasno identifikovano u poljima iz kolone *Protocol*, koja za portove N , M i P imaju vrednost protokola *ap_none*. Iz tabele sa slike 7.18 možemo videti da se pristup matricama A , B i C ostvaruje korišćenjem *ap_memory* protokola. Protokol *ap_memory* predstavlja protokol koji se koristi za pristup RAM memorijama i sastoji se iz tri porta: *address*, *ce* i *q*. Ovaj izbor Vivado HLS je napravio automatski, pretpostavljajući da će matrice A , B i C biti smeštene u odgovarajućim RAM memorijama. Na kraju, sa slike 7.18 možemo videti da je Vivado HLS alat automatski kreirao i izvestan broj dodatnih portova, koji ne odgovaraju direktno nijednom od argumenata algoritamskog modela „*Naive*“ množača matrica. Ovi portovi su na slici 7.18 označeni imenima koja imaju prefiks *ap_*. Vidimo da imamo ukupno šest ovakvih portova, koji zajedno implementiraju *ap_ctrl_hs* protokol. Portovi *ap_clk* i *ap_rst* su standardni klok i reset ulazni portovi, dok portovi *ap_start*, *ap_done*, *ap_ready* i *ap_idle* implementiraju varijaciju kontrolnog protokola koji je bio korišćen u glavi 4 prilikom hardverske implementacije algoritama primenom RT metodologije. U poglavlju 7.5.2 biće prikazani i detaljnije opisani standardni

komunikacioni protokoli koje Vivado HLS alat može automatski da implementira prilikom implementacije argumenata algoritamskog modela u RTL model.

NAPOMENA: Dizajner ima potpunu kontrolu nad načinom na koji će svaki od ulaznih i izlaznih argumenata algoritamskog modela biti implementiran unutar RTL modela. Primenom odgovarajućih optimizacionih direktiva dizajner može naložiti Vivado HLS alatu koji protokol želi da bude iskorišćen prilikom implementacije svakog od prisutnih argumenata unutar algoritamskog modela. Ukoliko dizajner ne navede nikakve specifikacije, Vivado HLS alat automatski bira protokol koji se čini najpogodnijim za implementaciju svakog od argumenata algoritamskog modela.

7.4.2. Korišćenje režima analize implementacije

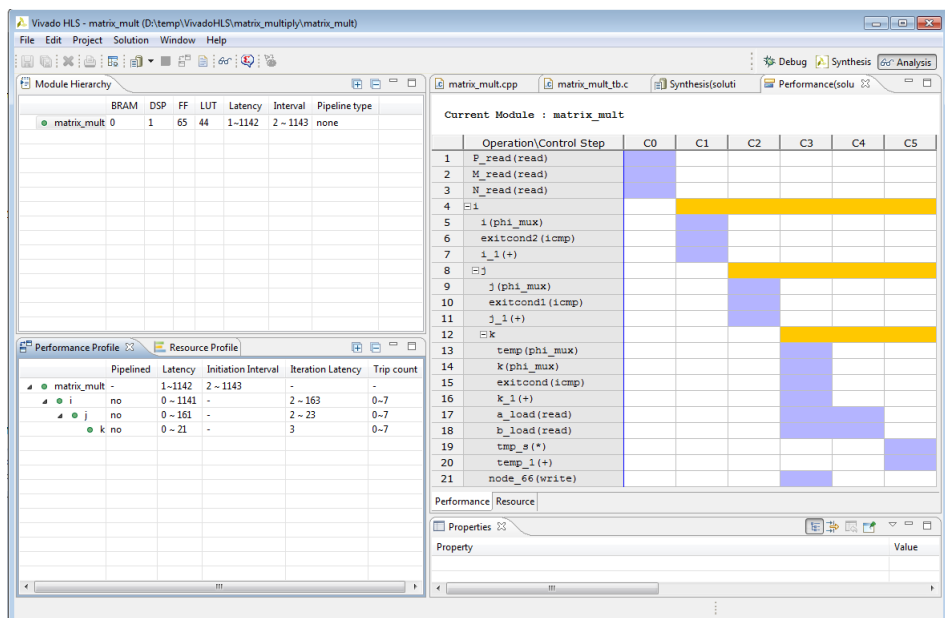
Pored izveštaja o rezultatima sinteze, Vivado HLS alat pruža i mogućnost detaljnije analize karakteristika tekuće implementacije, korišćenjem režima analize implementacije (*Analysis Perspective*). Pritiskom na *Analysis* dugme, koje se nalazi u gornjem desnom uglu glavnog Vivado HLS prozora, nakon što je završen postupak generisanja tekuće implementacije, Vivado HLS alat prelazi u režim analize implementacije, koji je prikazan na slici 7.19.

Kada se nalazi u režimu analize, glavni prozor Vivado HLS alata podeljen je na sledeće prozore.

Module Hierarchy prozor

U okviru ovog prozora može se analizirati hijerarhijska struktura implementiranog RTL modela. Kao što se može videti na slici 7.19, za svaki modul koja postoji unutar RTL modela sistema prikazani su hardverski resursi koji su potrebni za njegovu implementaciju, kao i performanse u terminima kašnjenja i inicijalizacionog intervala. Prolaskom kroz hijerarhijsku strukturu može se analizirati unutrašnja struktura svakog od modula, potrebni resursi za njegovu realizaciju i njegove performanse.

U slučaju RTL modela „Naive“ množača matrica možemo videti da se on sastoji samo iz jednog modula, *matrix_mult*. Ovo je i bilo očekivano jer unutar algoritamskog modela „Naive“ množača matrica ne postoji nikakva hijerarhijska struktura, pa ona samim tim ne postoji ni unutar automatski generisanog RTL modela. Analizom potrebnih hardverskih resursa i performansi *matrix_mult* modula, možemo videti da su one identične sa vrednostima koje su prikazane u izveštaju nakon završene sinteze RTL modela.



Slika 7.19. Iglad Vivado HLS alat u režimu analize implementacije

Performance Profile prozor

U okviru ovog prozora detaljno su prikazane informacije o performansama modula koji je trenutno selektovan unutar „Module Hierarchy“ prozora. Performanse modula zavise od performansi podmodula od kojih je on sastavljen, kao i od ostale logike koja se nalazi na ovom nivou hijerarhije. U okviru „Performance Profile“ prozora prikazani su svi objekti od čijih performansi zavise i performanse celokupnog modula.

Performanse su kao i ranije izražene u terminima kašnjenja i inicijalizacionog intervala. U slučaju postojanja petlji, pored ovih informacija prikazano je i trajanje jedne iteracije petlje kao i broj prolaza kroz petlju, slično kao i u slučaju izveštaja nakon završene sinteze RTL modela.

U slučaju analize implementacije „Naive“ množača matrica, sa slike 7.19 možemo opet videti da se *matrix_mult* modul sastoji iz tri ugnježdene petlje, *i*, *j* i *k*. Ovo su svi objekti čije performanse doprinose i ukupnim performansama *matrix_mult* modula. Za svaku od tri petlje prikazane su bitne informacije o njenim performansama. Upoređivanjem prikazanih vrednosti sa vrednostima iz izveštaja nakon sinteze, možemo zaključiti da su one identične, što je i bilo očekivano.

Resource Profile prozor

U okviru ovog prozora prikazani su hardverski resursi neophodni za implementaciju modula koji je trenutno selektovan unutar „*Module Hierarchy*“ prozora. Prikaz resursa organizovan je na isti način kao što je to bio slučaj u izveštaju o završenoj sintezi RTL modela, što se može videti sa slike 7.20. U slučaju da postoji hijerarhijska struktura unutar selektovanog modula, *Instances* vrsta mogla bi da se proširi i bilo bi moguće videti kako su hardverski resursi raspoređeni unutar individualnih instanci koje čine trenutno selektovani modul.

	BRAM	DSP	FF	LUT	Bits P0	Bits P1	Bits P2	Banks/Depth	Words	W*Bits*Banks
matrix_mult	0	0	65	44						
I/O Ports(6)					48					
a					8					
b					8					
c					23					
M					3					
N					3					
P					3					
Instances(0)	0	0	0	0						
Memories(0)	0	0	0	0				0	0	0
Expressions(6)	0	0	0	15	18	12	0			
+	0	0	0	9	9	3	0			
i_1_fu_151_p2	0	0	0	3	3	1	0			
j_1_fu_162_p2	0	0	0	3	3	1	0			
k_1_fu_186_p2	0	0	0	3	3	1	0			
icmp	0	0	0	6	9	9	0			
exitcond_fu_181_p2	0	0	0	2	3	3	0			
exitcond1_fu_157_p2	0	0	0	2	3	3	0			
exitcond2_fu_146_p2	0	0	0	2	3	3	0			
Registers(11)			65	65						
a_load_reg_291			8	8						
ap_CS_fsm			6	6						
b_load_reg_296			8	8						
c_addr_reg_268			6	6						
i_1_reg_255			3	3						
i_reg_99			3	3						
j_1_reg_263			3	3						
j_reg_111			3	3						
k_1_reg_276			3	3						
k_reg_135			3	3						
temp_reg_123			19	19						
Channels(0)	0	0	0	0				0	0	0
Multiplexers(5)	0	0	29	29				0		
ap_NS_fsm	0	0	1	1				0		
i_reg_99	0	0	3	3				0		
j_reg_111	0	0	3	3				0		
k_reg_135	0	0	3	3				0		
temp_reg_123	0	0	19	19				0		

Slika 7.20. Izgled „Resource Profile“ prozora

U slučaju RTL implementacije „Naive“ množača matrica, sa slike 7.20 možemo videti da RTL model nema nikakvu hijerarhijsku strukturu. Takođe možemo videti na koji način su ulazni i izlazni argumenti algoritamskog modela mapirani u odgovarajuće

portove RTL modela. Unutar grupe *Expressions* moguće je videti kako su realizovane funkcionalne jedinice, neophodne za implementaciju svih operacija prisutnih unutar algoritamskog modela. Na primer, možemo videti da tekuća RTL implementacija koristi tri dvoulazna sabirača, za realizaciju operacija inkrementacije iteratora petlji *i*, *j*, *k*. Pošto je maksimalni broj prolaza kroz svaku od ovih petlji jednak 7, jedan od ulaza u sabirač je trobitni, dok je drugi jednobitan. Takođe se mogu identifikovati i tri 3-bitna komparatora pomoću kojih se utvrđuje da li je izvršen potreban broj iteracija kroz svaku od petlji. Slično, unutar *Registers* grupe jasno možemo identifikovati registre za prihvatanje novih vrednosti elemenata matrica *A* i *B* (registri *a_load_reg_291* i *b_load_reg_296*), kao i za čuvanje tekuće vrednosti elementa matrice *C* (registar *temp_reg_123*), a vide se i registri koji služe za čuvanje tekućih vrednosti tri iteratora petlji (registri *i_reg_99*, *j_reg_111* i *k_reg_135*).

Performance prozor

Ovaj prozor je jedan od najbitnijih prozora prilikom analize performansi tekuće implementacije, jer sadrži prikaz vremenskog planiranja svih operacija unutar tekuće RTL implementacije algoritamskog modela sistema. Tipičan izgled ovog prozora prikazan je na slici 7.21.

Current Module : matrix_mult

	Operation\Control Step	C0	C1	C2	C3	C4	C5
1	P_read(read)						
2	M_read(read)						
3	N_read(read)						
4	⊖ i						
5	i(phi_mux)						
6	exitcond2(icmp)						
7	i_1(+)						
8	⊖ j						
9	j(phi_mux)						
10	exitcond1(icmp)						
11	j_1(+)						
12	⊖ k						
13	temp(phi_mux)						
14	k(phi_mux)						
15	exitcond(icmp)						
16	k_1(+)						
17	a_load(read)						
18	b_load(read)						
19	tmp_s(*)						
20	temp_1(+)						
21	node_66(write)						

Slika 7.21. Izgled Performance prozora

Prikaz vremenskog planiranja organizovan je u vidu tabele, gde vrste predstavljaju operacije koje postoje u algoritamskom modelu, a kolone kontrolne korake koji zapravo predstavljaju unutrašnja stanja koja Vivado HLS alat koristi da bi planirao izvođenje operacija. Ova unutrašnja stanja blisko su povezana sa stanjima konačnog automata implementiranog unutar kontrolne jedinice RTL modela, ali ne postoji jedan-na-jedan mapiranje između ove dve grupe stanja.

Prva kolona tabele sadrži sve operacije koje su identifikovane prilikom analize algoritamskog modela i koje je neophodno izvršiti kako bi se izvršio ceo algoritam. Postoje tri grupe operacije, koje su posebno označene:

- operacije koje su grupisane unutar podmodula, označene zelenom bojom,
- operacije koje se nalaze unutar petlji, označene ljubičastom bojom,
- sve ostale operacije, označene sivom bojom.

Ukoliko je neka od operacija složena, odnosno sastoji se iz većeg broja drugih operacija, pored njenog imena stoji znak +, čijim pritiskom se prikazuje unutrašnja struktura složene operacije. Ovo je uvek slučaj sa podmodulima i petljama.

Stanje ili stanja u kojima je posmatrana operacija planirana za izvršavanje označena su odgovarajućom bojom (zelenom, ljubičastom ili sivom) unutar tabele, u vrsti koja je pridružena posmatranoj operaciji.

Sa slike 7.21 možemo videti da je za izvršavanje svih operacija identifikovanih unutar algoritamskog modela „Naive“ množača matrica potrebno ukupno 6 unutrašnjih stanja (stanja $C0$, $C1$, $C2$, $C3$, $C4$ i $C5$). Stanje $C0$ je očigledno inicijalno stanje, jer se u njemu nalazimo samo na početku izvršavanja algoritma, i u ovom stanju se inicijalizuju granice tri petlje. Petlja i izvršava se u stanjima $C1$ - $C5$ i u sebi sadrži petlju j . U stanju $C1$ vrši se inkrementacija iteratora petlje i i proveru da li je dostignuta granična vrednost iteratora petlje. Stanje $C1$ je jedan od razloga za dodatne periode klok signala, koji su neophodni da se uđe i izađe iz odgovarajuće petlje, o čemu je bilo reči u poglavlju 7.4.1, prilikom analize sekcije izveštaja o sintezu posvećene estimaciji performansi.

Slično, petlja j izvršava se u stanjima $C2$ - $C5$, i u sebi sadrži petlju k . Ovaj put se stanje $C2$ koristi za inkrementaciju iteratora petlje j i proveru da li je dostignuta granična vrednost iteratora petlje. Konačno, petlja k izvršava se u stanjima $C3$ - $C5$.

Sa slike 7.21 možemo videti da se stanju $C3$ vrši inkrementacija iteratora petlje k i proveru da li je dostignuta granična vrednost iteratora petlje. U stanju $C3$ takođe započinje i proces čitanja narednih vrednosti elemenata matrica A i B (operacije $a_load(read)$ i $b_load(read)$) koje će biti pomnožene i akumulirane na tekuću vrednost sume. Vidimo da ove operacije traju dva ciklusa, jer se protežu preko stanja $C3$ i $C4$. Ovo je očekivano jer Vivado HLS alat, ukoliko to nije drugačije specificirano, pretpostavlja da će matrice A i B biti smeštene unutar *BlockRAM* memorija unutar FPGA komponente, koje imaju sinhroni izlaz za podatke. U stanju $C3$ biće generisana adresa naredne lokacije sa koje se želi pročitati podatak, a u stanju $C4$ će podatak biti pročitani i smešten u unutrašnji registar. Konačno, u stanju $C5$ vrši se množenje dva

elementa matrica A i B , kao i akumuliranje ovog proizvoda na tekuću vrednost sume (operacije $tmp_s(*)$ i $temp_1(+)$). Sam upis izračunate vrednosti elementa matrice C vrši se u stanju $C3$, nakon napuštanja petlje k .

Sa slike 7.21, a i na osnovu prethodne analize, možemo videti da je za izvršavanje jedne iteracije petlje k potrebno ukupno tri periode klok signala, jer moramo proći kroz stanja $C3$, $C4$ i $C5$, a svako od njih traje jednu periodu klok signala. Ova informacija je potpuno u saglasju sa „Iteration Latency“ vrednošću za petlju k iz izveštaja o rezultatima sinteze, prikazanoj na slici 7.16. Na osnovu analize vremenskog planiranja izvođenja operacija, postaje jasnije kako je Vivado HLS došao do tog podatka.

Radi olakšavanja korelacije između operacija prikazanih u tabeli sa vremenskim rasporedom i linijama unutar algoritamskog modela koje korespondiraju posmatranoj operaciji, Vivado HLS alat pruža još jednu mogućnost. Desnim klikom na polje unutar tabele sa slike 7.21 u kojem je neka od operacija planirana, i izborom opcije „Goto Source“, u okviru „C Source“ prozora koji se otvara na dnu Vivado HLS glavnog prozora, automatski se označava linija koda unutar algoritamskog modela kojoj odgovara odabrana operacija unutar tabele sa vremenskim planiranjem.

Current Module : matrix_mult

Operation\Control Step	C0	C1	C2	C3	C4	C5
1 P_read(read)						
2 M_read(read)						
3 N_read(read)						
4 i						
5 i(phi_mux)						
6 exitcond2(icmp)						
7 i_1(+)						
8 j						
9 j(phi_mux)						
10 exitcond1(icmp)						
11 j_1(+)						
12 k						
13 temp(phi_mux)						
14 k(phi_mux)						
15 exitcond(icmp)						
16 k_1(+)						
17 a_load(read)						
18 b_load(read)						
19 tmp_s(*)						
20 temp_1(+)						
21 node_66(write)						

Performance Resource

Properties C Source

File: D:\temp\VivadoHLS\matrix_multiply\matrix_mult.cpp

```

14 {
15     temp = 0;
16     k: for (int k = 0; k < M; k++) {
17         temp += a[i][k] * b[k][j];
18     }
19     c[i][j] = temp;
20 }

```

Slika 7.22. Efekat aktiviranja „Goto Source“ akcije na odabranoj operaciji iz tabele vremenskog planiranja

Na primer, ukoliko odaberemo operaciju $b_load(read)$ i odaberemo „Goto Source“ akciju, unutar „C Source“ prozora biće označena linija broj 17 unutar algoritamskog modela „Naive“ množača matrica, u kojoj se vrednost elementa matrice B množi sa vrednošću elementa matrice A i akumulira na tekuću vrednost promenljive $temp$, kao što je prikazano na slici 7.22.

Resource prozor

Poslednji prozor koji nam stoji na raspolaganju prilikom analize tekuće implementacije je *Resource* prozor, prikazan na slici 7.23. U ovom prozoru prikazano je korišćenje raspoloživih hardverskih resursa u različitim unutrašnjim stanjima sistema tokom izvršavanja implementiranog algoritma. Prva kolona sadrži oznake svih hardverskih resursa koji su prisutni u RTL implementaciji algoritamskog modela. Oni odgovaraju listi resursa prikazanoj unutar „Resource Profile“ prozora. Analizom tabele iz *Resource* prozora moguće je utvrditi da li se neki resurs koristi od strane većeg broja različitih operacija. Ukoliko se te operacije mogu izvršiti u istom unutrašnjem stanju, a koriste isti hardverski resurs, to može biti indikacija o mogućoj optimizaciji koja može dovesti do poboljšanja performansi projektovanog sistema.

Current Module : matrix_mult							
	Resource\Control Step	C0	C1	C2	C3	C4	C5
1	☐ I/O Ports						
2	M	read					
3	P	read					
4	N	read					
5	b(p0)				read		
6	a(p0)				read		
7	c(p0)				write		
8	☐ Memory Ports						
9	c(p0)				write		
10	b(p0)				read		
11	a(p0)				read		
12	☐ Expressions						
13	i_phi_fu_103		phi_mux				
14	i_1_fu_151		+				
15	exitcond2_fu_146		icmp				
16	j_phi_fu_115			phi_mux			
17	j_1_fu_162			+			
18	exitcond1_fu_157			icmp			
19	temp_phi_fu_127				phi_mux		
20	k_phi_fu_139				phi_mux		
21	k_1_fu_186				+		
22	exitcond_fu_181				icmp		
23	grp_fu_229						+

Performance Resource

Slika 7.23. Izgled Resource prozora

Na primer, sa slike 7.23 možemo videti da se ni jedan hardverski resurs ne koristi u više od jednog stanja, što znači da u tekućoj implementaciji nemamo deljenja resursa. Na primer, na osnovu slike 7.23 možemo da zaključimo da se operaciju inkrementovanja iteratora petlji koriste tri posebna sabirača (hardverski resursi *i_1_fu_151*, *j_1_fu_162* i *k_1_fu_186*).

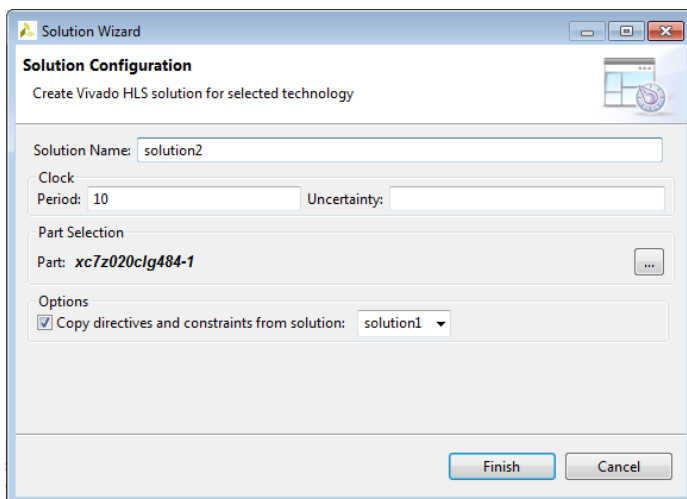
7.5. Kreiranje nove implementacije

Nakon što smo izvršili analizu tekuće implementacije algoritamskog modela i uočili potencijalna mesta za poboljšanje performansi ili racionalnije korišćenje hardverskih resursa, potrebno je formirati skup optimizacionih direktiva i generisati novu RTL implementaciju na osnovu njih. Vivado HLS alat pruža mogućnost da se rezultati tekuće RTL implementacije sačuvaju, a da se pored nje generiše nova implementacija. Ovakav pristup omogućava da se upoređuju karakteristike većeg broja različitih RTL implementacija i da se na taj način objektivno procene efekti odgovarajućih optimizacionih direktiva.

Pre nego što pokažemo koje je sve optimizacione direktive moguće definisati u okviru Vivado HLS alata, kao i načine kako se one zadaju u okviru Vivado HLS alata, prvo ćemo kreirati novu implementaciju. Prilikom kreiranja projekta već smo kreirali inicijalnu implementaciju (označili smo je kao *solution1*). Iako bi bilo moguće zadati željene optimizacione direktive i unutar postojeće implementacije i zatim je resintetizovati, ovo se ne preporučuje. Ovakvim pristupom izgubili bismo sve informacije u vezi sa performansama tekuće implementacije, pa bi bilo teško meriti efekte novih optimizacionih direktiva. Mnogo bolji pristup, koji Vivado HLS alat podržava, je da se kreira potpuno nova implementacija (možemo je nazvati potpuno proizvoljno, na primer *solution2*) i da se unutar nje definišu nove optimizacione direktive i onda izvrši sinteza algoritamskog modela na osnovu njih. Na ovaj način imaćemo dve implementacije, inicijalnu i novu, sa dodatnim optimizacionim direktivama. Sada će biti vrlo lako izvršiti poređenje performansi stare i nove implementacije, a samim tim i izvršiti objektivnu procenu efekata novih optimizacionih direktiva na performanse RTL implementacije.

Nova implementacija se u Vivado HLS alatu kreira na sledeći način. Prvo je neophodno izaći iz režima analize i vratiti se u režim sinteze. Ovo se postiže pritiskom na *Synthesis* dugme, koje se nalazi u gornjem desnom uglu Vivado HLS glavnog prozora.

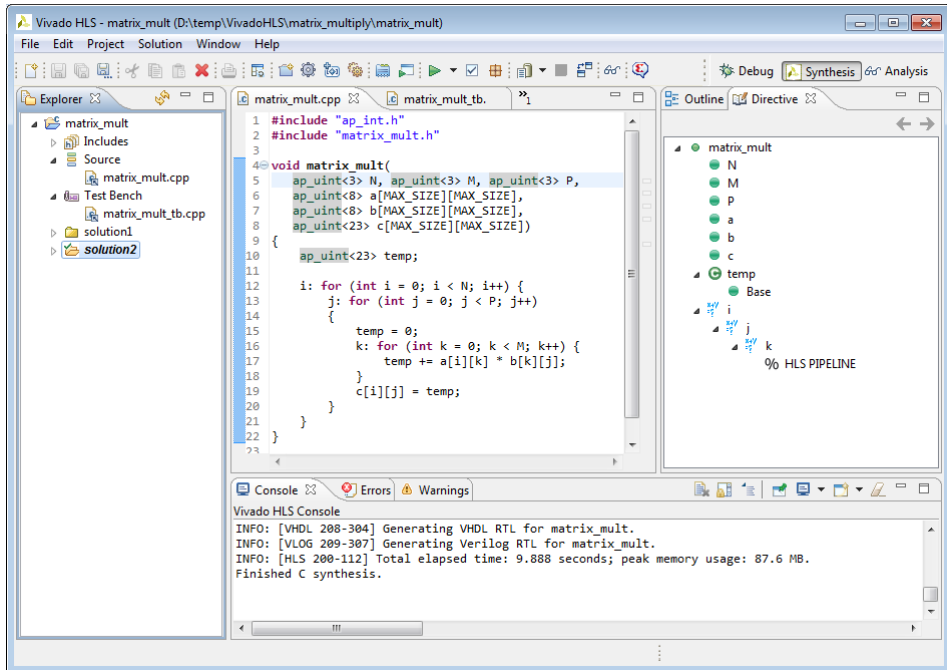
Pritiskom na „New Solution“ dugme, , ili izborom opcije *Project* → *New Solution* otvara se „Solution Configuration“ prozor, prikazan na slici 7.24.



Slika 7.24. „Solution Configuration“ prozor

Sa ovim prozorom smo se već sreli, prilikom inicijalnog kreiranja projekta. Tada smo specificirali karakteristike inicijalne implementacije, a ovaj put ćemo pomoću ovog prozora kreirati novu implementaciju. Kao i ranije, ime implementacije može se odabrati potpuno proizvoljno. Pored imena nove implementacije potrebno je definisati i maksimalnu periodu klok signala, kao i faktor nesigurnosti u njenoj proceni. Zatim je potrebno odabrati ciljnu FPGA komponentu. Vivado HLS alat automatski popunjava ova polja, koristeći njihov sadržaj iz prethodne implementacije, što je najčešće upravo ono što i želimo.

Poslednja stvar koju je potrebno specificirati je da li nova implementacija treba da preuzme ograničenja i optimizacione direktive neke od prethodnih implementacija. Ovo se postiže aktiviranjem opcije „Copy directives and constraints from solution“ i izborom imena implementacija odakle treba kopirati ograničenja i optimizacione direktive. Ovo je vrlo korisno, jer je proces sinteze visokog nivoa inkrementalan proces profinjenja tekuće implementacije primenom dodatnih optimizacionih direktiva, tako da je vrlo zgodno da nova implementacija može da nasledi skup svih ograničenja i optimizacionih direktiva prethodne implementacije i onda ih dodatno profini. U našem primeru želimo da nova implementacija, *solution2*, preuzme sva ograničenja i optimizacione direktive prethodne implementacije, *solution1*. Potrebna podešavanja prilikom kreiranja nove implementacije treba da odgovaraju onima sa slike 7.24. Kada smo zadovoljni sa konfiguracijom nove implementacije, pritiskom na dugme *Finish* Vivado HLS alat kreira novu implementaciju i vraća se u osnovni prozor. Međutim, sadržaj osnovnog prozora sada je promenjen, jer je u okviru *Explorer* prozora sada vidljiva još jedna implementacija, *solution2*, kao što se može videti sa slike 7.25. Obratite pažnju da je trenutno aktivna implementacija *solution2* (pored njenog imena nalazi se zelena kvačica). Ovo je važno, jer se sve akcije unutar Vivado HLS alata uvek izvode na trenutno aktivnoj implementaciji. Sve ostale implementacije ostaju nepromenjene.



Slika 7.25. Izgled Vivado HLS glavnog prozora nakon kreiranja nove implementacije, *solution2*

Nakon što smo kreirali novu implementaciju (*solution2*) i sačuvali rezultate inicijalne implementacije (*solution1*) za kasnije poređenje, možemo preći na definisanje optimizacionih direktiva u cilju poboljšavanja performansi ostvarenih pomoću prethodne implementacije (u našem slučaju to bi bila *solution1* implementacija).

7.6. Definisanje optimizacionih direktiva

U ovom poglavlju predstavimo najvažnije optimizacione direktive koje nam stoje na raspolaganju u procesu generisanja implementacije sa potrebnim karakteristikama. Tabela 7.1 prikazuje sve optimizacione direktive koje podržava Vivado HLS alat.

Optimizaciona direktiva	Opis
ALLOCATION	Specificira gornji limit na broj operacija, IP jezgara ili funkcija koje se koriste. Na ovaj način se može forsirati deljenje hardverskih resursa, ali se istovremeno može povećati kašnjenje.
ARRAY_MAP	Kombinuje veći broj manjih nizova u jedan veliki niz, u cilju redukcije potrebnog broja BlockRAM memorija.
ARRAY_PARTITION	Deli niz na veći broj manjih nizova ili na individualne registre, u cilju poboljšanja pristupa elementima niza i uklanjanja uskih grla prilikom korišćenja BlockRAM memorija.
ARRAY_RESHAPE	Reorganizuje niz sa velikim brojem elemenata u niz sa malim brojem elemenata, kod kojega svaki element predstavlja konkatenaciju većeg broja elemenata početnog niza. Korisno kod poboljšanja pristupa BlockRAM memorijama bez potrebe za korišćenjem većeg broja BlockRAM memorija.
DATA_PACK	Pakuje polja odabrane strukture podataka u jedan skalarni objekat koji predstavlja njihovu konkatenaciju.
DATAFLOW	Koristi protočnu obradu podataka na nivou zadatka, omogućavajući da se funkcije i petlje izvršavaju konkurentno. Koristi se da se minimizuje inicijalizacioni interval.
DEPENDENCE	Koristi se za pružanje dodatnih informacija o međuzavisnosti operacija prisutnih u algoritamskom modelu. Ovo je posebno korisno za definisanje međuzavisnosti operacija koje se nalaze unutar petlji, što zatim može olakšati njihovu implementaciju korišćenjem protočne obrade.
EXPRESSION_BALANCE	Omogućava da se automatsko balansiranje složenih izraza aktivira ili deaktivira.
FUNCTION_INSTANTIATE	Omogućava da se različite instance iste funkcije optimizuju lokalno (individualno).
INLINE	Uklanja hijerarhiju unutar obuhvatajuće funkcije zamenjujući poziv <i>inline</i> funkcije sa njenim telom. Na ovaj način omogućava se optimizacija logike preko postojećih granica između funkcija.

	Istovremeno se i umanjuje kašnjenje/inicijalizacioni interval obuhvatajuće funkcije, jer se na ovaj način uklanja vreme potrebno za poziv i povratak iz <i>inline</i> funkcije.
INTERFACE	Specificira na koji način će argumenti funkcije koja se sintetizuje biti mapirani na portove RTL modela.
LATENCY	Omogućava da se definiše minimalno i maksimalno kašnjenje u izvršavanju operacije, petlje, funkcije, itd.
LOOP_FLATTEN	Omogućava da se ugnježdene petlje zamene sa jednom novom petljom, u cilju smanjenja kašnjenja prilikom izvršavanja.
LOOP_MERGE	Omogućava da se susedne petlje spoje u jednu kako bi se smanjilo kašnjenje prilikom njihovog izvršavanja, povećala mogućnost za deljenje hardverskih resursa i poboljšali efekti optimizacije logike.
LOOP_TRIPCOUNT	Koristi se kod petlji sa promenljivim brojem iteracija. Pruža estimaciju broja iteracija kroz petlju. Ovaj podatak se ne koristi prilikom optimizacija u procesu sinteze, već se samo koristi u izveštaju nakon završene sinteze za tačniju procenu kašnjenja u izvršavanju algoritamskog modela.
OCCURENCE	Koristi se prilikom uvođenja protočne obrade unutar funkcija ili petlji da specificira da se deo algoritamskog modela izvršava manjom brzinom od okolnog dela modela.
PIPELINE	Koristi se za redukciju inicijalizacionog intervala dozvoljavajući da se operacije unutar petlji ili funkcija izvršavaju konkurentno.
PROTOCOL	Pomoću ove direktive odabrani region unutar algoritamskog modela proglašava se za protokol region. Protokol region se zatim može iskoristiti za manuelnu specifikaciju specifičnog protokola koji je potrebno koristiti prilikom razmene podataka.
RESET	Ova direktiva koristi se za dodavanje ili uklanjanje reset signala na specificiranoj varijabli unutar algoritamskog modela.
RESOURCE	Definiše koje IP jezgro (iz skupa raspoloživih IP jezgara koja Vivado HLS podržava) treba da se koristi za implementaciju odabranog objekta (niza, aritmetičke operacije ili argumenta funkcije) iz algoritamskog modela unutar RTL modela.
STREAM	Definiše da je odabrani niz potrebno implementirati kao FIFO ili RAM memorijski kanal prilikom optimizacija toka podataka.

UNROLL	Razmotava odabranu petlju kako bi se povećala mogućnost za konkurentno izvršavanje operacija unutar petlje.
--------	---

Tabela 7.1. Optimizacione direktive Vivado HLS alata

Optimizacione direktive navedene u tabeli 7.1 mogu se primenjivati na kompatibilne objekte unutar algoritamskog modela sistema. Vivado HLS alat automatski limitira skup raspoloživih optimizacionih direktiva koje se mogu primeniti na trenutno selektovani objekat. U opštem slučaju optimizacione direktive mogu se primeniti na sledeće objekte algoritamskog modela sistema:

- **Interfejse** – kada se optimizacione direktive odnosne na interfejs sistema, Vivado HLS alat ih primenjuje na funkciju na najvišem nivou hijerarhije unutar algoritamskog modela sistema (funkciju od koje započinje sinteza visokog nivoa), jer upravo ova funkcija sadrži argumente koji će činiti budući interfejs projektovanog RTL modela.
- **Funkcije** – kada se direktiva primeni na neku od funkcija unutar algoritamskog modela, Vivado HLS alat je primenjuje na sve objekte koji se nalaze unutar posmatrane funkcije. Efekat bilo koje direktive prestaje na funkcijama koje se nalaze na nižem nivou hijerarhije (koje se pozivaju unutar posmatrane funkcije), odnosno ne primenjuju se rekurzivno do najnižeg hijerarhijskog nivoa. Jedini izuzetak predstavljaju direktive koje podržavaju rekurzivnu primenu, kao što je PIPELINE direktiva.
- **Petlje** – kada se direktiva primeni na petlju, Vivado HLS alat je primenjuje na sve objekte koji se nalaze unutar petlje.
- **Nizove** – kada se direktiva primeni na niz, Vivado HLS alat je primenjuje na čitav region koji sadrži posmatrani niz.
- **Region** – kada se direktiva primeni na odabrani region, Vivado HLS alat je primenjuje na čitav region. Pod regionom podrazumevamo bilo koju oblast unutar algoritamskog modela uokvirenu sa vitičastim zgradama, {}.

U nastavku će ukratko biti opisane standardne optimizacije i tehnike koje se mogu koristiti u cilju poboljšanja performansi i veličine automatski generisanog digitalnog sistema pomoću Vivado HLS alata.

7.6.1. Definisanje klok i reset signala

Kao i svaki drugi sekvencijalni sistem automatski generisani RTL model mora da sadrži barem jedan klok i jedan reset ulazni port. Kako ova dva signala nikada nisu eksplicitno deklarirana unutar algoritamskog modela sistema (jer na algoritamskom nivou

apstrakcije ne postoji koncept kloka i reseta), Vivado HLS mora da ih generiše automatski.

U slučaju algoritamskih modela napisanih u C i C++ jeziku, Vivado HLS alat će generisati RTL model sa tačno jednim klok signalom, koji će biti distribuiran na ulaze svih sinhronih mreža unutar modela. U slučaju SystemC modela, svaki SC_MODULE objekat može definisati svoj klok.

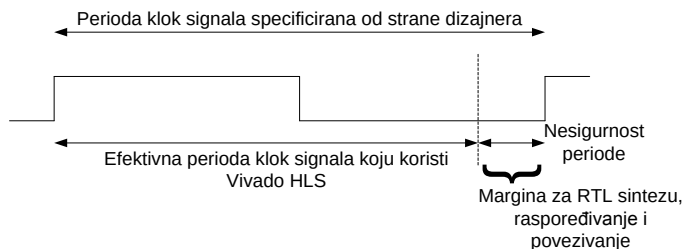
Ono što dizajner mora specificirati jeste potrebni period klok signala, ili ekvivalentno, potrebnu učestanost klok signala, na kojoj sistem koji se automatski generiše mora da bude sposoban da radi bez grešaka. O ovome je već bilo reči, prilikom opisivanja postupka kreiranja novog projekta, u poglavlju 7.2 (videti deo posvećen „*Solution Configuration*“ prozoru). Alternativno, dizajner u svakom trenutku može promeniti željenu periodu klok signala, izborom opcije *Solutions*→*Solution Setting*.

Vivado HLS alat se prilikom vremenskog planiranja operacija trudi da što veći broj operacija planira za izvršavanje u istom periodu klok signala (*Operator Chaining*), naravno ukoliko je tako nešto moguće. Ukoliko je perioda klok signala previše mala da omogućiti ovako nešto, Vivado HLS će u tom slučaju planirati izvršavanje operacija koje se proteže na veći broj sukcesivnih perioda klok signala (*Multicycle Operation*).

Vivado HLS koristi koncept nesigurnosti periode klok signala digitalnog sistema koji se projektuje, kako bi dizajneru omogućio da definiše izvesnu marginu prilikom specificiranja potrebne periode klok signala. Vivado HLS alat koristi informacije o vremenskim karakteristikama elemenata ciljne FPGA komponente, kako bi estimirao kašnjenja operacija pristunih u algoritamskom modelu. Međutim, kako Vivado HLS ne može znati kako će hardverski resursi, neophodni za implementaciju ovih operacija, na kraju biti raspoređeni i povezani po FPGA komponenti, on ne može imati precizne informacije o stvarnim kašnjenjima unutrašnjih signala digitalnog sistema koji se projektuje. Ove informacije će biti poznate tek nakon što se završi faza raspoređivanja i povezivanja. Koncept nesigurnosti zahtevane periode klok signala omogućava dizajneru da se na neki način osigura da će automatski generisani RTL model u procesu sinteze visokog nivoa ipak raditi na potrebnoj učestanosti i nakon što se završi faza fizičke implementacije.

Vivado HLS prilikom generisanja RTL modela kao efektivnu periodu klok signala koristi periodu koja se dobija oduzimanjem definisane nesigurnosti periode klok signala od specificirane periode klok signala, kao što je prikazano na slici 7.26.

Ovako definisana margina omogućava da se nakon što Vivado HLS alat završi sa automatskim generisanjem RTL modela, ovaj model uspešno implementira pomoću Vivado Design Suite alata i da se na osnovu njega uspešno generiše konfiguracioni fajl za programiranje FPGA komponente.



Slika 7.26. Način određivanja efektivne periode klok signala od strane Vivado HLS alata

Vivado HLS alat automatski uključuje i reset ulazni port, pomoću kojega se unutrašnji registri i BlockRAM memorije mogu inicijalizovati na njihovu specificiranu početnu vrednost. Ponašanje reset ulaznog porta može se promeniti pomoću komande *Solution*→*Solution Setting*→*General*→*Add*→*confir_rtl*. Pomoću ove komande mogu se podesiti željene karakteristike reset signala, njegov polaritet, da li je sinhroni ili asinhroni, kao i koji registri će biti povezani na reset ulazni port. Fina kontrola reset signala, u smislu da li neka promenljiva, deklarirana unutar algoritamskog modela sistema, treba da se inicijalizuje pomoću reset ulaznog porta ili ne, može se ostvariti pomoću RESET optimizacione direktive.

7.6.2. Definisanje ulazno-izlaznog interfejsa RTL modela

Unutar algoritamskog modela sve ulazne i izlazne operacije izvršavaju se u nultom vremenu, preko formalnih argumenata funkcije na najvišem nivou hijerarhije modela. Unutar RTL modela ove iste ulazne i izlazne operacije moraju se izvršiti preko odgovarajućih interfejs portova i uglavnom se moraju odvijati poštujući neki ulazno/izlazni (I/O) protokol. Vivado HLS alat omogućava dva načina za definisanje potrebnog interfejsa kao i I/O protokola:

- Automatskim kreiranjem interfejsa, korišćenjem jednog od industrijski standardnih protokola koje Vivado HLS alat podržava (*Interface Synthesis*). Ovo je preporučeni način definisanja ulazno/izlaznog interfejsa, koji se najčešće koristi.
- Manuelnim definisanjem željenog interfejsa i I/O protokola unutar algoritamskog modela (*Manual Interface Specification*). Na ovaj način dizajner je u mogućnosti da specificira proizvoljni I/O protokol, pa čak i da kreira potpuno novi, koji želi koristiti.

Kao što je već rečeno, automatsko kreiranje interfejsa je preporučeni vid kreiranja interfejsa, pa će u nastavku ovaj način biti detaljnije objašnjen.

Kada Vivado HLS sintetizuje funkciju algoritamskog modela koja se nalazi na najvišem nivou hijerarhije, svi argumenti (parametri) funkcije zamenjuju se odgovarajućim brojem ulaznih i izlaznih portova unutar RTL modela. Ovaj postupak unutar Vivado HLS alata naziva se **sinteza interfejsa** (*Interface Synthesis*).

U procesu sinteze interfejsa Vivado HLS alat kreira tri vrste portova unutar RTL modela:

- **Klok i reset portove** – ovi portovi su uvek pristuni i imaju sledeća imena: *ap_clk* i *ap_rst*.
- **Blok interfejs** – interfejs na nivou celog modula koji služi za kontrolisanje modula od strane njegovog okruženja (*Block-Level Interface*).
- **Port interfejse** – interfejsi za svaki od argumenata algoritamskog modela (*Port-Level Interface*).

Blok interfejs

Blok interfejs se automatski dodaje prilikom sinteze svakog algoritamskog modela. On se sastoji iz grupe signala koji omogućavaju upravljanje i nadgledanje rada projektovanog sistema. Blok interfejs uvek sadrži odgovarajući broj komandnih i statusnih signala, pomoću kojih okolna logika može da upravlja radom sintetizovanog modula. Blok interfejs uvek sadrži sledeći signale:

- signal koji signalizira da li je modul spreman za prihvatanje novih podataka,
- signal koji signalizira da je tekuća operacija završena,
- signal koji signalizira da je modul u stanju neaktivnosti i da je spreman da započne izvršavanje sledeće komande.

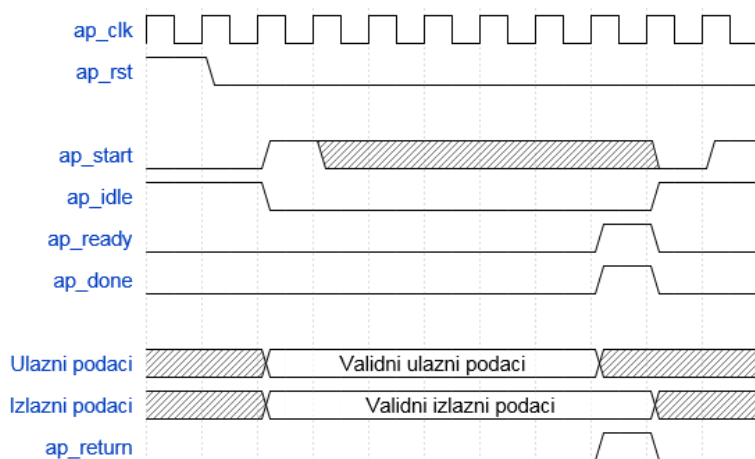
Vivado HLS alat pruža veći broj različitih blok interfejsa, koje korisnik može odabrati kao blok interfejs unutar svog dizajna:

- ***ap_ctrl_none*** – ovo je najjednostavniji vid kontrolnog interfejsa, kod kojega ne postoji niti jedan kontrolni niti statusni port.
- ***ap_ctrl_hs*** – ovo je podrazumevani kontrolni interfejs koji Vivado HLS alat koristi prilikom sinteze svakog algoritamskog modela. Sastoji se iz četiri porta, *ap_start*, *ap_idle*, *ap_ready* i *ap_done*.
- ***ap_ctrl_chain*** – ovo je protokol koji je sličan *ap_ctrl_hs* protokolu ali sadrži dodatni port *ap_continue*, koji omogućava jednostavno kaskadno vezivanje modula.

Podrazumevani blok interfejs *ap_ctrl_hs*, sastoji se iz sledećih portova:

- ***ap_start*** – ovaj ulazni port kontroliše započinjanje izvršavanja sledeće komande od strane RTL modula. Mora se postaviti na vrednost 1 da bi modul započeo izvršavanje sledeće komande. Treba da se drži na jedinici sve dok *ap_ready* signal ne postane 1. Kada *ap_ready* postane visok, *ap_start* može da ostane visok i na taj način započne izvršavanje sledeće transakcije čim se tekuća završi, ili se može spustiti na nulu, što će izazvati zaustavljanje modula nakon završetka izvršavanja tekuće transakcije. Ukoliko *ap_start* postane 0 pre nego što *ap_ready* postane 1, može doći do nekorektnog rada sistema, pa čak i do ignorisanja zadate komande.
- ***ap_ready*** – ovaj izlazni port služi za indicaciju da je RTL modul spreman za prihvatanje novih podataka. Kada se *ap_ready* postavi na 1, RTL modul je spreman za prihvatanje novih podataka, što predstavlja indicaciju da su svi podaci iz prethodne transakcije uspešno učitani. Ovaj signal se koristi da se donese odluka kada treba primeniti nove vrednosti na ulaze podataka RTL modula i kada treba započeti sledeću transakciju, podizanjem *ap_start* porta na 1. Ako je *ap_start* port na niskom nivou, *ap_ready* signal takođe pada na 0, kada RTL model završi sve operacije vezane za tekuću transakciju.
- ***ap_done*** – ovaj izlazni port koristi se da obavesti okruženje da je RTL modul završio sve operacije vezane za tekuću transakciju. Kada je na logičkoj jedinici to predstavlja indicaciju da je RTL modul završio sve operacije u tekućoj transakciji. To ujedno predstavlja i završetak tekuće transakcije, kada je *ap_done* na visokom logičkom nivou to ujedno znači i da su podaci na izlaznom *ap_return* portu validni. Port *ap_return* postoji samo u slučaju kada funkcija koja se implementira u hardveru vraća povratnu vrednost.
- ***ap_idle*** – ovaj izlazni port služi za indicaciju da li je RTL modul trenutno aktivan (izvršava tekuću transakciju) ili se nalazi u stanju mirovanja (*idle*). Kada je RTL modul neaktivan *ap_idle* port ima vrednost 1. Kada RTL modul započne izvršavanje tekuće transakcije, *ap_idle* port postaje 0.

Radi boljeg i lakšeg razumevanja *ap_ctrl_hs* protokola, slika 7.27 prikazuje tipične talasne oblike signala *ap_ctrl_hs* protokola.



Slika 7.27. Ponašanje signala *ap_ctrl_hs* protokola

Nakon deaktiviranja reset signala (*ap_rst*) odvija se sledeća sekvenca događaja na *ap_ctrl_hs* interfejsu:

1. RTL modul čeka da *ap_start* port postane visok, što predstavlja indicaciju da treba započeti izvršavanje sledeće transakcije.
2. Izlazni port *ap_idle* postaje 0, neposredno nakon što *ap_start* postane visok, što predstavlja indicaciju da RTL model više nije u neaktivnom stanju.
3. *ap_start* port može i dalje da ostane na visokom logičkom nivou, ili se može vratiti u stanje logičke nule. RTL model će proveriti stanje *ap_start* porta u trenutku kada port *ap_ready* postane 1.
4. Podaci se mogu razmenjivati između RTL modula i njegovog okruženja preko ulaznih i izlaznih portova podataka, koristeći odgovarajuće port interfejse.
5. Kada RTL modul završi obradu tekuće transakcije, izlazni port *ap_done* postaje 1. U slučaju da RTL modul poseduje i *ap_return* izlazni port, podaci koji se trenutno nalaze na njemu su takođe validni dokle god je *ap_done* na visokom nivou.
6. Kada je RTL modul spreman da prihvati nove ulazne podatke, *ap_ready* port opet postaje 1. Ponašanje *ap_ready* porta zavisi od načina kako je RTL modul implementiran:
 - *ap_ready* port je neaktivan sve dok RTL modul ne započne izvršavanje transakcije.
 - U slučaju da RTL modul ne koristi protočnu obradu, *ap_ready* postaje 1 u trenutku kada *ap_done* postaje 1.

- U slučaju da RTL modul koristi protočnu obradu, *ap_ready* može postati 1 u bilo kom periodu klock signala, nakon što *ap_start* postane 1. Tačni trenuci u kojima će se to desiti zavise od načina kako je izvedena protočna obrada unutar RTL modula.
- Ako je *ap_start* 0 kada je *ap_ready* 1, RTL modul nastavlja sa radom sve dok *ap_done* ne postane 1, kada prestaje sa radom.
- Ako je *ap_start* 1 kada je *ap_ready* 1, sledeća transakcija započinje odmah, i RTL modul ne prekida sa svojim radom.

7. *ap_idle* port predstavlja indikaciju da li je RTL modul aktivan ili neaktivan:

- Ako je *ap_start* 0 kada je *ap_ready* 1, RTL modul prestaje sa radom i *ap_idle* port postaje 1 jedan takt nakon što *ap_done* postane 1.
- Ako je *ap_start* 1 kada je *ap_ready* 1, RTL modul nastavlja sa radom, a *ap_idle* ostaje 0.

Port interfejsi

Port interfejsi se kreiraju za svaki ulazni i izlazni argument algoritamskog modela sistema. Korišćeni I/O protokol na svakom interfejsu zavisi od tipa argumenta algoritamskog modela. Rasploživi I/O protokoli za svaki tip argumenta algoritamskog modela, kao i podrazumevani I/O protokol prikazani su u sledećoj tabeli.

Tip argumenta	Skalarni tip		Niz			Pointer ili referenca			HLS:: Stream
I/O protokol	Ulaz	Return	I	I/O	O	I	I/O	O	I i O
ap_ctrl_none									
ap_ctrl_hs		D							
ap_ctrl_chain									
axis									
s_axilite									
m_axi									
ap_none	D					D			
ap_stable									
ap_ack									
ap_vld								D	
ap_ovld							D		
ap_hs									
ap_memory			D	D	D				
bram									
ap_fifo									
ap_bus									

Tabela 7.2. Rasploživi port interfejs protokoli u zavisnosti od tipa argumenta:
(D – podrazumevani protokol; podržan protokol ■)

Blok protokoli *ap_ctrl_none*, *ap_ctrl_hs* i *ap_ctrl_chain* diskutovani su ranije, te ih na ovom mestu nećemo dalje analizirati.

Sledeću grupu I/O protokola čine AXI4 protokoli: AXI-Stream protokol *axis*, AXI-Lite protokol *s_axilite* i AXI-Full protokol *m_axi*. AXI-Stream protokol *axis* može se koristiti na ulaznim ili izlaznim argumentima, a ne može se koristiti na ulazno/izlaznim argumentima. AXI-Lite protokol *s_axilite* može se koristiti sa bilo kojim tipom argumenata, bilo kog smera, osim na argumentima koji su tipa niz. Moguće je grupisati veći broj argumenata u jedan AXI-Lite interfejs. AXI-Full master protokol *m_axi* može se koristiti na argumentima koji su deklarirani kao nizovi ili pokazivači (odnosno reference u C++). Takođe je moguće grupisanje većeg broja ovakvih argumenata u jedan AXI-Full interfejs.

Grupu protokola kod kojih zapravo ne postoje nikakvi dodatni kontrolni signali čine *ap_none* i *ap_stable* I/O protokoli. Protokol *ap_none* je podrazumevani protokol za skalarne ulazne argumente, dok je *ap_stable* protokol namenjen za implementaciju konfiguracionih portova, čije vrednosti se menjaju samo kada je RTL modul u režimu reseta.

Narednu grupu I/O protokola čine *ap_ack*, *ap_vld*, *ap_ovld* i *ap_hs* protokoli. Kod ovih protokola uvek postoje dodatni kontrolni signali. Protokol *ap_hs* sadrži dva dodatna porta, *valid* i *acknowledge* koji omogućavaju standardnu *handshake* komunikaciju. Ovaj protokol može se koristiti i kada je argument tipa niz, ali pristup elementima niza mora biti sekvencijalan. Protokol *ap_vld* sličan je sa *ap_hs* protokolom, ali kod njega postoji samo *valid* kontrolni signal. Slično je i sa *ap_ack* protokolom, ali ovaj protokol sadrži samo *acknowledge* signal, a nema *valid* signal. Protokol *ap_ovld* koristi se za implementaciju ulazno/izlaznih argumenata. Vivado HLS ulazno/izlazne argumente uvek razdvaja u ulazni i izlazni par argumenata. Ulazni argument realizuje se korišćen *ap_none* protokola, dok se izlazni argument realizuje korišćenjem *ap_vld* protokola. Ovaj protokol predstavlja standardni način za implementaciju argumenata tipa pokazivač.

Vivado HLS alat sadrži i grupu I/O protokola koji služe za implementaciju memorijskih interfejsa. Ovi grupu protokola čine *ap_memory*, *bram* i *ap_fifo* I/O protokoli, i obično se koriste za implementaciju argumenata koji su deklarirani kao nizovi. Protokol *ap_memory* predstavlja standardni memorijski protokol za pristup memorijama sa sinhronim upisom i čitanjem (to su BlockRAM memorije unutar Xilinx FPGA komponenti) i uvek se sastoji iz adresnog porta (*address*) i porta dozvole rada (*ce*). Ukoliko je argument algoritamskog modela koji se mapira u RTL model ulazni, onda *ap_memory* protokol sadrži i ulazni port podataka (*q*), a ako je argument izlazni, onda *ap_memory* protokol sadrži izlazni port podataka (*d*) i port za dozvolu upisa (*we*). Protokol *bram* je identičan sa *ap_memory* protokolom, jedino se razlikuje način kako se ovi protokoli prikazuju unutar Vivado Design Suite alata (konkretno unutar *IP Integrator* alata). Protokol *ap_memory* prikazuje se kao skup odvojenih ulaznih i izlaznih portova, dok se *bram* protokol prikazuje kao standardni BlockRAM memorijski interfejs, omogućavajući jednostavno povezivanje unutar većih sistema. Ukoliko se elementima niza pristupa u sekvencijalnom redosledu, može se koristiti *ap_fifo* protokol. Ovaj protokol može se koristiti za implementaciju ulaznih ili izlaznih argumenata, ali ne i ulazno/izlaznih argumenata tipa niz.

Poslednji protokol koji stoji na raspolaganju je *ap_bus*. Ovaj protokol može se koristiti za ostvarivanje komunikacije sa mrežnim kontrolerom koji može biti prisutan u sistemu. Ovaj protokol ne pridržava se ni jednog standardnog protokola systemske magistrale, već je dovoljno generički tako da se relativno lako može koristiti za povezivanje sa različitim mrežnim kontrolerima.

Ukoliko nije drugačije naznačeno od strane dizajnera, Vivado HLS alat koristi sledeće I/O protokole za implementaciju različitih tipova argumenata algoritamskog modela:

- Podrazumevani način implementaciju ulaznih skalarnih tipova kao i pokazivača jeste pomoću običnog porta bez ikakvih dodatnih kontrolnih signala (*ap_none* protokol).
- Izlazni pokazivači se implementiraju pomoću izlaznog porta sa dodatnim izlaznim kontrolnim portom koji signalizira kada su podaci na izlaznom portu validni (*ap_vld* protokol).
- Nizovi se uvek implementiraju korišćenjem standardnog memorijskog interfejsa (*ap_memory* protokol), koji sadrži adresni port, port podataka i port dozvole pristupa memoriji.
- Ukoliko algoritamski model koji se sintetizuje sadrži ulazno/izlazne argumente, oni se u RTL modelu implementiraju preko razdvojenih ulaznih i izlaznih portova sa odgovarajućim podrazumevanim I/O protokolima.
- Ukoliko algoritamski model vraća povratnu vrednost, Vivado HLS automatski generiše *ap_return* izlazni port, preko koga se okruženju prosleđuje povratna vrednost. Kada RTL model kompletira jednu transakciju, što odgovara jednom izvršavanju algoritamskog modela (na primer C funkcije), asocirani podrazumevani blok protokol *ap_ctrl_hs* će indicirati da je tekuća transakcija kompletirana, podizanjem *ap_done* izlaznog porta. Visoka vrednost *ap_done* porta istovremeno predstavlja i indikaciju da je tekuća vrednost *ap_return* izlaznog porta validna.

NAPOMENA: Ukoliko dizajner želi da promeni korišćeni I/O protokol na nekom od argumenata algoritamskog modela, može to uraditi korišćenjem INTERFACE direktive na odabranom argumentu.

Kako bismo ilustrovali način na koji Vivado HLS vrši sintezu interfejsa posmatrajmo sledeći algoritamski model hipotetičkog sistema.

```

uint9 mac_and_add (uint8 a, uint8 b, uint20 *mac)
{
    uint9 temp;

    for (int i=0; i < 4; i++)
        *mac = a * b + *mac;

    temp = a + b;

    return temp;
}

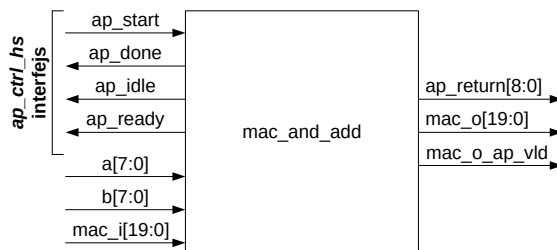
```

Slika 7.28. Algoritamski model hipotetičkog sistema

Algoritamski model sa slike 7.28 sadrži sledeće argumente:

- Dva skalarna ulazna argumenta, *a* i *b*.
- Jedan ulazno/izlazni argument, *mac*, koji je tipa pokazivač.
- Model takođe vraća i povratnu vrednost, koja je jednaka vrednosti unutrašnje promenljive *temp*.

Ukoliko bi Vivado HLS alat izvršio sintezu interfejsa za algoritamski model sa slike 7.28, uz korišćenje podrazumevanih I/O protokola, sintetizovani RTL model imao bi portove ka što je prikazano na slici 7.29.



Slika 7.29. Portovi RTL modela nakon sinteze interfejsa sa podrazumevanim podešavanjima I/O protokola

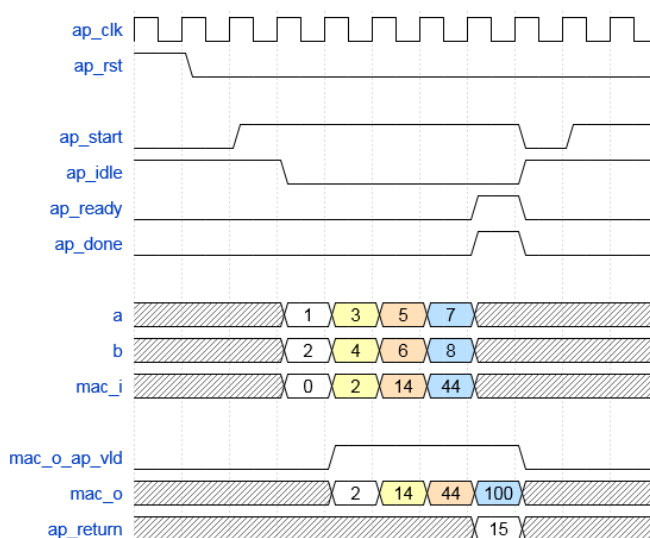
Vivado HLS alat automatski generiše podrazumevani blok interfejs, *ap_ctrl_hs*, koji se sastoji iz *ap_start*, *ap_done*, *ap_idle* i *ap_ready* signala. Dva skalarna ulazna argumenta algoritamskog modela, *a* i *b*, realizuju se pomoću *ap_none* protokola, koji se sastoji samo od jednog ulaznog porta. Zbog toga se svakom od njih pridružuje po jedan ulazni port odgovarajuće širine (koja zavisi od definicije skalarnog tipa kome pripada

argument). U našem primeru, ulazni argumenti *a* i *b* deklarirani su kao tip *uint8*, pa se svakom od njih pridružuje po jedan 8-bitni ulazni *std_logic_vector* port, *a*[7:0] i *b*[7:0].

Ulazno/izlazni argument *mac* mapira se u dva odvojena porta odgovarajuće širine koja koriste *ap_ovld* protokol. Kako je argument *mac* deklarisan kao tip *uint20*, on će biti mapiran u jedan ulazni 20-bitni *std_logic_vector* port, *mac_i*[19:0], i jedan izlazni 20-bitni *std_logic_vector* port, *mac_o*[19:0]. Pored izlaznog porta *mac_o*[19:0] biće generisan i još jedan dodatni port, *mac_o_ap_vld*, koji predstavlja indikaciju validnosti podatka na *mac_o*[19:0] izlaznom portu, kao deo *ap_ovld* I/O protokola.

Na kraju, pošto algoritamski model sa slike 7.28 vraća povratnu vrednost, tipa *uint9*, automatski će biti generisan i 9-bitni *std_logic_vector* izlazni port *ap_return*[19:0], preko kojega će biti prosleđena povratna vrednost na kraju obrade svake transakcije.

Na slici 7.30 prikazan je i izgled talasnih oblika svih signala koji čine interfejs RTL modela hipotetičkog sistema sa slike 7.28.



Slika 7.30. Talasni oblici signala koji čine interfejs RTL modela hipotetičkog sistema

Obrada jedne transakcije prikazana je na slici 7.30 i sastoji se iz sledećih faza:

1. RTL modul započinje sa obradom sledeće transakcije kada ulazni port *ap_start* postane 1.
2. Izlazni port *ap_idle* postaje 0, kao indikacija da je RTL modul aktivan obradom tekuće transakcije.
3. Ulazni podaci se čitaju u trenucima koji zavise od načina na koji su planirane operacije unutar RTL modula. Kako se za prenos podataka *a* i *b* ne koriste nikakvi kontrolni signali, okolna logika mora znati u kojim trenucima je

potrebno dovoditi nove vrednosti na ulazne portove *a* i *b*. U primeru sa slike 7.30, RTL modul očekuje nove podatke na portovima *a* i *b* nakon svakog takta, od trenutka kada započne sa obradom transakcije.

4. Nakon što se učitaju svi potrebni podaci, *ap_ready* signal postaje 1.
5. Kako se računaju nove vrednosti sume proizvoda, one se pojavljuju na izlaznom portu *mac_o*. Izlazni port *mac_o_ap_vld* pruža indikaciju kada su ove vrednosti validne.
6. Kada RTL modul završi sa obradom tekuće transakcije, izlazni port *ap_done* postaje 1. Ovo takođe predstavlja indikaciju su podaci na izlaznom portu *ap_return* validni.
7. Port *ap_idle* ponovo postaje 1, kao indikacija da je RTL modul završio sa obradom tekuće transakcije i da je spreman za prihvatanje naredne.

7.6.3. Optimizacije u cilju povećanja propusne moći sistema

U cilju povećanja propusne moći (*throughput*) RTL sistema koji se generiše u postupku sinteze visokog nivoa pomoću Vivado HSL alata, moguće je koristiti sledeće optimizacione direktive:

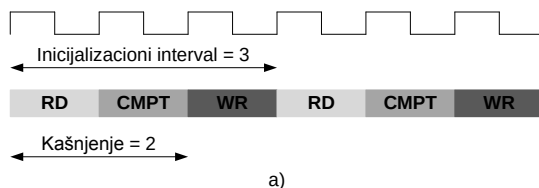
- Uvođenje protočne obrade podataka u sistem (*Task Pipelining*), bilo na nivou funkcija ili na nivou petlji.
- Podelu nizova (*Array Partitioning*), kako bi se obezbedio efikasniji tok podataka ka i od sistema.
- Razmotavanje petlji (*Loop Unrolling*), kako bi se povećao stepen paralelizma u izvođenju operacija.

Uvođenje protočne obrade podataka u sistem (*Task Pipelining*)

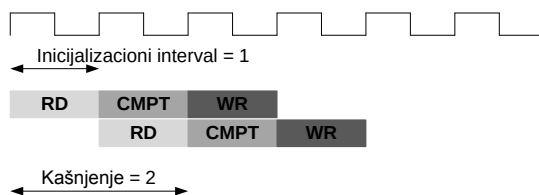
Kao što je poznato, protočna obrada povećava propusnu moć sistema dozvoljavajući da se operacije izvode konkurentno, na različitim instancama ulaznih podataka. Protočna obrada unutar Vivado HLS alata može se primeniti na funkcije i na petlje. Slika 7.31 prikazuje efekte uvođenja protočne obrade prilikom izvršavanja funkcije.

Bez primene protočne obrade funkcija čita ulazne argumente svaka 3 takta, a generiše izlaze svaka 2 takta (vidi sliku 7.31a). Na osnovu ove observacije može se zaključiti da funkcija ima inicijalizacioni interval (II) od 3 takta, a kašnjenje od 2 takta. Primenom protočne obrade novi podaci učitavaju se svakog takta (II=1), bez promene u kašnjenju (vidi sliku 7.31b). Obratite takođe da se i broj hardverskih resursa potrebnih za implementaciju sistema nakon uvođenja protočne obrade nije promenio.

```
void func(...) {
    op_Read;   RD
    op_Compute; CMPT
    op_Write;  WR
}
```



a)



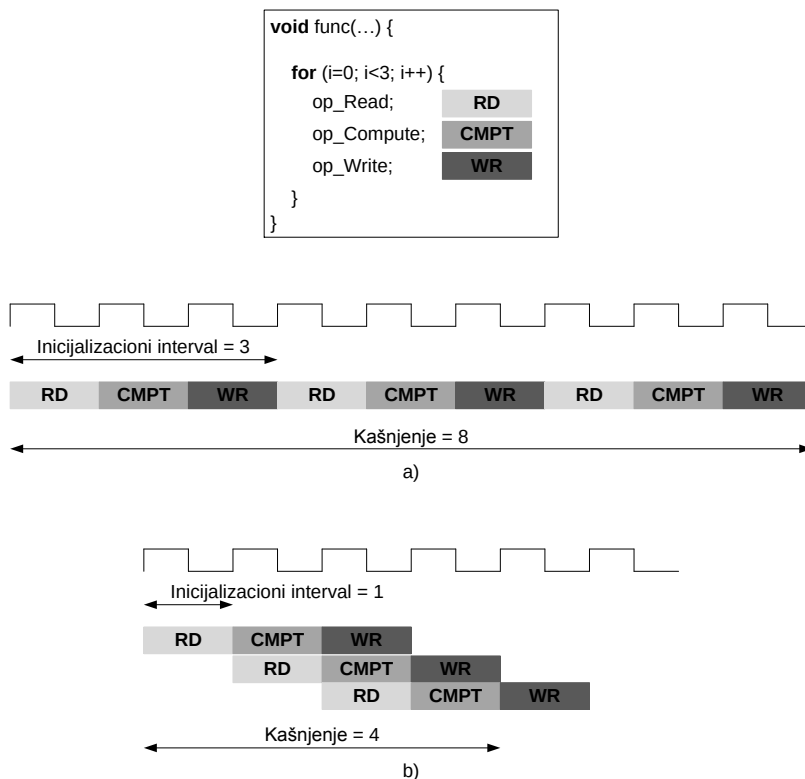
b)

Slika 7.31. Efekti protočne obrade prilikom izvršavanja funkcije
a) performanse sistema bez korišćenja protočne obrade
b) performanse sistema sa protočnom obradom

Uvođenje protočne obrade u izvršavanje tela petlje omogućava da se operacije iz tela izvode konkurentno, na različitim podacima, kao što je ilustrovano na slici 7.32.

Bez primene protočne obrade, telo petlje izvršava se na sekvencijalni način, kao što je prikazano na slici 7.32a, gde je potrebno da protekne 3 takta između čitanja sukcesivnih podataka (ili drugačije rečeno, jedna iteracija petlje traje 3 takta), što znači da je inicijalizacioni interval petlje jednak 3 takta, $II=3$. Sa druge strane, vreme potrebno za izvršavanje čitave petlje iznosi ukupno 8 taktova, što ujedno predstavlja i kašnjenje prilikom izvršavanja petlje (*latency*).

Ukoliko se primeni tehnika protočne obrade, onda dolazi do preklapanja izvršavanja operacija iz sukcesivnih iteracija petlje, kao što je prikazano na slici 7.32b. U ovom slučaju inicijalizacioni interval iznosi 1 takt, $II=1$, jer se novi podatak čita u svakoj periodi klock signala. Obratite pažnju da se uvođenjem protočne obrade smanjilo i kašnjenje prilikom izvršavanja petlje, koje sada iznosi 4 takta. Vidimo da je došlo do značajnog poboljšanja performansi sistema, bez povećavanja potrebnih hardverskih resursa za njegovu implementaciju.



Slika 7.32. Efekti protočne obrade prilikom izvršavanja petlje

U Vivado HLS alatu protočna obrada se uvodi u sistem korišćenjem optimizacione direktive PIPELINE. Podrazumevani inicijalizacioni interval koji je potrebno ostvariti iznosi 1 takt, ukoliko dizajner nije definisao drugačije.

Kada se PIPELINE direktiva primeni na odabranu funkciju ili petlju unutar algoritamskog modela, ona se automatski primenjuje i na sve petlje koje se nalaze unutar posmatrane funkcije ili petlje. Međutim, ova direktiva se automatski **NE PRIMENJUJE** i na funkcije koje se pozivaju unutar posmatrane funkcije ili petlje! Ako želimo da uvedemo protočnu obradu i unutar ovih funkcija, moramo to eksplicitno navesti, korišćenjem odgovarajućeg broja dodatnih PIPELINE direktiva.

Podela nizova (Array Partitioning)

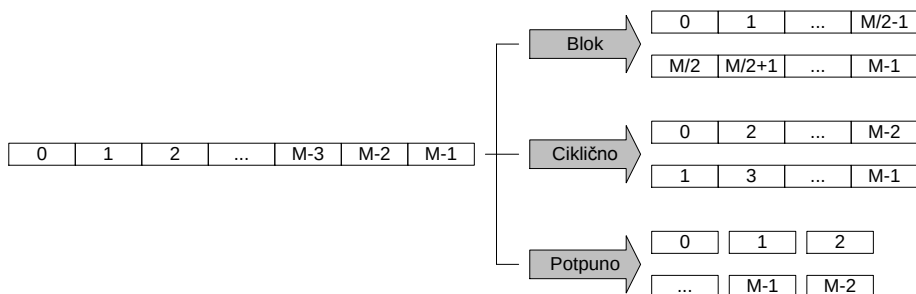
Kako se korišćenjem protočne obrade podataka povećava propusna moć sistema, vrlo često se dešava da postojeći interfejs za prenos podataka jednostavno nema dovoljnu propusnu moć da prenese sve potrebne podatke do sistema za obradu podataka. U ovom slučaju sistem za protočnu obradu radi ispod svojih mogućnosti, pa su i efekti uvođenja protočne obrade su limitirani. Ovaj slučaj redovno se javlja kada Vivado HLS alat ima

potrebu da implementira pristup nizovima, koji se uvek realizuju pomoću odgovarajućih memorija. Unutar FPGA komponenti memorije se implementiraju pomoću postojećih BlockRAM hardverskih resursa. BlockRAM memorije projektovane su tako da imaju dva porta za pristup podacima, što značajno limitira maksimalni propusni opseg memorijskog interfejsa. Jedan od načina da se ovaj problem prevaziđe je da se polazni niz (smešten unutar jedne BlockRAM memorije) podeli na veći broj manjih nizova (smeštenih u odvojenim BlockRAM memorijama). Na ovaj način se efektivno uvećava broj portova za pristup memorijskom sistemu sa 2, u slučaju polaznog niza, na $2N$, u slučaju njegove podele na N podnizova.

Vivado HLS direktiva `ARRAY_PARTITION` omogućava da se definiše način kako se polazni niz deli na podnizove. Vivado HLS podržava tri načina podele polaznog niza na podnizove:

- **podela na blokove** (*block*) – u ovom slučaju polazni niz deli se na N podnizova jednake veličine, koji se sastoje od sukcesivnih elemenata polaznog niza. Parametar N definiše dizajner.
- **ciklična podela** (*cyclic*) – polazni niz deli se na N podnizova jednake veličine, ali su ovaj put elementi polaznog niza ciklično raspoređeni po podnizovima.
- **potpuna podela** (*complete*) – polazni niz deli se na M jednoelementnih nizova, gde M predstavlja broj elemenata polaznog niza. Ovo je podrazumevani način podele niza unutar Vivado HLS alata, pri čemu se umesto memorije za smeštanje M elemenata koristi M registara.

Na slici 7.33 prikazan je način podele originalnog niza primenom ova tri načina.



Slika 7.33. Tri načina podele niza na podnizove koje podržava Vivado HLS alat

Ako se koriste *block* i *cyclic* načini, dodatni parametar, *factor*, definiše u koliko podnizova se deli polazni niz. Na slici 7.33 polazni niz podeljen je na dva podniza, pa bi parametar *factor* u ovom slučaju imao vrednost 2. Ukoliko ukupni broj elemenata polaznog niza, M , ne predstavlja celobrojni umnožak broja podnizova na koji se polazni niz deli, N , poslednji podniz ima manji broj elemenata od ostalih podnizova.

```
int niz[10][6][4].
```

```
niz[10][6][4] → ARRAY_PARTITION, mode=complete → niz_0[10][6]  
dimension=3                                niz_1[10][6]  
                                           niz_2[10][6]  
                                           niz_3[10][6]
```

```
niz[10][6][4] → ARRAY_PARTITION, mode=complete →
                                     dimension=1
niz_0[6][4]
niz_1[6][4]
niz_2[6][4]
niz_3[6][4]
niz_4[6][4]
niz_5[6][4]
niz_6[6][4]
niz_7[6][4]
niz_8[6][4]
niz_9[6][4]
```

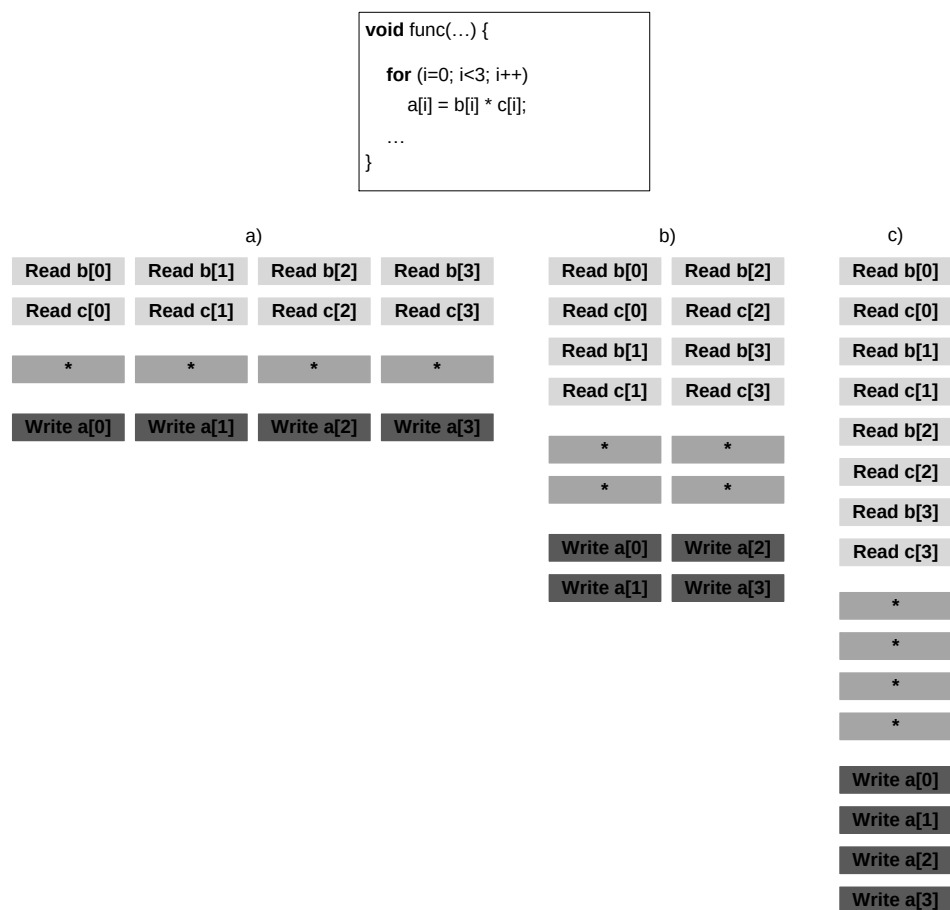
Slika 7.34. Podela višedimenzionalnog niza na podnizove

Razmotavanje petlji (*Loop Unrolling*)

Podrazumevani način sinteze petlji unutar Vivado HLS alata je da se one drže „umotanim“, odnosno da se iteracije petlje izvršavaju sekvencijalno, jedna nakon druge. Ovaj način izvršavanja petlje odgovara načinu kako se one izvršavaju na mikroprocesoru. Na ovaj način dobija se vrlo efikasna hardverska implementacija, po pitanju potrošnje hardverskih resursa, jer se iste funkcionalne jedinice koriste za izvođenje operacija is svih iteracija petlje. Na žalost, ovakva hardverska implementacija je ujedno i najsporija, rezultujući u često neprihvatljivo velikim kašnjenjima u izvršavanju petlje.

Kako bi se izvršavanje petlje učinilo efikasnijim, na uštrb povećane potrošnje hardverskih resursa, Vivado HLS alat pruža mogućnost da se odabrana petlja delimično ili u potpunosti razmota, korišćenjem UNROLL optimizacione direktive.

Na slici 7.35 prikazane su prednosti razmotavanja petlje, ali i problemi koji se u procesu razmotavanja petlje mogu pojaviti.



Slika 7.35. Različiti načini razmotavanja petlje
a) nerazmotana petlja, b) delimično razmotana petlja, sa faktorom 2, c) potpuno razmotana petlja

Primer sa slike 7.35 pretpostavlja da su nizovi *a*, *b* i *c* smešteni u BlockRAM memorijama. Na slikama 7.35a, 7.35b i 7.35c prikazane su tri različita načina realizacije petlje:

- **Nerazmotana petlja** (slika 7.35a) – kada se petlja nerazmota, svaka iteracija petlje izvršava se u posebnoj periodu klok signala. Ova implementacija zahteva 4 periode klok signala za izvršavanje petlje, ali zahteva samo jedan množak od

hardverskih resursa. Takođe, nizovi a , b i c mogu se smestiti u jednopristupne RAM memorije.

- **Delimično razmotana petlja**, sa faktorom razmotavanja 2 (slika 7.35b) – u ovom slučaju u svakoj periodi klok signala računaju se vrednosti dva elementa rezultujuće matrice c . Ova implementacija zahteva 2 periode klok signala za izvršavanje petlje, ali zahteva istovremeno postojanje 2 množača. Ono što je još važnije, u ovoj implementaciji RAM memorije u koje su smešteni nizovi a , b i c moraju imati dva pristupa, jer se u svakom taktu čitaju po dva elementa matrica a i b , i upisuju vrednosti dva elementa u matricu c . Ovo ograničenje i dalje ne predstavlja veliki problem jer, kako je ranije rečeno, unutar Xilinx FPGA komponenti postoje dvopristupni BlockRAM memorijski moduli koji se mogu iskoristiti u ovu svrhu.
- **Potpuno razmotana petlja** (slika 7.35c) – u ovom slučaju petlja je u potpunosti razmotana, tako da se celokupna petlja može izvršiti samo jednom taktu, što predstavlja četverostruko ubrzanje u odnosu na inicijalnu implementaciju. Međutim, ova implementacija zahteva 4 množača, što je takođe četverostruko uvećanje hardverskih resursa u odnosu na inicijalnu implementaciju. Ono što je još važnije, ovaj put potrebno je u istom taktu pročitati po 4 vrednosti elemenata nizova a i b i upisati četiri vrednosti elemenata niza c . Kako u FPGA komponentama ne postoje četvoropristupne memorije, interfejs ka memorijskom podsistemu nema potrebnu propusnu moć. Jedini način da se ovo ograničenje prevaziđe je da se iskoristi optimizaciona tehnika podele nizova a , b i c , opisana ranije.

Da bi se izvršilo razmotavanje petlje, na odabranu petlju potrebno je primeniti UNROLL optimizacionu direktivu. Ovu direktivu moguće je primeniti i na čitavu funkciju, pri čemu se razmotavaju sve petlje unutar funkcije.

Ukoliko se petlja razmotava u potpunosti, sve operacije iz svih iteracija petlje će se izvršiti u paraleli, pod uslovom da ne postoje međuzavisnosti između njih (*data dependency*). Ako operacije iz tekuće iteracije petlje zahtevaju rezultate izvršavanja operacija iz prethodne iteracije, one se ne mogu izvršiti u paraleli, već tek u trenutku kada ovi rezultati postanu dostupni.

Vivado HLS alat ne zahteva da prilikom delimičnog razmotavanja petlje broj prolaza kroz petlju bude celobrojni umnožak faktora razmotavanja petlje. Vivado HLS će automatski dodati potrebne provere kako bi obezbedio da ponašanje delimično razmotane petlje bude ekvivalentno ponašanju nerazmotane petlje. Na primer, ako je originalna petlja definisana na sledeći način

```
for (int i = 0; i < N; i++)  
    a[i] = b[i] + c[i];
```

gde je N parametar čija vrednost nije poznata u trenutku sinteze, već se može menjati u toku rada sistema. Kada dizajner specificira da želi da se ova petlja delimično razmoti sa faktorom 2, Vivado HLS alat će transformisati prethodnu petlju na sledeći način

```
for (int i = 0; i < N; i++) {  
    a[i] = b[i] + c[i];  
    if (i+1 >= N) break;  
    a[i+1] = b[i+1] + c[i+1];  
}
```

7.6.4. Optimizacije u cilju smanjenja kašnjenja sistema

U cilju smanjenja kašnjenja prilikom obrade podataka (*latency*) RTL sistema koji se generiše u potupku sinteze visokog nivoa pomoću Vivado HSL alata, moguće je koristiti sledeće optimizacione direktive:

- Specificiranje minimalnog i maksimalnog kašnjenja (*Latency Constraint*) u izvođenju operacija.
- Spajanje susednih petlji (*Loop Merging*).
- Uklanjanje ugnježđenih petlji (*Loop Flattening*).

Specificiranje minimalnog i maksimalnog kašnjenja (*Latency Constraint*)

Vivado HLS alat omogućava da dizajner specificira potrebno kašnjenje prilikom izvršavanja bilo kog objekta unutar algoritamskog modela. Specifikacija potrebnog kašnjenja vrši se korišćenjem LATENCY optimizacione direktive.

Kada se na nekom objektu, ili regionu, unutar algoritamskog modela sistema, definišu minimalno ili maksimalno kašnjenje, Vivado HLS alat će pokušati da ispuni postavljene zahteve, planirajući operacije na takav način da se završe u specificiranom broju taktova.

LATENCY direktiva primenjena na petlju specificira potrebno kašnjenje (broj taktova) za prilikom izvršavanja jedne iteracije petlje. Na ovaj način se zapravo specificira kašnjenje tela petlje, kao što je prikazano u sledećem primeru

```
for (int i = 0; i < N; i++) {  
    #pragma HLS latency max=10  
    ... telo petlje ...  
}
```

Iz prethodnog primera može se videti kako se korišćenjem *#pragma* direktive optimizacione direktive mogu umetnuti direktno u algoritamski model. Ovo je jedan od mogućih načina za specifikaciju željenih direktiva. Drugi način se bazira na korišćenju odgovarajućeg Tcl fajla. O ovome će biti reči kasnije. Prethodni primer

sadrži LATENCY direktivu, kojom je specificirano da maksimalno trajanje izvršavanja tela petlje ne bude veće od 10 taktova klok signala.

U slučaju da se želi ograničiti trajanje izvršavanja cele petlje (tokom svih iteracija), LATENCY direktivu treba primeniti na region koji obuhvata celu petlju, na sledeći način

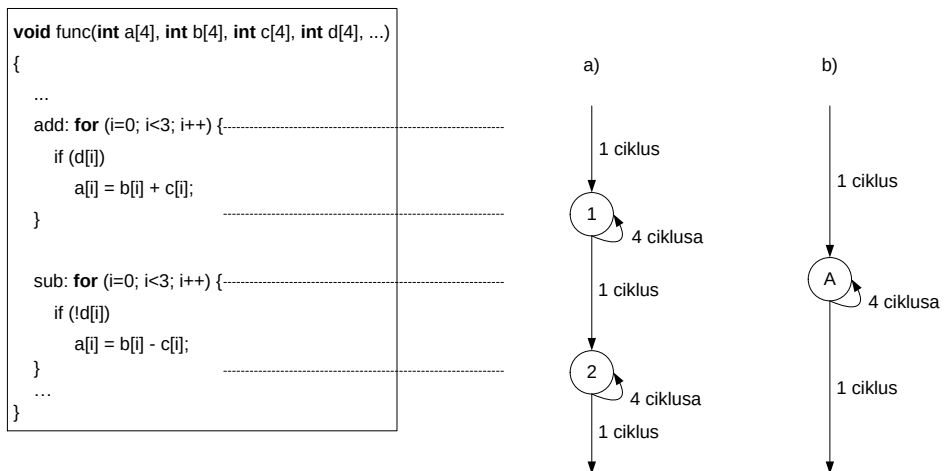
```
region_petlje: {
  #pragma HLS latency max=10
  for (int i = 0; i < N; i++) {
    ... telo petlje ...
  }
}
```

U slučaju da Vivado HLS ne može da ispuni postavljene zahteve u pogledu kašnjenja, generiše se odgovarajuće obaveštenje, i istovremeno relaksira postavljeno ograničenje u meri u kojoj je neophodno da se ono ispuni.

U slučaju kada je specificirano minimalno kašnjenje, a Vivado HLS alat je u stanju da generiše RTL model sa manjim kašnjenjem, umetanjem praznih ciklusa (*dummy clock cycles*) ispuniće se zahtevano ograničenje u pogledu minimalnog kašnjenja.

Spajanje susednih petlji (*Loop Merging*)

Kao što je ranije već rečeno, Vivado HLS alat prilikom RTL implementacije petlji kreira barem jedno stanje unutar upravljačkog automata za svaku petlju iz algoritamskog modela. U slučaju da imamo više sukcesivnih petlji, ovakav način implementacije može rezultovati u dodavanju većeg broja nepotrebnih ciklusa, i ugroziti dodatne optimizacije. Primer sa slike 7.36 ilustruje kako potpuno korektan algoritamski stil modelovanja može negativno uticati na performanse RTL modela.



Na slici 7.36a prikazan je deo dijagrama stanja konačnog automata (koji bi bio generisan od strane Vivado HLS alata u procesu sinteze visokog nivoa) u slučaju kada se dve petlje iz algoritamskog modela ne spajaju. Kao što je već rečeno, Vivado HLS prilikom implementacije petlji za svaku petlju kreira barem jedno stanje unutar upravljačkog automata. U primeru sa slike 7.36 vidimo da će u rezultujućem konačnom automatu biti kreirana dva stanja, po jedno za svaku od dve petlje iz algoritamskog modela. Kretanje između ovih stanja troši dodatne cikluse klok signala. Pod pretpostavkom da se telo petlje izvršava u jednom taktu, implementacija bez spajanja petlji zahteva 11 taktova za izvršavanje dve petlje:

- 1 ciklus za ulazak u *add* petlju
- 4 ciklusa za izvršenje *add* petlje
- 1 ciklus za izlazak iz *add* petlje i ulazak u *sub* petlju
- 4 ciklusa za izvršenje *sub* petlje
- 1 ciklus za izlazak iz *sub* petlje

Korišćenjem `LOOP_MERGE` optimizacione direktive Vivado HLS alatu može se naložiti da spoji ove dve petlje u jednu. U slučaju da se *add* i *sub* petlje spoje u jednu, odgovarajući deo konačnog automata izgledao bi kao na slici 7.36b. U ovom slučaju potreban broj taktova za izvršavanje ove dve spojene petlje iznosio bi 6 taktova, gotovo upola manje nego u varijanti sa slike 7.36a.

Spajanje petlji takođe omogućava da se operacije inače definisane u odvojenim telima petlji optimizuju zajedno. Na ovaj način povećava se mogućnost deljenja hardverskih resursa, što može dovesti do efikasnijih hardverskih implementacija.

Uklanjanje ugnježđenih petlji (*Loop Flattening*)

Na sličan način koji je bio diskutovan u prethodnom odeljku, kretanje između ugnježđenih petlji takođe zahteva dodatne cikluse klok signala. Jedan takt potreban je da se iz spoljašnje petlje uđe u unutrašnju, i jedan takt je potreban da se iz unutrašnje vratimo u spoljašnju petlju.

Sledeći primer ilustruje kako se, ukoliko se ne vodi računa o ovome, može potrošiti dodatnih 200 taktova na ove procese, prilikom izvršavanja spoljašnje petlje.

```
void func(int a, int b, int c, int d)
{
    ...
    spoljna_petlja: while (j < 100) {
        untrasnja_petlja: while (i < 6) {
            ...
            TELO_PETLJE
            ....
        }
    }
    ...
}
```

}

Za svaki ulazak u unutrašnju petlju troši se jedan takt i za svaki povratak u spoljašnju petlju troši se jedan takt. Kako ukupno imamo 100 iteracija spoljašnje petlje, trošimo dodatnih 200 taktova. Ukoliko bi bilo moguće na neki način kombinovati ove dve petlje u jednu, vreme izvršavanja ovog dela algoritamskog modela skratilo bi se za 200 ciklusa klok signala.

Vivado HLS alat obezbeđuje optimizacionu direktivu `LOOP_FLATTEN` pomoću koje je moguće ukloniti savršene ili polu-savršene ugnježdene petlje:

- **Savršene ugnježdene petlje** – samo krajnja unutrašnja petlja ima telo petlje, ne postoji nikakva logika između ugnježđenih petlji i broj iteracija kroz svaku petlju je konstantan.
- **Polu-savršene ugnježdene petlje** - samo krajnja unutrašnja petlja ima telo petlje, ne postoji nikakva logika između ugnježđenih petlji, broj iteracija kroz spoljašnju petlju može biti promenljiv.

U slučaju nesavršenih ugnježđenih petlji, kod kojih unutrašnje petlje imaju promenljivi broj iteracija, ili postoji logika koja se nalazi između ugnježđenih petlji, potrebno je pokušati modifikovati algoritamski model tako da rezultujuće petlje postanu savršene ili polu-savršene.

7.6.5. Optimizacije u cilju smanjenja potrebnih hardverskih resursa

U cilju smanjenja hardverskih resursa potrebnih za implementaciju RTL sistema koji se generiše u potupku sinteze visokog nivoa pomoću Vivado HSL alata, moguće je koristiti sledeće optimizacione direktive:

- Korišćenje tipova podataka sa tačno potrebnom širinom (*Bitwidth Narrowing*).
- Inlajnovanje funkcija (*Function Inlining*).
- Mapiranje većeg broja nizova u jedan (*Array Mapping*).
- Restrukturiranje nizova (*Array Reshaping*).
- Kontrolu korišćenog broja hardverskih resursa (*Resource Allocation*).

Korišćenje tipova podataka sa tačno potrebnom širinom (*Bitwidth Narrowing*)

Standardni C, C++ i SystemC tipovi podataka podrazumevaju da je broj bitova potreban za smeštanje vrednosti objekta najčešće multipl broja 8. Ova organizacija je savršeno logična ako se pretpostavi da će se program izvršavati na nekom od standardnih mikroprocesora, čija su magistrale podataka takođe širine koja je multipl broja 8. Međutim, u slučaju kada se navedeni jezici koriste za opis algoritamskog modela

digitalnog sistema koji će zatim automatski biti preveden u odgovarajući RTL model, pogodan za implementaciju u ASIC ili FPGA tehnologiji, ovakvi tipovi podataka mogu rezultovati u vrlo neefikasnim implementacijama, kako u pogledu performansi tako i u pogledu potrebnog broja hardverskih resursa za njihovu realizaciju. Na primer, ukoliko u algoritamskom modelu imamo celobrojnu promenljivu za koju pouzdano možemo utvrditi da njen opseg vrednosti ne izlazi iz opsega vrednosti koji se može predstaviti pomoću 17-bitnog broja, u standardnom C, C++ ili SystemC jeziku bili bismo primorani da je deklariramo kao tipa *int*, koji se reprezentuje pomoću 32-bitnih digitalnih reči. Prilikom automatske sinteze hardvera ova promenljiva smestila bi se u registar veličine 32 bita, a ukoliko ona recimo učestvuje u operaciji sabiranja, morao bi se koristiti 32-bitni sabirač, iako bi 17-bitni registar i sabirač bili sasvim dovoljni. Rezultujuća implementacija bila bi skoro dvostruko veća u terminima potrebnih elementarnih hardverskih resursa (flip-flopova i logičkih kapija), a pri tome bi imala i potencijalno dvostruko veće kašnjenje od optimalne (17-bitne) implementacije.

Upravo iz navedenog razloga Vivado HLS alat dolazi sa posebno definisanim tipovima podataka koji su pogodni za korišćenje unutar algoritamskih modela sistema koji su namenjeni za hardversku implementaciju. Glavna karakteristika ovih novih tipova podataka je da dizajner prilikom njihovog korišćenja može definisati i potrebnu širinu digitalne reči koja će biti korišćena za reprezentaciju vrednosti dotičnog tipa. U Vivado HLS alatu postoji veći broj novodefinisanih tipova podataka, od kojih su najvažniji:

- **[u]int<W>** (C jezik), **ap_[u]int<W>** (C++ jezik) - celobrojni tip podataka sa proizvoljnom širinom digitalne reči (specificiranom pomoću parametra *W*, maksimalna dozvoljena vrednost 1024) koji se može koristiti za predstavljanje označenih i neoznačenih celih brojeva.
- **ap_[u]fixed<W, I, Q, O, N>** (C++ jezik) – tip podataka pomoću kojega je moguće predstaviti necelobrojne vrednosti u sistemu sa fiksnom tačkom. Parametar *W* predstavlja ukupnu širinu digitalne reči izraženu u bitima, parameter *I* broj bita koji se koristi za predstavljanje celog dela broja, parametar *Q* određuje način na koji se vrši kvantizacija vrednosti, parameter *O* kako se postupa prilikom prekoračenja opsega vrednosti, a parametar *N* definiše broj saturacionih bita.

Inlajnovanje funkcija (*Function Inlining*)

Inlajnovanjem funkcija uklanja se hijerarhija između funkcija. Funkcija se inlajnuje u Vivado HLS alatu korišćenjem optimizacione direktive `INLINE`.

Inlajnovanjem funkcije može se redukovati veličina rezultujućeg sistema jer se na ovaj način omogućava da se objekti unutar inlajnovane funkcije optimizuju zajedno sa objektima unutar funkcije u kojoj se nalazi inlajnovana funkcija. Na ovaj način povećava se mogućnost za deljenje resursa, što može rezultovati u manjem dizajnu.

Kao ilustraciju benefita koji se mogu postići inlajnovanjem funkcija posmatrajmo sledeći algoritamski model

```

fsub (int p, int q)
{
    int q1 = q + 10;
    f1 (p1,q); // treća instanca f1 funkcije
    ...
}

void top (int a, int b, int c, int d)
{
    ...
    f1 (a,b); // prva instanca f1 funkcije
    f1 (a,c); // druga instanca f1 funkcije
    fsub(a,d);
    ...
}

```

Kada se na osnovu prethodnog algoritamskog modela generiše RTL model, u njemu će postojati tri instance modula koji implementira funkcionalnost opisanu unutar funkcije *f1*. Inlajnovanjem *fsub* funkcije i korišćenjem ALLOCATION optimizacione direktive (o kojoj će još biti reči kasnije) da se ograniči postojanje samo jedne instance modula koji implementira funkcionalnost funkcije *f1*, rezultujući RTL model biće oko tri puta manji od prethodnog.

```

fsub (int p, int q)
{
    #pragma HLS INLINE
    int q1 = q + 10;
    f1 (p1,q);
    ...
}

void top (int a, int b, int c, int d)
{
    #pragma HLS ALLOCATION instances=f1 limit=1 function
    ...
    f1 (a,b);
    f1 (a,c);
    fsub(a,d);
    ...
}

```

INLINE optimizaciona direktiva je vrlo moćna direktiva, koja omogućava značajnu modifikaciju strukture generisanog RTL modela bez potrebe da se menja algoritamski model, i kao takva predstavlja vrlo moćnu metodu u procesu pretrage prostora dizajna.

Mapiranje većeg broja nizova u jedan (*Array Mapping*)

U slučaju da algoritamski model koristi veći broj malih nizova, njihovim mapiranjem u jedan veliki niz tipično se postiže redukcija broja potrebnih BlockRAM memorija za njihovu implementaciju.

Vivado HLS alat svaki niz mapira u odvojenu BlockRAM memoriju. Osnovna veličina BlockRAM memorija unutar Xilinx FPGA komponenti iznosi 18 kilobita. Ukoliko mali nizovi pojedinačno ne koriste puni kapacitet individualnih BlockRAM memorija, njihovim mapiranjem u jedan veliki niz postiže se ušteda u BlockRAM modulima.

Mapiranje većeg broja nizova u jedan veći postiže se korišćenjem `ARRAY_MAP` optimizacione direktive. Ova direktiva podržava dva načina mapiranja nizova:

- **Horizontalno mapiranje** – u ovom slučaju nizovi se spajaju prostom konkatencijom. Na ovaj način se kreira rezultujući niz čiji broj elemenata je jednak sumi elemenata nizova koji se spajaju.
- **Verikalno mapiranje** – u ovom slučaju novi niz sadrži elemente koji nastaju konkatencijom pojedinačnih elemenata nizova koji se spajaju. Na ovaj način kreira se rezultujući niz kod kojeg su elementi širine koja predstavlja zbir širina elemenata nizova koji se spajaju.

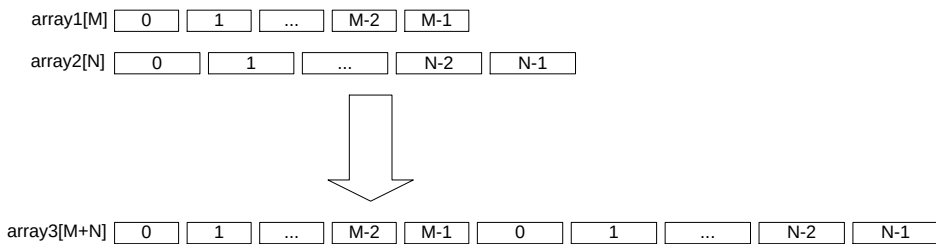
Sledeći primer ilustruje princip horizontalnog spajanja nizova. Pretpostavimo da u algoritamskom modelu imamo dva niza

```
void func (...) {
    int8 array1[M];
    int12 array2[N];
    ...
    loop_1: for (i=0;i<M;i++) {
        array1[i] = ...;
        array2[i] = ...;
    }
    ...
}
```

Nizovi *array1* i *array2* mogu se mapirati u jedan niz, *array3* na sledeći način

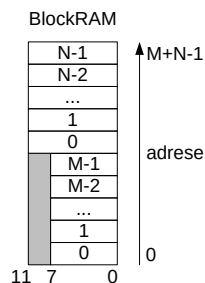
```
void func (...) {
    int8 array1[M];
    int12 array2[N];
    #pragma HLS ARRAY_MAP variable=array1 instance=array3 horizontal
    #pragma HLS ARRAY_MAP variable=array2 instance=array3 horizontal
    ...
    loop_1: for (i=0;i<M;i++) {
        array1[i] = ...;
        array2[i] = ...;
    }
    ...
}
```

U prethodnom primeru, korišćenjem `ARRAY_MAP` direktive, nizovi *array1* i *array2* mapirani su u niz *array3* kao što je prikazano na slici 7.37.



Slika 7.37. Horizontalno mapiranje dva niza

Redosled mapiranja manjih nizova u jedan veći određen je redosledom kojim su specificirane ARRAY_MAP direktive unutar algoritamskog modela. Kako broj bitova za reprezentaciju vrednosti elemenata nizova koji se mapiraju može biti različit, širina digitanih reči u rezultujućem nizu jednaka je najširem elementu iz nizova koji se mapiraju. U slučaju nizova *array1* i *array2*, organizacija memorije u kojoj će biti smešten rezultujući niz prikazana je na slici 7.38.



Slika 7.38. Organizacija memorije u kojoj se smešta rezultujući niz nastao horizontalnim mapiranjem dva niza

U slučaju vertikalnog mapiranja, nizovi se spajaju tako da rezultujući niz ima broj elemenata koji je jednak najdužem od nizova koji se spajaju, pri čemu je širina svakog elementa jednaka zbiru širina elemenata nizova koji se spajaju. Način na koji je potrebno definisati optimizacione direktive da bi se nizovi *array1* i *array2* iz prethodnog primera vertikalno mapirali, prikazan je u nastavku.

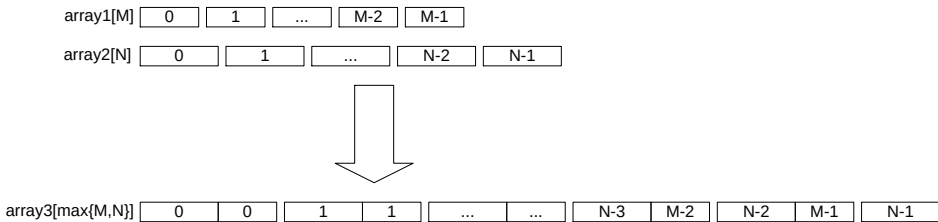
```
void func (...) {
    int8 array1[M];
    int12 array2[N];
    #pragma HLS ARRAY_MAP variable=array2 instance=array3 vertical
    #pragma HLS ARRAY_MAP variable=array1 instance=array3 vertical
    ...
    loop_1: for (i=0;i<M;i++) {
        array1[i] = ...;
```

```

        array2[i] = ...;
    ...
}
    ...
}

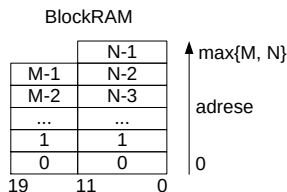
```

Struktura niza *array3*, koji nastaje vertikalnim mapiranjem nizova *array1* i *array2*, prikazana je na slici 7.39.



Slika 7.39. Vertikalno mapiranje dva niza

Redosled mapiranja elemenata manjih nizova u jedan element rezultujućeg niza, određen je redosledom kojim su specificirane `ARRAY_MAP` direktive unutar algoritamskog modela. Kako broj bitova za reprezentaciju vrednosti elemenata nizova koji se mapiraju može biti različit, širina digitanih reči u rezultujućem nizu jednaka je zbiru širina elemenata iz nizova koji se mapiraju. U slučaju nizova *array1* i *array2* organizacija memorije u kojoj će biti smešten rezultujući niz prikazana je na slici 7.38.

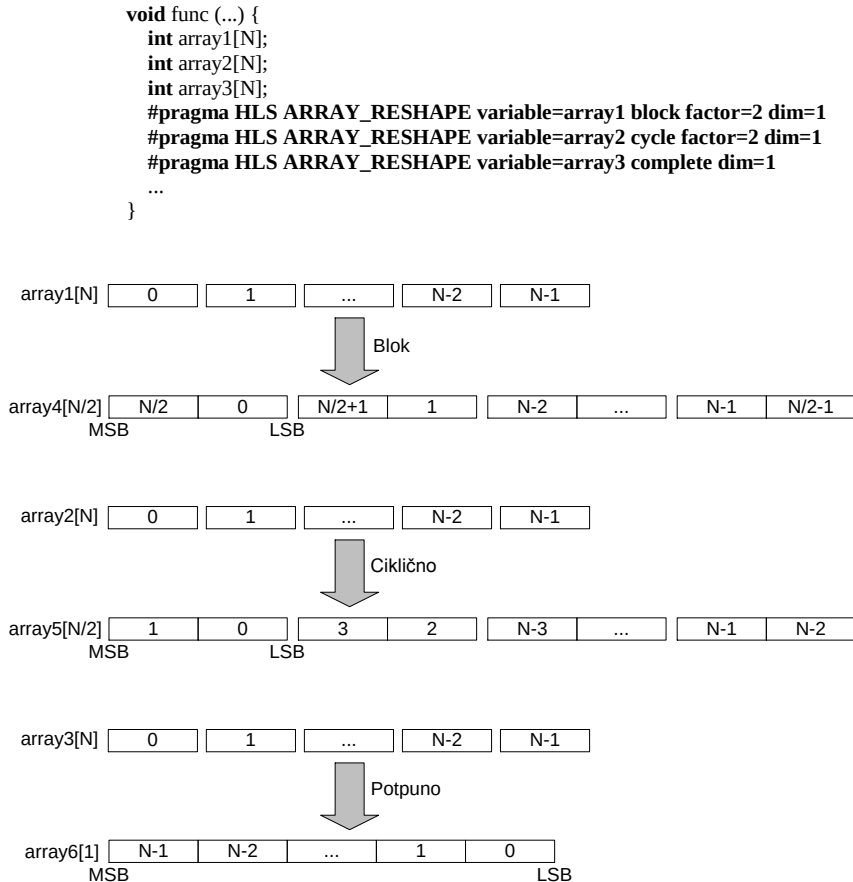


Slika 7.40. Organizacija memorije u kojoj se smešta rezultujući niz nastao vertikalnim mapiranjem dva niza

Restrukturiranje nizova (*Array Reshaping*)

Restrukturiranje niza kombinuje podelu niza (*Array Partitioning*) sa vertikalnim mapiranjem niza (*Vertical Array Mapping*), kako bi se smanjila potrošnja memorijskih resursa, a istovremeno sačuvala dobre osobine koje proističu iz podele niza, paralelnu obradu podataka.

Unutar VivadoHLS alata restrukturiranje niza postiže se korišćenjem `ARRAY_RESHAPE` optimizacione direktive. Na primer, primenom `ARRAY_RESHAPE` direktive na tri niza deklarisanu unutar sledećeg algoritamskog modela, oni se restrukturiraju na način prikazan na slici 7.41.



Slika 7.41. Izgled restrukturiranih nizova nakon primene `ARRAY_RESHAPE` direktive

Kontrola hardverskih resursa (*Resource Allocation*)

Vivado HLS alat se prilikom automatske sinteze RTL modela prvenstveno trudi da maksimizuje performanse sistema, na uštrb povećane potrošnje hardverskih resursa. Zbog toga je u nekim slučajevima neophodno eksplicitno ograničiti broj hardverskih resursa koji stoje na raspolaganju za izvođenje odabranih operacija iz algoritamskog modela. Ograničavanje broja raspoloživih hardverskih resursa predstavlja vrlo efikasan način za kontrolu veličine finalnog dizajna, podstičući deljenje resursa prilikom sinteze visokog nivoa.

Vivado HLS ALLOCATION optimizaciona direktiva omogućava dizajneru da specificira maksimalni broj operatora, IP jezgara ili funkcija odgovarajućeg tipa, koji se smeju koristiti unutar generisanog RTL modela. Na primer, posmatrajmo sledeći algoritamski model u kojem postoji potreba za izvođenjem ukupno 300 operacija množenja, ali pretpostavimo da ciljna FPGA komponenta pomoću koje ovaj sistem treba da bude realizovan sadrži samo 256 DSP blokova (unutar kojih postoje množači). Korišćenjem ALLOCATION direktive Vivado HLS je ograničen da prilikom generisanja RTL modela koristi maksimalno 256 DSP blokova

```
int32 mac_unit (int16 d[317]) {
    static int32 mac;
    int i;
    #pragma HLS ALLOCATION instances=mul limit=256 operation
    for (i=0;i<300;i++) {
        #pragma HLS UNROLL
        mac += mac * d[i];
    }
    rerun mac;
}
```

U prethodnom primeru, korišćenjem direktive UNROLL forsirali smo razmotavanje petlje po iteratoru *i*. Kako je specificirano potpuno razmotavanje petlje, rezultujući RTL model koristio bi 300 množača, što prevazilazi broj množača na ciljnoj FPGA komponenti. Korišćenjem ALLOCATION direktive ograničili smo broj množača (selektovanih pomoću parametra *instances=mul*) na maksimalno 256 (parametar *limit=256 operation*).

NAPOMENA: Ukoliko se pomoću ALLOCATION direktive broj odabranih operatora ograniči na vrednost koja je veća od potrebne, Vivado HLS alat će pokušati da koristi broj operatora specificiran u ALLOCATION direktivi, odnosno maksimalni broj potrebnih operatora, što će za posledicu imati minimizaciju deljenja odabranog operatora unutar RTL modela.

Kao što se iz prethodnog primera može videti, pomoću parametra *instances* može se specificirati operator koji želimo da ograničimo. Vivado HLS dozvoljava da se ograničenja u pogledu korišćenih instanci nametnu na operatore prikazane u tabeli 7.3.

Operator	Objašnjenje
add	Sabiranje
sub	Oduzimanje
mul	Množenje
icmp	Kkomparacija
sdiv	Delenje označenih brojeva
udiv	Delenje neoznačenih brojeva
srem	Ostatak prilikom deljenje označenih brojeva

urem	Ostatak prilikom deljenje neoznačenih brojeva
lshr	Logičko pomeranje u desno
ashr	Aritmetičko pomeranje u desno
shl	Pomeranje u levo

Tabela 7.3. Spisak operatora za koje se može definisati ograničenje u pogledu maksimalnog broja instanci

Kada se završi sinteza visokog nivoa Vivado HLS alat, na osnovu specificiranih vremenskih ograničenja, vremenskih karakteristika odabrane FPGA komponente, raspoloživih hardverskih resursa na odabranoj FPGA komponenti i zadatih optimizacionih direktiva, odlučuje koji će konkretni hardverski resurs biti zadužen za izvođenje svake operacije koja je identifikovana unutar algoritamskog modela sistema. Na primer, prilikom odlučivanja kako će fizički biti realizovana operacija množenja, Vivado HLS alat može da koristi kombinacionu mrežu za množenje sa ili bez protočne obrade, koja će biti implementirana pomoću programabilne logike, ili se može odlučiti da koristi posvećene hardverske množače, koji se nalaze unutar DSP hardverskih blokova, ukoliko takvi blokovi postoje na ciljnoj FPGA komponenti.

Ukoliko dizajner želi da ima kontrolu nad načinom na koji će operacije iz algoritamskog modela biti mapirane u hardverske resurse koji stoje na raspolaganju, može koristiti optimizacionu direktivu RESOURCE. Na primer, sledeći primer pokazuje kako se pomoću RESOURCE direktive može specificirati željeni hardverski resurs koji Vivado HLS treba da koristi prilikom implementacije odabrane operacije unutar algoritamskog modela.

```
int func (int a, int b) {
    int c, d;
    #pragma HLS RESOURCE variable=c latency=2
    c = a*b;
    d = a*c;
    return d;
}
```

U prethodnom primeru, korišćenjem RESOURCE direktive, nametnuto je ograničenje u pogledu hardverskog resursa koji će biti korišćen prilikom implementacije operacije čiji rezultat se dodeljuje promenljivoj *c*. Kako je vrednost promenljive *c* jednaka proizvodu ulaznih argumenata *a* i *b*, a zahtevano je da kašnjenje prilikom izvođenja ove operacije množenja iznosi 2 periode klok signala, Vivado HLS alat će za realizaciju ove operacije množenja koristiti množač sa protočnom obradom u dve faze.

Sledeći primer prikazuje kako se korišćenjem RESOURCE direktive može eksplicitno navesti tip hardverskog resursa koji je potrebno koristiti prilikom implementacije odabrane operacije. U ovom primeru želimo da se operacija sabiranja izvede korišćenjem sabirača smeštenog unutar hardverskog DSP bloka. Podrazumevana implementacija operacije sabiranja unutar Vivado HLS alata se bazira na korišćenju sabirača implementiranih u programabilnoj logici (korišćenjem LUT hardverskih

resursa), a primenom RESOURCE direktive može se promeniti ovakvo mapiranje (postavljanjem parametra *core* na vrednost *AddSub_DSP*).

```
void apint_arith(int16 inA, int16 inB, int17 *out1) {
    int17 temp;
    #pragma HLS RESOURCE variable=temp core=AddSub_DSP
    temp = inB + inA;
    *out1 = temp;
}
```

U tabeli 7.4 prikazani su hardverski resursi, podržani od strane Vivado HLS alata, koji se mogu koristiti za implementaciju standardnih aritmetičkih operacija.

Hardverski resurs	Objašnjenje
AddSub	Standardno IP jezgro za implementaciju operacija sabiranja i oduzimanja, implementirano kao kombinaciona mreža unutar programabilne logike.
AddSubnS	Sabirač, odnosno oduzimač, sa protočnom obradom podataka od N faza. Vivado HLS alat automatski određuje potreban broj faza u protočnoj obradi u zavisnosti od potreba.
AddSub_DSP	Sabirač, odnosno oduzimač, koji se nalazi unutar hardverskog DSP48 bloka.
DivnS	Delitelj sa protočnom obradom od N faza.
DSP48	Hardverski DSP blok koji dozvoljava izvođenje operacije množenja sa ili bez protočne obrade. Maksimalni broj faza protočne obrade u ovom slučaju iznosi 4.
Mul	Množač realizovan kao kombinaciona mreža implementirana unutar programabilne logike
MulnS	Množač sa protočnom obradom podataka u N faza. Koristi se kao podrazumevani način implementacije množenja, kada širine operanada prevazilaze širinu množača unutar DSP bloka.
Mul_LUT	Množač realizovan korišćenjem LUT hardverskih resursa.

Tabela 7.4. Spisak operatora za koje se može definisati ograničenje u pogledu maksimalnog broja instanci

RESOURCE direktiva može se koristiti i za specifikiranje načina na koji je potrebno implementirati memorijske resurse. Raspoloživa IP jezgra koja stoje na raspolaganju prikazana su u tabeli 7.5.

Hardverski resurs	Objašnjenje
FIFO	FIFO memorija. Vivado HLS određuje način na koji će memorija biti implementirana, pomoću BlockRAM ili distribuiranog RAM resursa.
FIFO_BRAM	FIFO memorija implementirana korišćenjem BlockRAM hardverskog resursa.
FIFO_LUTRAM	FIFO memorija implementirana korišćenjem distribuiranog RAM hardverskog resursa.
FIFO_SRL	FIFO memorija implementirana pomoću SRL hardverskog resursa.
RAM_1P	Jednoprístupna RAM memorija. Vivado HLS određuje način na koji će memorija biti implementirana, pomoću BlockRAM ili distribuiranog RAM resursa.
RAM_1P_BRAM	Jednoprístupna RAM memorija implementirana korišćenjem BlockRAM hardverskog resursa.
RAM_1P_LUTRAM	Jednoprístupna RAM memorija implementirana pomoću distribuiranog RAM hardverskog resursa.
RAM_2P	Dvoprístupna RAM memorija kod koje se jedan pristup koristi samo za čitanje, a drugi i za čitanje i za upis. Vivado HLS određuje način na koji će memorija biti implementirana, pomoću BlockRAM ili distribuiranog RAM resursa.
RAM_2P_BRAM	Dvoprístupna RAM memorija kod koje se jedan pristup koristi samo za čitanje, a drugi i za čitanje i za upis implementirana korišćenjem BlockRAM hardverskog resursa.
RAM_2P_LUTRAM	Dvoprístupna RAM memorija kod koje se jedan pristup koristi samo za čitanje, a drugi i za čitanje i za upis implementirana korišćenjem distribuiranog RAM hardverskog resursa.
RAM_S2P_BRAM	Dvoprístupna RAM memorija kod koje se jedan pristup koristi za čitanje a drugi za upis podataka, implementirana korišćenjem BlockRAM hardverskog resursa.
RAM_S2P_LUTRAM	Dvoprístupna RAM memorija kod koje se jedan pristup koristi za čitanje a drugi za upis podataka, implementirana korišćenjem distribuiranog RAM hardverskog resursa.
RAM_T2P_BRAM	Prava dvoprístupna RAM memorija kod koje je na oba pristupa moguć i upis i čitanje podataka, implementirana korišćenjem BlockRAM hardverskog resursa.
ROM_1P	Jednoprístupna ROM memorija. Vivado HLS određuje način na koji će memorija biti implementirana, pomoću BlockRAM ili

	distribuiranog ROM resursa.
ROM_1P_BRAM	Jednoprístupna ROM memorija implementirana korišćenjem BlockRAM hardverskog resursa.
ROM_nP_BRAM	Jednoprístupna RAM memorija implementirana pomoću BlockRAM hardverskog resursa. Vivado HLS određuje potreban broj pristupa.
ROM_1P_LUTRAM	Jednoprístupna ROM memorija implementirana korišćenjem distribuiranog ROM hardverskog resursa.
ROM_nP_LUTRAM	Višepriístupna ROM memorija implementirana pomoću distribuiranog ROM hardverskog resursa. Vivado HLS određuje potreban broj pristupa.
ROM_2P	Dvopriístupna ROM memorija. Vivado HLS određuje način na koji će memorija biti implementirana, pomoću BlockRAM ili distribuiranog ROM resursa.
ROM_2P_BRAM	Dvopriístupna ROM memorija implementirana korišćenjem BlockRAM hardverskog resursa.
ROM_2P_LUTRAM	Dvopriístupna ROM memorija implementirana korišćenjem distribuiranog ROM hardverskog resursa.
XPM_MEMORY	Memorija realizovana korišćenjem UtraRAM hardverskih resursa. Ovi resursi postoje samo u UltraScale+ FPGA familijama kompanije Xilinx.

Tabela 7.5. Spisak hardverskih resursa koji se mogu koristiti za realizaciju memorijskih elemenata definisanih unutar algoritamskog modela

RESOURCE direktiva koristi navedenu promenljivu kao ciljni objekat na koji se primenjuje. Ako se u naredbi dodele vrednosti promenljivoj koja je označena sa nekom RESOURCE direktivom nalazi veći broj operatora na koji se RESOURCE direktiva može odnositi, Vivado HLS je primenjuje na svaki od njih. Ukoliko to nije željeno ponašanje, algoritamski model se mora modifikovati na takav način da izoluje željeni operator na koji se želi primeniti RESOURCE direktiva.

7.6.6. Optimizacija performansi RTL modela „Naive“ množača matrica

Nakon što smo detaljno objasnili postupak definisanja optimizacionih direktiva, vratimo se primeru projektovanja „Naive“ množača matrica i pokažimo kako se performanse inicijalne implementacije mogu poboljšati korišćenjem odgovarajućih optimizacionih direktiva.

Analiza inicijalne RTL implementacije „Naive“ množača matrica, sprovedena u poglavlju 7.4 pokazala je da kašnjenje prilikom množenja matrica maksimalnih dimenzija 8x8 elemenata iznosi 1142 periode klok signala (vidi sliku 7.16). Daljom

analizom utvrđeno je da je najdublja ugnježdjena petlja (po parametru k) u najvećoj meri odgovorna za ovoliko kašnjenje, jer svaka iteracija ove petlje traje 3 periode klok signala (vidite sliku 7.16). Detaljnom analizom razloga za ovoliko trajanje izvršavanja tela petlje k , utvrđeno je da ono proizilazi zbog toga što se u svakoj iteraciji petlje 2 periode klok signala troše na učitavanje vrednosti elemenata matrica A i B koje je zatim potrebno pomnožiti, a da se dodatni period klok signala troši na izvođe ovog množenja i akumuliranje proizvoda na tekuću vrednost sume, smeštene u promenljivoj *temp* (vidite sliku 7.21).

U okviru ove glave, u poglavlju 7.6.3, analizirane su optimizacione tehnike koje se mogu koristiti za povećanje performansi generisanog RTL modela. Jedna od tih tehnika uključuje i uvođenje protočne obrade unutar petlji koje postoje u algoritamskom modelu sistema koji se projektuje. Kako je ranije rečeno, u slučaju postojanja ugnježđenih petlji, protočna obrada uvodi se počevši od najdublje petlje. U našem primeru, prva optimizacija koju ćemo primeniti biće uvođenje protočne obrade unutar petlje po iteratoru k . Da bismo naložili Vivado HLS alatu da želimo da koristi protočnu obradu prilikom implementacije ove petlje, potrebno je definisati odgovarajuću PIPELINE optimizacionu direktivu.

Pre nego što pokažemo kako je to moguće uraditi, potrebno je naglasiti da ćemo ovu optimizaciju primeniti na *solution2* implementaciju, koju smo generisali u poglavlju 7.5. Na ovaj način ćemo, nakon što generišemo novi RTL model sistema, moći da uporedimo njegove performanse sa performansama inicijalnog RTL modela, generisanog unutar *solution1* implementacije.

Definisanje optimizacionih direktiva unutar Vivado HLS alat može se izvesti na dva načina:

- **Direktnom modifikacijom algoritamskog modela, korišćenjem odgovarajućeg broja *#pragma* direktiva**

Ovaj način već je bio korišćen u primerima iz poglavlja 7.6.4 i 7.6.5. Glavni nedostatak ovakvog definisanja optimizacionih direktiva jeste da one u ovom slučaju postaju integralni deo algoritamskog modela i biće korišćene u svakoj od implementacija, što u nekim slučajevima nije željeno ponašanje. U fazi optimizacije dizajna vrši se isprobavanje velikog broja optimizacionih tehnika, od kojih neke neće rezultovati u poboljšanju performansi sistema koji se projektuje (čak mogu dovesti i do njihove degradacije). Pre nego što se generiše naredna implementacija, potrebno je ručno obrisati problematične direktive iz algoritamskog modela.

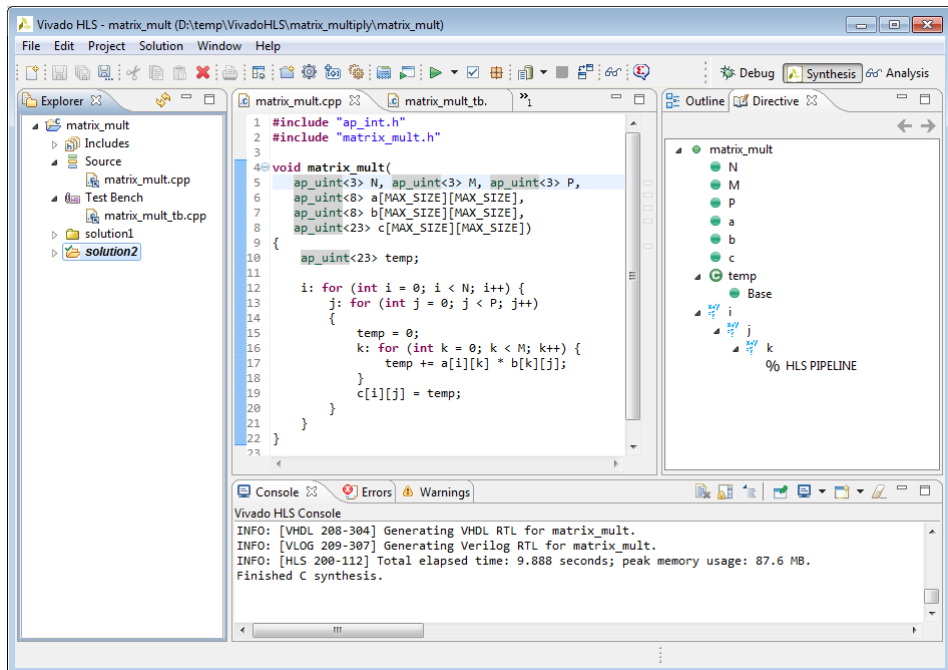
- **Definisanjem direktiva unutar posebnog Tcl fajla**

Ovaj način razdvaja direktive od samog algoritamskog modela, smeštajući ih u posebni Tcl fajl. Svaka implementacija ima svoj Tcl fajl, u kome se nalaze direktive koje treba primeniti prilikom generisanja RTL modela. Prilikom generisanja naredne implementacije, korisnik može specificirati da li želi da koristi direktive definisane u nekoj od prethodnih implementacija, ili želi da započne rad sa potpunom čistim algoritamskim modelom. Obratite pažnju da je prilikom generisanja *solution2* implementacije, u poglavlju 7.5, u okviru

„Solution Configuration“ prozora postojala opcija „Copy directives and constraints from solution“ (vidi sliku 7.24), koja je pružala mogućnost da odaberemo koji set ograničenja i optimizacionih direktiva želimo da postane osnova za novu implementaciju. Ovaj način definisanja je vrlo fleksibilan i preporučljiv je za korišćenje u fazi razvoja novog dizajna. Prvi način koristan je u trenutku kada smo optimizovali dizajn u dovoljnoj meri, i želimo da isporučimo konačnu verziju RTL modela. Tada je zgodno uključiti potrebne optimizacione direktive u sam algoritamski model, jer one predstavljaju integralni deo optimalne implementacije.

Kako se mi nalazimo u fazi optimizacije dizajna, koristićemo drugi način za definisanje optimizacionih direktiva.

Da bi se kreirala nova optimizaciona direktiva, Vivado HLS alat mora biti u režimu sinteze (u režim sinteze prelazi se pritiskom na *Synthesis* dugme, u gornjem desnom uglu glavnog Vivado HLS prozora) i mora biti otvoren i selektovan fajl koji sadrži algoritamski model u kojem želimo da definišemo optimizacionu direktivu. U našem slučaju potrebno je otvoriti *matrix_mult.cpp* fajl, koji sadrži algoritamski model „Naive“ množača matrica koji želimo da optimizujemo. Nakon što smo otvorili navedeni fajl, pritiskom na *Directive* karticu (koja se nalazi u okviru kranje desnog prozora) možemo videti listu objekata, definisanih unutar algoritamskog modela „Naive“ množača, za koje možemo definisati optimizacione direktive, kao što je prikazano na slici 7.42.



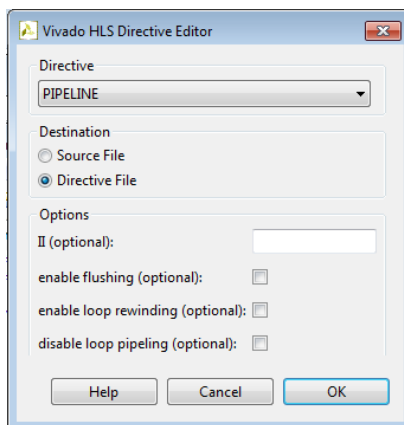
Slika 7.42. Vivado HLS alat u režimu sinteze, pripremljen za definisanje novih optimizacionih direktiva nad objektima iz algoritamskog modela „Naive“ množača

Inspekcijom sadržaja *Directive* kartice sa slike 7.42 možemo videti da u okviru algoritamskog modela „Naive“ množača matrica postoje sledeći objekti:

- Funkcija *matrix_mult* – kao što je ranije rečeno, Vivado HLS alat omogućava definisanje optimizacionih direktiva koje će se primeniti na čitavu funkciju.
- Ulazni i izlazni argumenti, *N*, *M*, *P*, *a*, *b* i *c* – za svaki od ovih argumenata mogu se definisati različite optimizacione direktive, na primer INTERFACE optimizaciona direktiva.
- Tri petlje, *i*, *j* i *k* – nad petljama je takođe moguće definisati veliki broj različitih optimizacionih direktiva.

Spisak optimizacionih direktiva koje se mogu definisati za odabrani objekat iz *Directive* prozora može se videti ako se u *Directive* prozoru odabere željeni objekat, pritisne desno dugme miša i izabere „Insert Directive...” opcija.

Kao što smo ranije rekli, prva optimizacija koju ćemo uvesti biće protočna obrada unutar petlje *k*. Ovo ćemo postići definisanjem PIPELINE optimizacione direktive za petlju *k*. Izborom petlje *k* u okviru *Directive* prozora, desnim klikom i izborom „Insert Directive...” opcije otvara se „Directive Editor“ prozor prikazan na slici 7.43.

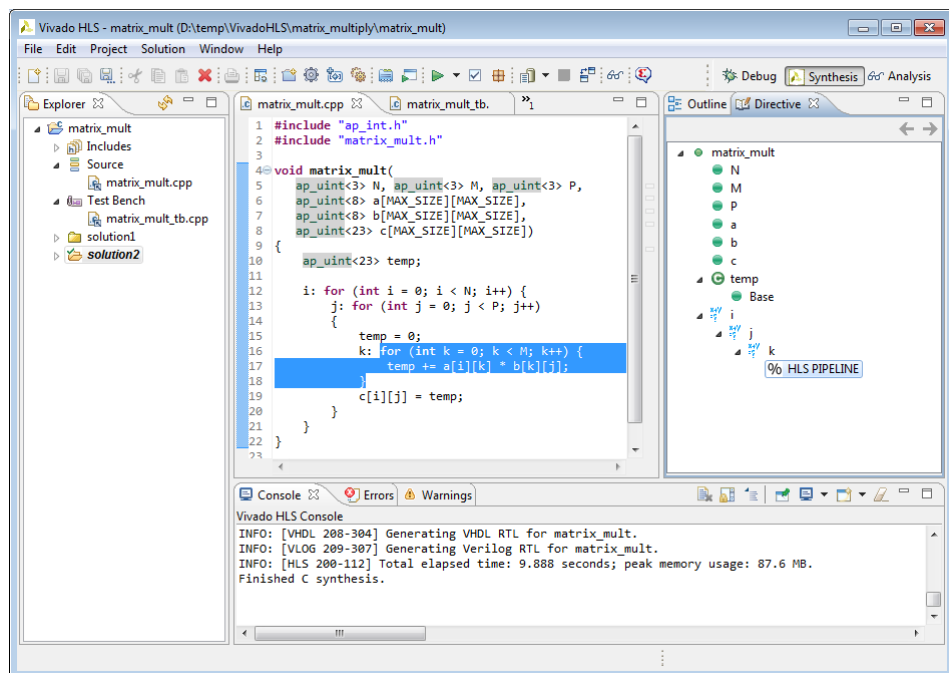


Slika 7.43. „Directive Editor“ prozor sa odabranom PIPELINE optimizacionom direktivom

Izborom opcije *Directive* pojavljuje se lista optimizacionih direktiva koje se mogu definisati za trenutno odabrani objekat algoritamskog modela. Kako mi želimo da definišemo PIPELINE direktivu potrebno ju je izabrati iz liste ponuđenih direktiva. Sadržaj „Directive Editor“ prozora menja se u zavisnosti od odabrane optimizacione direktive, kako bi omogućilo definisanje potrebnih parametara. U slučaju PIPELINE direktive dodatni parametri koje je potrebno definisati mogu se videti na slici 7.43, u

okviru *Options* polja. U većini slučajeva ovi dodatni parametri se mogu ostaviti praznim, u tom slučaju Vivado HLS koristi podrazumevane vrednosti za potrebne parametre. Poslednje polje je *Destination* polje, pomoću kojega je moguće specificirati mesto na koje želimo da smestimo optimizacionu direktivu. Ako se selektuje „*Source File*“ opcija, optimizaciona direktiva se unosi direktno u algoritamski model (pretvara se u odgovarajuću *#pragma* direktivu), i ovo je ekvivalentno direktnom unošenju direktive u algoritamski model o kojem je bilo reči ranije. Selektovanjem opcije „*Directive File*“, direktiva se smešta u prateći Tcl fajl, o čemu je takođe bilo reči ranije.

U našem primeru nije potrebno podesiti nikakve dodatne parametre, i želimo da se ova direktiva smesti u Tcl fajl sa direktivama, pa podešavanja treba da odgovaraju onim sa slike 7.43. Pritiskom na dugme *OK* Vivado HLS alat kreira novu direktivu i vraća se u osnovni prozor u kome se sada jasno može videti nova direktiva na petlji *k* (u okviru *Directive* prozora), kao što je prikazano na slici 7.44.



Slika 7.44. Izgled glavnog prozora Vivado HLS alata nakon definisanja PIPELINE direktive na petlji *k*

Pošto je ovo za sada jedina optimizaciona direktiva koju ćemo definisati, potrebno je da vidimo kakvi su njeni efekti. Da bi smo mogli analizirati efekte korišćenja PIPELINE direktive, prethodno moramo da sintetizujemo novi RTL model, koji će koristiti protočnu obradu podataka prilikom implementacije petlje *k*, pritiskom na „*C Synthesis*“

dugme. Nakon što Vivado HLS alat završi sintezu visokog nivoa, automatski će otvoriti fajl koji sadrži rezultate sinteze, čiji sadržaj je prikazan na slici 7.45.

Analizom izveštaja možemo primetiti da novi RTL model ima kašnjenje od 540 periode klok signala, što predstavlja više nego duplo skraćenje u odnosu na inicijalnu implementaciju (koja je imala kašnjenje od 1142 periode klok signala). Na osnovu ovoga možemo zaključiti da je uvođenje protočne obrade unutar petlje k rezultovalo u značajnom poboljšanju performansi sistema. Inspekcijom potrebnih hardverskih resursa možemo primetiti da se broj potrebnih DSP blokova takođe udvostručio. Ovo je tipični slučaj prilikom optimizovanja performansi sistema, kada se poboljšanje u brzini obrade „plaća“ povećanjem potrebnog broja hardverskih resursa.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	6.38	1.25

Latency (clock cycles)

Summary

Latency		Interval		Type
min	max	min	max	
1	540	2	541	none

Detail

Instance

Loop

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	1	-	-
Expression	-	1	0	33
FIFO	-	-	-	-
Instance	-	-	-	-
Memory	-	-	-	-
Multiplexer	-	-	-	36
Register	-	-	83	-
Total	0	2	83	69
Available	280	220	106400	53200
Utilization (%)	0	~0	~0	~0

Slika 7.45. Performanse RTL modela koji koristi protočnu obradu unutar petlje k

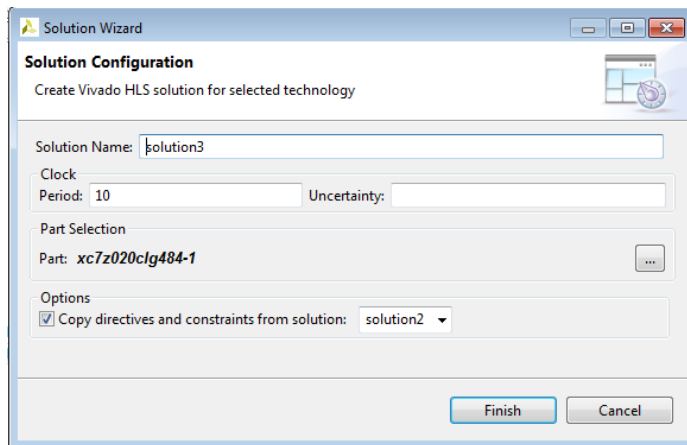
Pokušajmo dalje da optimizujemo dizajn. Sledeća optimizacija koju ćemo iskoristiti jeste razmatranje petlje k . Kao što je ranije već bilo rečeno, razmatranjem petlje potencijalno se povećava broj operacija koje se mogu izvršiti konkurentno, što bi trebalo da rezultuje u daljem skraćanju vremena potrebnog za množenje matrica.

Da bismo kasnije mogli da analiziramo pojedinačne efekte svake od primenjenih optimizacija, pre nego što definišemo direktivu koja će razmotati petlju k , kreiraćemo novu implementaciju, *solution3*. Pritiskom na „New Solution“ dugme, kao što je

objašnjeno u poglavlju 7.5, pojavljuje se „*Solution Configuration*“ prozor, prikazan na slici 7.46.

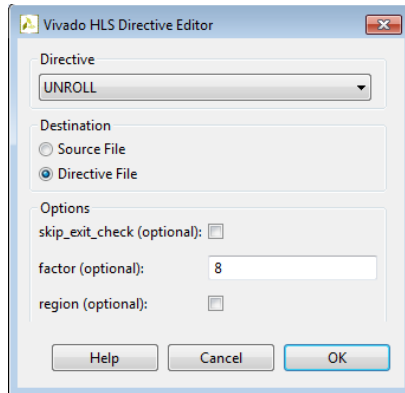
Kako želimo da zadržimo optimizacione direktive koje smo definisali u okviru *solution2* implementacije i samo ih proširimo sa dodatnim, „*Copy directives and constraints from solution*“ opcija treba da bude aktivirana, kao što je prikazano na slici 7.46.

Pritiskom na dugme *OK*, kreira se nova implemetacija, nazvana *solution3*, i Vivado HLS se vraća u osnovni prozor.



Slika 7.46. „*Solution Configuration*“ prozor sa potrebnim podešavanjima za kreiranje *solution3* implementacije

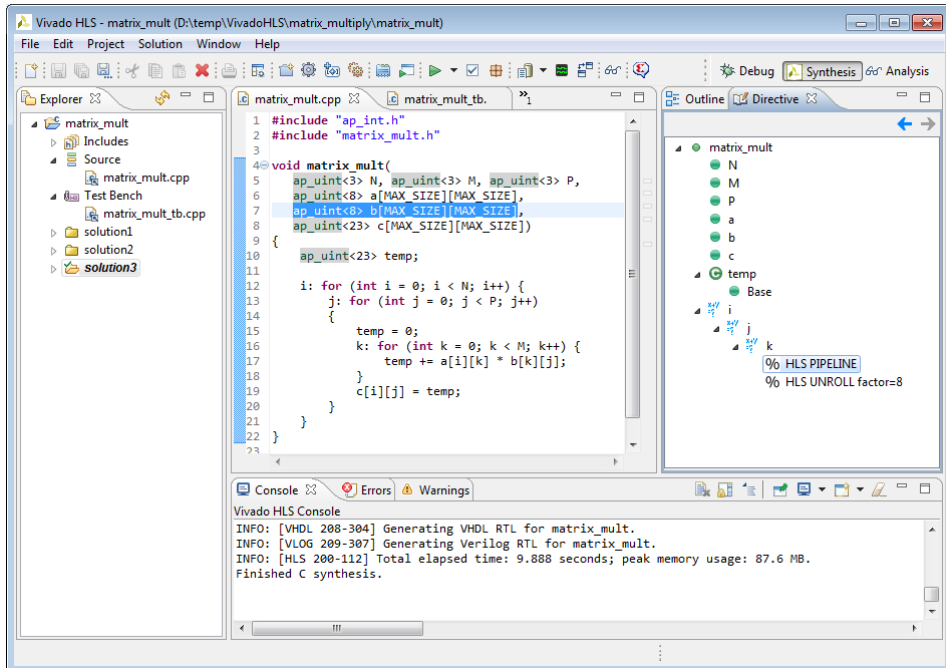
Kako bismo razmotali petlju *k*, potrebno je na nju primeniti UNROLL optimizacionu direktivu. Desnim klikom na petlju *k* u okviru *Directive* prozora i izborom opcije „*Insert Directive...*“ ponovo se otvara „*Directive Editor*“ prozor, prikazan na slici 7.47.



Slika 7.47. „Directive Editor“ prozor sa odabranom UNROLL optimizacionom direktivom

Ovaj put želimo da koristimo UNROLL optimizacionu direktivu, pa je treba izabrati iz liste ponuđenih direktiva. Kada izaberemo UNROLL direktivu menja se sadržaj *Options* polja, koje sada prikazuje parametre koje možemo podesiti. Parametar *factor* omogućava nam da definišemo faktor razmotavanja petlje. Ukoliko se ovo polje ostavi praznim petlja će se razmotati u potpunosti. Međutim, kada petlja ima promenljivi broj iteracija, ona se ne može razmotati u potpunosti jer je u ovom slučaju faktor razmotavanja takođe varijabilan. Zbog toga je potrebno navesti faktor sa kojim želimo da razmotamo petlju. Kako smo prilikom formiranja algoritamskog modela definisali ograničenje u maksimalnom broju elemenata duž bilo koje dimenzije matrica na osam, definišaćemo da se petlja k razmoti sa faktorom 8, kao što je prikazano na slici 7.47.

Pritiskom na dugme OK Vivado HLS alat kreira novu optimizacionu direktivu na petlji k i vraća se u osnovni prozor. U *Directive* prozoru sada se mogu videti dve optimizacione direktive definisane za petlju k , PIPELINE direktiva koja je bila definisana u okviru *solution2* implementacije ali smo je preneli i u *solution3* implementaciju, i nova UNROLL direktiva sa faktorom razmotavanja petlje 8, kao što se može videti na slici 7.48.



Slika 7.48. Izgled glavnog prozora Vivado HLS alata nakon definisanja UNROLL direktive na petlji k

Da bismo videli efekte razmatranja petlje `k`, potrebno je da sintetizujemo novi RTL model, pritiskom na „C Synthesis“ dugme. Kada Vivado HLS alat završi sintezu novog RTL modela automatski prikazuje izveštaj o sintezi, čiji deo sa performansama je prikazan na slici 7.49.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	11.34	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
1	589	2	590	none

Detail

Instance

Loop

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	7	-	-
Expression	-	2	0	543
FIFO	-	-	-	-
Instance	-	-	-	-
Memory	-	-	-	-
Multiplexer	-	-	-	172
Register	-	-	272	-
Total	0	9	272	715
Available	280	220	106400	53200
Utilization (%)	0	4	~0	1

Slika 7.49. Performanse RTL modela koji koristi protočnu obradu unutar petlje k i razmotava petlju k sa faktorom 8

Analizom performansi novog RTL modela možemo videti da su se one pogoršale. Za kompletiranje procesa množenja dve matrice u najgorem slučaju korišćenjem *solution3* implementacije potrebno je 589 perioda klok signala, što predstavlja povećanje od oko 10% u odnosu na performanse *solution2* implementacije (540 perioda klok signala). Takođe, sa slike 7.49 možemo videti da se značajno povećao i broj potrebnih hardverskih resursa, konkretno DSP blokova, sa 2 na 9. Ovaj porast je razumljiv, jer smo razmotali petlju k sa faktorom 8, pa sada imamo mogućnost da izvršimo 8 operacija množenja elemenata konkurentno. Međutim, ono što nam informacije o kašnjenju govore jeste da razmatavanje petlje nije imalo željeni efekat na smanjenje vremena potrebnog za množenje matrica, iako se sada operacije množenja izvode u paraleli. Kao što je bilo napomenuto prilikom diskutovanja optimizacije tehnike razmatavanja petlje (u poglavlju 7.6.3), razlog za ovo može da se krije u nemogućnosti interfejsa ka memorijskom podsistemu da obezbedi potrebnu količinu podataka.

Inspekcijom dela izveštaja posvećenom interfejsu kreiranog RTL modela, možemo proveriti način na koji je kreiran interfejs za matrice A i B u *solution3* implementaciji. Informacije o kreiranim interfejsima prikazane su na slici 7.50.

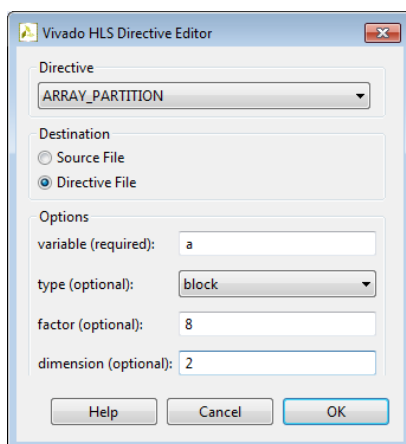
Interface					
Summary					
RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	matrix_mult	return value
ap_rst	in	1	ap_ctrl_hs	matrix_mult	return value
ap_start	in	1	ap_ctrl_hs	matrix_mult	return value
ap_done	out	1	ap_ctrl_hs	matrix_mult	return value
ap_idle	out	1	ap_ctrl_hs	matrix_mult	return value
ap_ready	out	1	ap_ctrl_hs	matrix_mult	return value
N_V	in	3	ap_none	N_V	scalar
M_V	in	3	ap_none	M_V	scalar
P_V	in	3	ap_none	P_V	scalar
a_V_address0	out	6	ap_memory	a_V	array
a_V_ce0	out	1	ap_memory	a_V	array
a_V_q0	in	8	ap_memory	a_V	array
a_V_address1	out	6	ap_memory	a_V	array
a_V_ce1	out	1	ap_memory	a_V	array
a_V_q1	in	8	ap_memory	a_V	array
b_V_address0	out	6	ap_memory	b_V	array
b_V_ce0	out	1	ap_memory	b_V	array
b_V_q0	in	8	ap_memory	b_V	array
b_V_address1	out	6	ap_memory	b_V	array
b_V_ce1	out	1	ap_memory	b_V	array
b_V_q1	in	8	ap_memory	b_V	array
c_V_address0	out	6	ap_memory	c_V	array
c_V_ce0	out	1	ap_memory	c_V	array
c_V_we0	out	1	ap_memory	c_V	array
c_V_d0	out	23	ap_memory	c_V	array

Slika 7.50. Informacije o kreiranom interfejsu RTL modela koji koristi protočnu obradu unutar petlje k i razmatra petlju k sa faktorom 8

Sa slike 7.50 možemo videti da se interfejs ka memoriji u kojoj se nalaze elementi matrice A sastoji iz dva kanala (pristupa), a_V_q0 i a_V_q1 . Ovo je bilo očekivano, jer je podrazumevani način za kreiranje interfejsa ka memorijama u kojima se nalaze nizovi koji predstavljaju argumente algoritamskog modela, interfejs ka dvoprístupnim memorijama (jer u većini FPGA komponenti postoje BlockRAM memorije koje imaju po dva pristupa). Ovo je upravo ono što je Vivado HLS alat i uradio za matrice A i B . Međutim to znači da se u svakom taktu iz memorija gde su smešteni elementi matrica mogu preneti vrednosti samo dva elementa, a mi imamo mogućnost množenja ukupno osam elemenata u svakom taktu. Očigledno je da trenutni interfejs ka memorijskom podsisistemu nema dovoljni propusni opseg, i predstavlja usko grlo zbog kojega se efekti razmatavanja petlje k ne vide u poboljšanju performansi sistema. Jasno je da, ukoliko želimo da u potpunosti iskoristimo efekte razmatavanja petlje k , moramo redizajnirati interfejs ka memorijama u kojima su smeštene matrice A i B . U poglavlju 7.6.3 prikazan je način na koji je ovo moguće izvesti, korišćenjem optimizacione tehnike podele nizova (*Array Partitioning*).

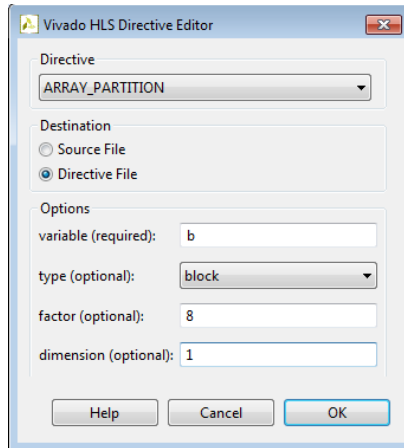
Kako smo petlju k razmotali sa faktorom 8, potrebno je i matrice A i B podeliti na 8 blokova, duž odgovarajućih dimenzija. Za ovo se može iskoristiti `ARRAY_PARTITION` direktiva.

Podelimo prvo matricu A na osam blokova, duž njene druge dimenzije (jer se analizom algoritamskog modela može videti da se iterator petlje k koristi za indeksiranje druge dimenzije matrice A). Izborom argumenta a iz *Directive* prozora, desnim klikom na njega i izborom „Insert Directive...” akcije ponovo se otvara otvara „Directive Editor“ prozor, prikazan na slici 7.51. Potrebna podešavanja za podelu matrice A treba da odgovaraju onima sa slike 7.51.



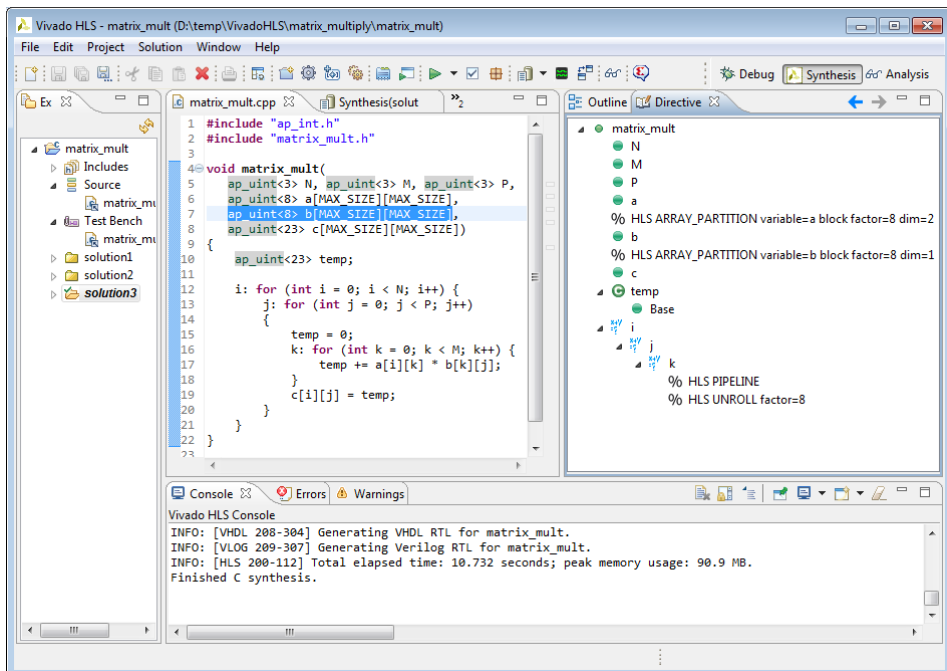
Slika 7.51. „Directive Editor“ prozor sa odabranom `ARRAY_PARTITION` optimizacionom direktivom i potrebnim podešavanjima u slučaju podele matrice A

Slično se vrši i podela matrice B na osam blokova, ali ovaj put duž njene prve dimenzije (jer se analizom algoritamskog modela može videti da se iterator petlje k koristi za indeksiranje prve dimenzije matrice B). Izborom argumenta b iz *Directive* prozora, desnim klikom na njega i izborom „Insert Directive...” akcije ponovo se otvara otvara „Directive Editor“ prozor, prikazan na slici 7.52. Potrebna podešavanja za podelu matrice B treba da odgovaraju onima sa slike 7.52.



Slika 7.52. „Directive Editor“ prozor sa odabranom `ARRAY_PARTITION` optimizacionom direktivom i potrebnim podešavanjima u slučaju podele matrice *B*

Nakon što smo definisali ove dve optimizacione direktive, izgled osnovnog prozora Vivado HLS alata trebalo bi da odgovara onom prikazanom na slici 7.53.



Slika 7.53. Izgled glavnog prozora Vivado HLS alata nakon definisanja `ARRAY_PARTITION` direktiva na ulaznim argumentima *a* i *b*

Nakon završetka sinteze novog RTL modela, sa particionisanim matricama A i B , Vivado HLS generiše izveštaj čiji deo posvećen performansama je prikazan na slici 7.54.

Performance Estimates

Timing (ns)

Summary

Clock	Target	Estimated	Uncertainty
ap_clk	10.00	15.14	1.25

Latency (clock cycles)

Summary

Latency		Interval		
min	max	min	max	Type
1	344	2	345	none

Detail

Instance

Loop

Utilization Estimates

Summary

Name	BRAM_18K	DSP48E	FF	LUT
DSP	-	-	-	-
Expression	-	9	0	648
FIFO	-	-	-	-
Instance	-	-	-	-
Memory	-	-	-	-
Multiplexer	-	-	-	138
Register	-	-	412	-
Total	0	9	412	786
Available	280	220	106400	53200
Utilization (%)	0	4	~0	1

Slika 7.54. Performanse RTL modela koji koristi protočnu obradu unutar petlje k , razmotava petlju k sa faktorom 8, i particioniše matrice A i B

Analizom performansi ovog RTL modela možemo videti da je došlo do značajnog poboljšanja performansi, što znači da je optimizacija načina na koji je realizovan interfejs ka memorijama u kojima su smeštene matrice A i B dala rezultate. Ova verzija *solution3* implementacije množi dve matrice za 344 periode klok signala, što je sada značajno bolje od *solution2* implementacije (kojoj treba 540 perioda klok signala).

Inspekcijom sekcije izveštaja o sintezi u vezi sa generisanim interfejsom sada se može uočiti da argumenti a i b imaju po 8 kanala preko kojih se prenose vrednosti elemenata matrica A i B , kao što se može videti na slici 7.55 za interfejs ulaznog argumenta a .

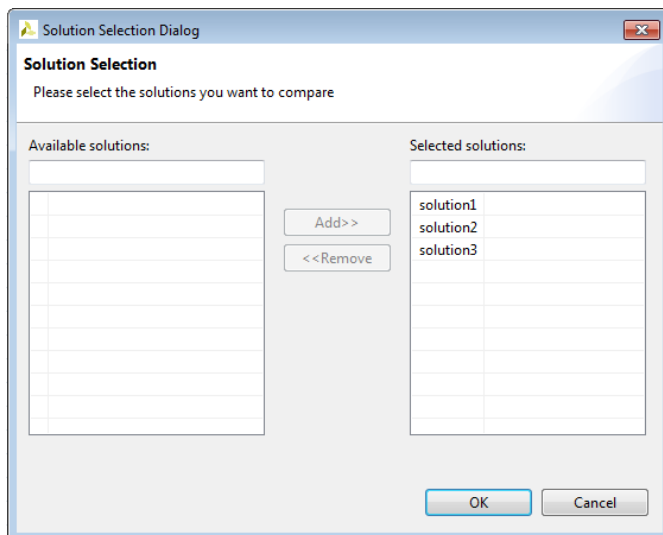
Interface					
Summary					
RTL Ports	Dir	Bits	Protocol	Source Object	C Type
ap_clk	in	1	ap_ctrl_hs	matrix_mult	return value
ap_rst	in	1	ap_ctrl_hs	matrix_mult	return value
ap_start	in	1	ap_ctrl_hs	matrix_mult	return value
ap_done	out	1	ap_ctrl_hs	matrix_mult	return value
ap_idle	out	1	ap_ctrl_hs	matrix_mult	return value
ap_ready	out	1	ap_ctrl_hs	matrix_mult	return value
N_V	in	3	ap_none	N_V	scalar
M_V	in	3	ap_none	M_V	scalar
P_V	in	3	ap_none	P_V	scalar
a_0_V_address0	out	3	ap_memory	a_0_V	array
a_0_V_ce0	out	1	ap_memory	a_0_V	array
a_0_V_q0	in	8	ap_memory	a_0_V	array
a_1_V_address0	out	3	ap_memory	a_1_V	array
a_1_V_ce0	out	1	ap_memory	a_1_V	array
a_1_V_q0	in	8	ap_memory	a_1_V	array
a_2_V_address0	out	3	ap_memory	a_2_V	array
a_2_V_ce0	out	1	ap_memory	a_2_V	array
a_2_V_q0	in	8	ap_memory	a_2_V	array
a_3_V_address0	out	3	ap_memory	a_3_V	array
a_3_V_ce0	out	1	ap_memory	a_3_V	array
a_3_V_q0	in	8	ap_memory	a_3_V	array
a_4_V_address0	out	3	ap_memory	a_4_V	array
a_4_V_ce0	out	1	ap_memory	a_4_V	array
a_4_V_q0	in	8	ap_memory	a_4_V	array
a_5_V_address0	out	3	ap_memory	a_5_V	array
a_5_V_ce0	out	1	ap_memory	a_5_V	array
a_5_V_q0	in	8	ap_memory	a_5_V	array
a_6_V_address0	out	3	ap_memory	a_6_V	array
a_6_V_ce0	out	1	ap_memory	a_6_V	array
a_6_V_q0	in	8	ap_memory	a_6_V	array
a_7_V_address0	out	3	ap_memory	a_7_V	array
a_7_V_ce0	out	1	ap_memory	a_7_V	array
a_7_V_q0	in	8	ap_memory	a_7_V	array
b_0_V_address0	out	3	ap_memory	b_0_V	array

Slika 7.55. Informacije o kreiranom interfejsu RTL modela za ulazni argument a koji koristi protočnu obradu unutar petlje k , razmotava petlju k sa faktorom 8 i particioniše matrice A i B

Ovim će proces dalje optimizacije RTL implementacije „Naive“ množača matrica biti završen.

Kako smo u ovom primeru kreirali ukupno tri posebne implementacije, *solution1*, *solution2* i *solution3*, Vivado HLS alat ima mogućnost da generiše uporedni izveštaj koji će obuhvatati sve tri implementacije radi lakše analize.

Pritiskom na dugme „Compare Reports...“, , pojavljuje se „Solution Selection“ prozor, prikazan na slici 7.56.



Slika 7.56. „Solution Selection“ prozor sa izabranim implementacijama za koje želimo da se generiše uporedni izveštaj

Izborom sve tri implementacije, kao na slici 7.56, i pritiskom na dugme *OK*, Vivado HLS generiše uporedni izveštaj i otvara ga u okviru osnovnog prozora. U našem primeru uporedni izveštaj trebalo bi da ima izgled kao na slici 7.57.

U okviru uporednog izveštaja vide se podaci o:

- Estimiranoj minimalnoj periodi klok signala za koju generisana RTL implementacija radi pouzdano
- Kašnjenju i inicijalizacionom intervalu
- Potrebnim hardverskim resursima za realizaciju

Vivado HLS Report Comparison

All Compared Solutions

[solution1](#): xc7z020clg484-1

[solution2](#): xc7z020clg484-1

[solution3](#): xc7z020clg484-1

Performance Estimates

Timing (ns)

Clock		solution1	solution2	solution3
ap_clk	Target	10.00	10.00	10.00
	Estimated	6.38	6.38	15.14

Latency (clock cycles)

		solution1	solution2	solution3
Latency	min	1	1	1
	max	1142	540	344
Interval	min	2	2	2
	max	1143	541	345

Utilization Estimates

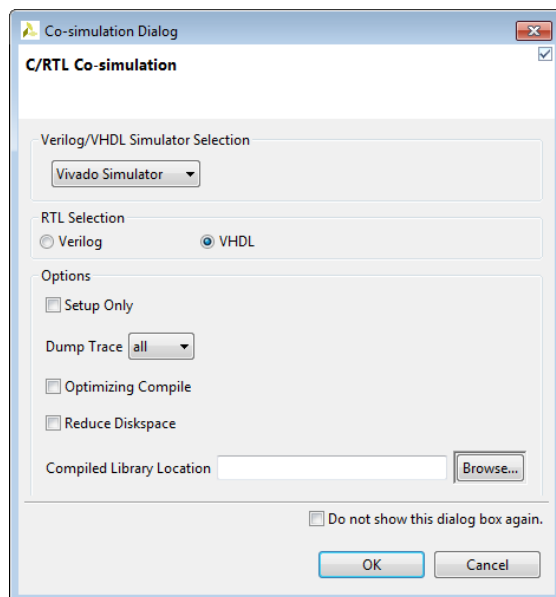
	solution1	solution2	solution3
BRAM_18K	0	0	0
DSP48E	1	2	9
FF	65	83	412
LUT	44	69	786

Slika 7.57. Uporedni izveštaj sa glavnim karakteristikama tri generisane implementacije

7.7. Verifikacija generisanog RTL modela

Nakon što smo zadovoljni sa performansama generisanog RTL modela, možemo izvršiti njegovu simulaciju kako bi verifikovali da je željena funkcionalnost, specificirana u algoritamskom modelu, korektno implementirana i unutar RTL modela. Vivado HLS alat pruža i ovu mogućnost, generišući sve potrebne fajlove za RTL simulaciju pomoću odgovarajućeg HDL simulatora. Ono što je bitno naglasiti je da se kao testbenč za RTL simulaciju može koristiti postojeći C testbenč koji je korišćen za verifikaciju algoritamskog modela, što predstavlja veliku uštedu u vremenu. Kako se i prilikom simulacije RTL modela koristi C testbenč, ova vrsta simulacije unutar Vivado HLS alata se zove C/RTL kosimulacija (*C/RTL Cosimulation*).

Simulacija RTL modela pokreće se pritiskom na dugme „Run C/RTL Cosimulation“, ☒, nakon čega se pojavljuje „C/RTL Co-simulation“ prozor, prikazan na slici 7.58.



Slika 7.58. „C/RTL Co-simulation“ prozor

U okviru „C/RTL Co-simulation“ prozora mogu se podesiti parametri predstojeće HDL simulacije. U okviru polja „Verilog/VHDL Simulator Selection“ može se odabrati HDL simulator koji želimo da koristimo. U okviru „RTL Selection“ polja može se izabrati jezik u kojem će biti generisano HDL verifikaciono okruženje.

U okviru *Options* polja nalaze se sledeće opcije:

- „Setup Only“ – aktiviranjem ove opcije Vivado HLS alat kreira sve neophodne fajlove za RTL simulaciju, ali ne izvršava samu simulaciju.
- „Dump Trace“ – ova opcija kontroliše način na koji će biti generisan *trace* fajl, koji sadrži talasne oblike RTL signala za detaljnu inspekciju od strane dizajnera. Moguće je sačuvati talasne oblike svih signala unutar dizajna, samo signale koji predstavljaju interfejs RTL modula ili niti jedan talasni oblik.
- „Optimizing Compile“ – ako je aktivna prilikom kompajliranja C testbenča koristiće se visok nivo optimizacije. Na ovaj način može se skratiti vreme potrebno za završetak RTL simulacije, ali se gubi mogućnost interaktivnog debagovanja C testbenča.
- „Reduce Disk Space“ – kako se u procesu RTL simulacije vrši kosimulacija C modela i HDL modela, mora postojati način kako da se vrednosti signala prenose iz jednog domena u drugi. Vivado HLS za ovu potrebu koristi fajlove koje smešta na disk. Kako ovi fajlovi mogu biti ogromni, pomoću ove opcije se proces kosimulacije vrši u koracima od po jedne transakcije, što će

redukovati potrebni prostor na disku. Međutim, kosimulacija će u ovom slučaju biti usporena.

- „*Compiled Library Location*“ – pomoću ove opcije specificira se lokacija na kojoj se nalaze simulacione biblioteke u slučaju korišćenja RTL simulatora razvijenog od strane drugih kompanija.

Potrebna podešavanja za C/RTL kosimulaciju u našem slučaju treba da izgledaju kao na slici 7.58. Pritiskom na dugme *OK* započinje proces kosimulacije. Tok kosimulacije može se pratiti unutar „*Vivado HLS Console*“ prozora. Obratite pažnju da će se u procesu kosimulacije ispisati sve poruke koje su definisane unutar C testbenča, kao i u slučaju C simulacije algoritamskog modela, tako da će se dobiti izveštaj o rezultatima kosimulacije i da li je ona prošla onako kako je očekivano.

Kada se kosimulacija završi, pritiskom na dugme „*Open Wave Viewer...*“, , može se pokrenuti Vivado Wave Viewer alat pomoću kojega je moguće pregledati talasne oblike signala koji predstavljaju rezultat kosimulacije. Ukoliko je C testbenč pravilno napisan (organizovan kao samoproveravajući testbenč) u principu nema potrebe za vizuelnom inspekcijom talasnih oblika RTL signala, jer će informacije o uspešno ili neuspešno završenom procesu funkcionalne verifikacije RTL modela već biti dostupne, u okviru „*Vivado HLS Console*“ prozora, kao što je već rečeno u prethodnom paragrafu.

U našem primeru talasni oblici svih interfejs signala optimizovanog RTL modela (*solution3* implementacija) „*Naive*“ množača matrica prikazani su na slici 7.59.

Na slici 7.59 mogu se videti signali koji čine interfejs RTL modela „*Naive*“ množača matrica. Na vrhu se nalaze *ap_clk* i *ap_rst* signali. Zatim slede signali koji čine odabrani blok interfejs. U našem slučaju korišćen je standardni *ap_ctrl_hs* blok interfejs koji se sastoji od četiri signala: *ap_start*, *ap_ready*, *ap_done* i *ap_idle*.

Nakon signala koji čine blok interfejs dolazi grupa signala i protokola pomoću kojih su implementirani svi ulazni argumenti algoritamskog modela. Sa slike 7.59 izostavljeni su signali koji čine interfejs za ulazne argumente *N*, *M* i *P*. Nakon njih slede signali koji čine interfejs ulaznih argumenata *a* i *b*. Kako je reč o RTL modelu *solution3* implementacije, ulazni nizovi *a* i *b* su particionisani u 8 blokova. Za svaki od ovih blokova Vivado HLS alat je generisao po jedan *ap_memory* memorijski interfejs, što se lepo može videti na slici 7.59.

Na samom dnu se nalaze signali koji čine interfejs za izlazne argumente algoritamskom modela. U našem primeru to su signali još jednog *ap_memory* memorijskog interfejsa, preko koga se pristupa memoriji u kojoj se nalaze smešteni elementi matrice *C*.

Inspekcijom talasnih oblika ovih signala može se dobiti bolje razumevanje kako Vivado HLS koristi različite interfejse za prenos podaka od i ka RTL modulu koji implementira „*Naive*“ algoritam množenja matrica. Takođe se inspekcijom validnih vrednosti koje se pojavljuju na izlaznom portu *c_V_d0[22:0]* može proveriti korektan rad RTL modela. Na slici 7.59 vidljiva je sledeća sekvenca vrednosti, 84, 90, 96, 201, 216, 231, 318, 342, 366, koja tačno odgovara očekivanim vrednostima elemenata matrice *C*.



Slika 7.59. Rezultat simulacije solution3 RTL implementacije „Naive“ množača matrica

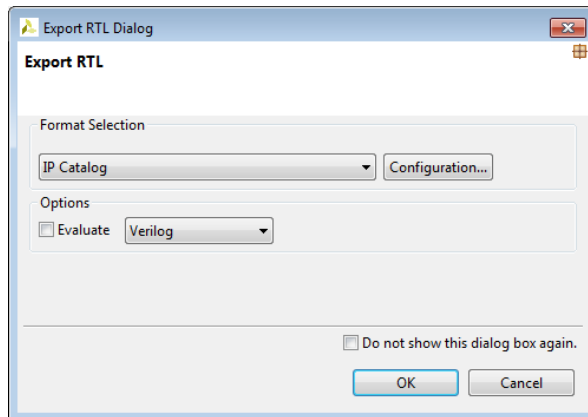
7.8. Eksportovanje generisanog RTL modela

Nakon uspešne sinteze RTL modela za željenim karakteristikama u pogledu performansi i potrebnih hardverskih resursa za njegovu implementaciju, i nakon uspešno završene C/RTL kosimulacije, poslednji korak u razvoju digitalnog sistema pomoću Vivado HLS alata predstavlja korak eksportovanja RTL modela u jednom od podržanih formata, kako bi se on mogao koristiti unutar drugih Xilinx alata:

- IP jezgro zapakovano u formatu koji omogućava da se jezgro uključi u IP Catalog Vivado Design Suite alata – IP jezgro zapakovano u *zip* arhivu koja se zatim može dodati u IP katalog i koristi kao i svako drugo Xilinx IP jezgro.
- IP jezgro zapakovano u formatu koji omogućava da se koristi unutar Xilinx System Generator for DSP alata – na ovaj način se hardverska implementacije odabranog algoritma može koristiti unutar MATLAB Simulink alata.
- RTL model eksportovan u formi *synthesized checkpoint*-a – ovaj checkpoint se može uključiti u svaki Vivado Design Suite projekat u formi sintetizovane netliste.

Pored zapakovanih dizajn fajlova, svi dizajn fajlovi koji čine generisani RTL model algoritamskog modela su na raspolaganju u odgovarajućim *verilog* ili *vhdl* direktorijuma koji se nalaze unutar implementacijskog direktorijuma *<ime projekta>/<ime implementacije>/impl*, trenutno aktuelne implementacije. Pored dizajn fajlova, u okviru ovih direktorijuma nalaze se i projektni fajlovi za Vivado Design Suite alat. Startovanjem *project.xpr* projektnog fajla, odgovarajući projekat (Verilog ili VHDL) se otvara unutar Vivado Design Suite alata, gde se dizajn dalje može analizirati.

Pritiskom na „Export RTL“ dugme, , ili izborom *Solution→Export RTL* opcije iz menija, pojavljuje se „Export RTL“ prozor prikazan na slici 7.60, u okviru kojega se mogu podesiti parametri vezani za eksportovanje dizajna.



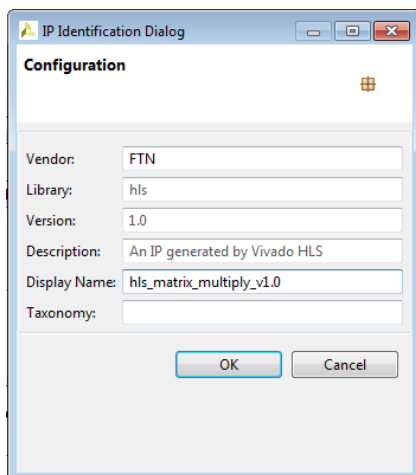
Slika 7.60. „Export RTL“ prozor

U okviru „Format Selection“ opcije, moguće je odabrati jedan od tri raspoloživa formata („IP Catalog“, „System Generator for DSP“ i „Synthesized Checkpoint“) u kojem će biti eksportovan RTL model. U zavisnosti od odabranog formata pritiskom na dugme „Configuration...“ moguće je podesiti dodatne parametre, vezane za odabrani format.

„Evaluate“ opcija može se iskoristiti za pokretanje RTL sinteze kako bi se pre samog pakovanja proverile performanse generisanog RTL modela, konkretno minimalna perioda klok signala na kojoj RTL model radi pouzdano. Kao što je ranije već bilo rečeno, Vivado HLS alat prilikom sinteze visokog nivoa algoritamskog modela daje samo estimaciju minimalne periode klok signala na kojoj će generisani RTL model raditi pouzdano. Tačna vrednost minimalne periode klok signala biće poznata tek nakon RTL sinteze i implementacije. Opcijom „Evaluate“ može se pokrenuti Vivado XST kompajler i dobiti tačnija estimacija.

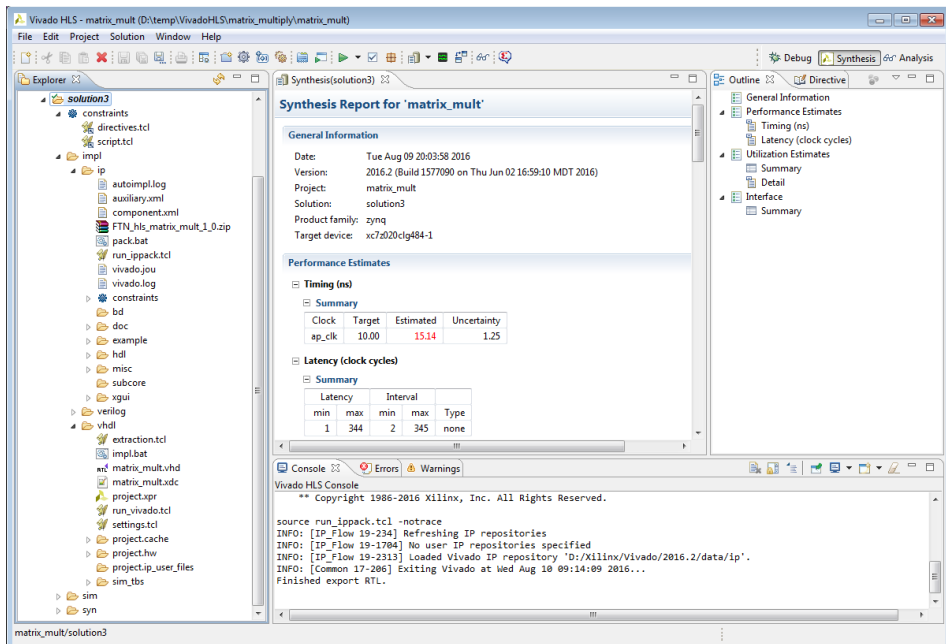
U našem primeru mi ćemo eksportovati generisani RTL model „Naive“ množača matrica u formatu „IP Catalog“ kompatibilnog IP jezgra, pa podešavanja treba da odgovaraju onim sa slike 7.60. Pre nego što eksportujemo RTL dizajn, podesićemo dodatne opcije „IP Catalog“ formata, pritiskom na dugme „Configuration...“, nakon čega se pojavljuje *Configuration* prozor prikazan na slici 7.61.

Polja *Configuration* prozora slična su odgovarajućim poljima koja su bila diskutovana prilikom rada sa Vivado IP Packager alatom, u poglavlju 6.3, pa ovde neće biti detaljno razmatrana. U našem primeru vrednosti polja treba da odgovaraju onima sa slike 7.61.



Slika 7.61. „Configuration“ prozor, u slučaju kada se RTL model eksportuje u IP Catalog formatu

Pritiskom na dugme *OK* vraćamo se u „Export RTL“ prozor. Pritiskom na *OK* dugme u okviru „Export RTL“ prozor Vivado HLS alat eksportuje RTL model u željenom formatu i vraća se u osnovni prozor. Nakon završenog procesa eksportovanja, u okviru trenutno aktivne implementacije pojavljuje se dodatni direktorijum, *impl*, koji sadrži sve RTL dizajn fajlove, kao i zapakovani RTL dizajn u odabranom formatu, kao što se može videti na slici 7.62, u slučaju eksportovanja RTL modela „Naive“ množača matrica.



Slika 7.62. Struktura impl direktorijuma nakon završenog eksportovanja RTL modela

U našem primeru eksportovali smo RTL model generisan u okviru *solution3* implementacije. Sa slike 7.62 može se videti da se u okviru *solution3* implementacije pojavio *impl* direktorijum, kao i njegova unutrašnja organizacija. U okviru *ip* direktorijuma nalaze se fajlovi generisani prilikom eksportovanja RTL modela „Naive“ množača matrica u „IP Catalog“ formatu. Ključni fajl u ovom direktorijumu je *FTN_hls_matrix_multi_1_0.zip*, koji zapravo predstavlja zip arhivu zapakovanog RTL modela množača, spremnu za uključivanje u IP Catalog Vivado Design Suite alata.

Pored *ip* direktorijuma, možemo videti da je Vivado HLS alat kreirao i *verilog* i *vhdl* direktorijume. U ovim direktorijumima nalaze se Verilog, odnosno VHDL, RTL modeli „Naive“ množača matrica, zajedno sa Vivado Design Suite projektnim fajlovima (*project.xpr*), koji omogućavaju njihovo lako pokretanje u okviru Vivado Design Suite alata.

Ovim je proces projektovanja „Naive“ množača matrica pomoću Vivado HLS alata završen.

Zadaci za vežbu

Zadatak 7.1.

Korišćenjem Vivado HLS alata projektovati hardverski modul koji realizuje *Doolittle*-ov algoritam LU dekompozicije kvadratne matrice A dimenzija $n \times n$. LU dekompozicija kvadratne matrice A , dekomponuje matricu A na gornju trougaonu matricu U i donju trougaonu matricu L , tako da važi sledeće

$$A = L \cdot U$$

$$A = \begin{bmatrix} a_{1,1} & \dots & a_{1,n} \\ \dots & \dots & \dots \\ a_{n,1} & \dots & a_{n,n} \end{bmatrix}, L = \begin{bmatrix} 1 & 0 & 0 \\ l_{k,1} & 1 & 0 \\ l_{n,1} & \dots & 1 \end{bmatrix}, U = \begin{bmatrix} u_{1,1} & \dots & u_{1,n} \\ 0 & u_{k,k} & u_{k,n} \\ 0 & \dots & u_{n,n} \end{bmatrix}$$

Doolittle-ov algoritam LU dekompozicije kvadratne matrice A prikazan je u nastavku

Za svako $k = 1, 2, \dots, n$:

$$u_{k,m} = a_{k,m} - \sum_{j=1}^{k-1} l_{k,j} \cdot u_{j,m}, \text{ za } m = k, k+1, \dots, n$$

$$l_{i,k} = \frac{a_{i,k} - \sum_{j=1}^{k-1} l_{i,j} \cdot u_{j,k}}{u_{k,k}}, \text{ za } i = k+1, k+2, \dots, n; l_{k,k} = 1$$

- Napisati algoritamski model *Doolittle*-og algoritma LU dekompozicije kvadratne matrice A u C programskom jeziku i izvršiti njegovu funkcionalnu verifikaciju pomoću C simulacije.
- Izvršiti sintezu visokog nivoa razvijenog algoritamskog modela pomoću Vivado HLS alata.
- Izvršiti potrebne optimizacije pomoću kojih se može ubrzati rad digitalnog sistema koji implementira *Doolittle*-ov algoritam LU dekompozicije.
- Eksportovati optimalni VHDL model u formi IP jezgra.

Zadatak 7.2.

LU dekompozicija kvadratne matrice A dimenzija $n \times n$ može se iskoristiti za rešavanje sistema linearnih jednačina sa n nepoznatih. Pretpostavimo da je sistem linearnih jednačina zadat u matričnoj formi na sledeći način

$$A \cdot x = b$$

$$A = \begin{bmatrix} a_{1,1} & \dots & a_{1,n} \\ \dots & \dots & \dots \\ a_{n,1} & \dots & a_{n,n} \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ \dots \\ x_n \end{bmatrix}, \quad b = \begin{bmatrix} b_1 \\ \dots \\ b_n \end{bmatrix}$$

Pretpostavimo da nam je na raspolaganju LU dekompozicija matrice $A=L \cdot U$. Tada sistem linearnih jednačina ima sledeći oblik

$$L \cdot U \cdot x = b$$

U ovom slučaju do rešenja sistema linearnih jednačina možemo doći u dva koraka:

1. Rešavajući prvi pomoćni sistem linearnih jednačina po novom vektoru nepoznatih, y ,

$$L \cdot y = b$$

2. Rešavajući drugi pomoćni sistem linearnih jednačina po originalnom vektoru nepoznatih, x ,

$$U \cdot x = y$$

Obratite pažnju da se prethodna dva pomoćna sistema trivijalno rešavaju, jer su matrice L i U , donja i gornja traougaona matrica. Oba sistema se rešavaju primenom direktne i obrnute supstitucije.

- a) Napisati algoritamski model sistema za rešavanja sistema linearnih jednačina baziran na LU dekompoziciji matrice A .
- b) Izvršiti sintezu visokog nivoa razvijenog algoritamskog modela pomoću Vivado HLS alata.
- c) Izvršiti potrebne optimizacije pomoću kojih se može ubrzati rad digitalnog sistema koji implementira algoritam za rešavanje sistema linearnih jednačina baziran na LU dekompoziciji matrice A .
- d) Eksportovati optimalni VHDL model u formi IP jezgra.

Zadatak 7.3.

Sistem linearnih jednačina može se rešiti i primenom dobro poznatog Gausovog algoritma eliminacije

- Napisati algoritamski model sistema za rešavanja sistema linearnih jednačina baziran na Gausovom algoritmu eliminacije.
- Izvršiti sintezu visokog nivoa razvijenog algoritamskog modela pomoću Vivado HLS alata.
- Izvršiti potrebne optimizacije pomoću kojih se može ubrzati rad digitalnog sistema koji implementira algoritam za rešavanje sistema linearnih jednačina baziran na Gausovom algoritmu eliminacije.
- Eksportovati optimalni VHDL model u formi IP jezgra.

Zadatak 7.4.

Vrlo česta transformacija koja se koristi u gotovo svim savremenim sistemima za kompresiju slike i video signala je dvodimenzionalna diskretna kosinusna transformacija (*2D Discrete Cosine Transform, 2D-DCT*). U praksi se najčešće sreće slučaj računanja 2D-DCT transformacije na bloku od 8x8 piksela. U ovom slučaju, direktna 2D-DCT transformacija definisana je sledećim izrazom

$$F(u, v) = \frac{1}{4} C(u) \cdot C(v) \left[\sum_{x=0}^7 \sum_{y=0}^7 f(x, y) \cdot \cos \frac{(2x+1) \cdot u \cdot \pi}{16} \cdot \cos \frac{(2y+1) \cdot v \cdot \pi}{16} \right]$$

$$\text{gde su: } C(u), C(v) = \frac{1}{\sqrt{2}}, \quad \text{za } u, v = 0; \quad 0 \leq u \leq 7, \quad 0 \leq v \leq 7$$

$$C(u), C(v) = 1, \quad \text{za } u, v \neq 0;$$

$$f(x, y) - \text{vrednost piksela na koordinatama } (x, y)$$

$$F(u, v) - \text{vrednost DCT koeficijenta na koordinatama } (u, v)$$

- Napisati algoritamski model sistema za računanje 2D-DCT transformacije na bloku od 8x8 piksela.
- Izvršiti sintezu visokog nivoa razvijenog algoritamskog modela pomoću Vivado HLS alata.
- Izvršiti potrebne optimizacije pomoću kojih se može ubrzati rad digitalnog sistema za računanje 2D-DCT transformacije na bloku od 8x8 piksela.
- Eksportovati optimalni VHDL model u formi IP jezgra.

Zadatak 7.5.

Dvodimenzionalna diskretna kosinusna transformacija, definisana u zadatku 7.4 koristi se prilikom kompresije slike. U procesu dekompresije slike neophodno je koristiti inverznu dvodimenzionalnu diskretnu kosinusnu transformaciju (*2D Inverse Discrete*

Cosine Transform, 2D-IDCT), kako bi se rekonstruisale originalne vrednosti piksela slike, definisanu sledećim izrazom

$$f(x, y) = \frac{1}{4} \left[\sum_{u=0}^7 \sum_{v=0}^7 C(u) \cdot C(v) \cdot F(u, v) \cdot \cos \frac{(2x+1) \cdot u \cdot \pi}{16} \cdot \cos \frac{(2y+1) \cdot v \cdot \pi}{16} \right]$$

$$\text{gde su: } C(u), C(v) = \frac{1}{\sqrt{2}}, \quad \text{za } u, v = 0; \quad 0 \leq u \leq 7, \quad 0 \leq v \leq 7$$

$$C(u), C(v) = 1, \quad \text{za } u, v \neq 0;$$

- a) Napisati algoritamski model sistema za računanje 2D-IDCT transformacije na bloku od 8x8 piksela.
- b) Izvršiti sintezu visokog nivoa razvijenog algoritamskog modela pomoću Vivado HLS alata.
- c) Izvršiti potrebne optimizacije pomoću kojih se može ubrzati rad digitalnog sistema za računanje 2D-IDCT transformacije na bloku od 8x8 piksela.
- d) Eksportovati optimalni VHDL model u formi IP jezgra.

